



UNIVERSITAS INDONESIA

**PENGEMBANGAN *BACKEND* APLIKASI ANOTASI *DATASET*
GRAMMATICAL ERROR CORRECTION BERBASIS WEB
DENGAN METODOLOGI *AGILE***

SKRIPSI

**JUAN DHARMANANDA KHUSUMA / 2206081521 / ILMU
KOMPUTER
LIM BODHI WIJAYA / 2206082410 / SISTEM INFORMASI**

**FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JANUARI 2026**



UNIVERSITAS INDONESIA

**PENGEMBANGAN *BACKEND* APLIKASI ANOTASI *DATASET*
GRAMMATICAL ERROR CORRECTION BERBASIS WEB
DENGAN METODOLOGI *AGILE***

SKRIPSI

Diajukan sebagai salah satu syarat untuk memperoleh gelar
Sarjana Ilmu Komputer

**JUAN DHARMANANDA KHUSUMA / 2206081521 / ILMU
KOMPUTER
LIM BODHI WIJAYA / 2206082410 / SISTEM INFORMASI**

**FAKULTAS ILMU KOMPUTER
PROGRAM STUDI ILMU KOMPUTER
DEPOK
JANUARI 2026**

HALAMAN PERNYATAAN ORISINALITAS

**Skripsi ini adalah hasil karya kami sendiri,
dan semua sumber baik yang dikutip maupun dirujuk
telah kami nyatakan dengan benar.**

Nama Penulis 1 : Juan Dharmananda Khusuma

NPM Penulis 1 : 2206081521

Tanda Tangan Penulis 1 : 

Nama Penulis 2 : Lim Bodhi Wijaya

NPM Penulis 2 : 2206082410

Tanda Tangan Penulis 2 : 

Tanggal : 5 Januari 2026

HALAMAN PENGESAHAN

Tugas Akhir ini diajukan oleh :

Nama/NPM/Prodi Mahasiswa : Juan Dharmananda Khusuma/2206081521/Sarjana Ilmu
1 Komputer

Nama/NPM/Prodi Mahasiswa : Lim Bodhi Wijaya/2206082410/Sarjana Sistem
2 Informasi

Judul : Pengembangan Backend Aplikasi Anotasi Dataset
Grammatical Error Correction Berbasis Web dengan
Metodologi Agile

Telah berhasil dipertahankan di hadapan Dewan Penguji dan diterima sebagai bagian persyaratan yang diperlukan untuk memperoleh gelar Sarjana Ilmu Komputer pada Program Studi Sarjana Ilmu Komputer/Sarjana Sistem Informasi, Fakultas Ilmu Komputer, Universitas Indonesia

DEWAN PENGUJI

Pembimbing 1 : Dr. Ika Alfina, S.Kom., M.Kom.

(Nilai telah diberikan melalui
SISIDANG pada 22-12-2025,
12:00:57)
(Revisi telah disetujui melalui
SISIDANG pada 05-01-2026,
13:07:07)

Penguji : Dr. Baginda Anggun Nan Cenka,
S.Kom., M.Kom.

(Nilai telah diberikan melalui
SISIDANG pada 22-12-2025,
11:17:25)
(Revisi telah disetujui melalui
SISIDANG pada 31-12-2025,
19:18:01)

Penguji : Arawinda Dinakaramani, S.Kom.,
M.Hum.

(Nilai telah diberikan melalui
SISIDANG pada 02-01-2026,
14:06:40)
(Revisi telah disetujui melalui
SISIDANG pada 02-01-2026,
14:07:20)

Ditetapkan di : Depok, Jawa Barat

Tanggal : 05 Januari 2026

KATA PENGANTAR

Puji syukur kehadirat Tuhan YME atas segala rahmat dan karunia-Nya, sehingga tim penulis dapat menyelesaikan skripsi yang berjudul Pengembangan *Backend* Aplikasi Anotasi Dataset *Grammatical Error Correction* Berbasis Web dengan Metodologi *Agile*. Penulisan skripsi ini dilakukan dalam rangka memenuhi salah satu syarat untuk mencapai gelar Sarjana Ilmu Komputer pada Fakultas Ilmu Komputer, Universitas Indonesia.

Tim penulis menyadari bahwa penyelesaian skripsi ini tidak terlepas dari bantuan, bimbingan, dan dukungan dari berbagai pihak. Dengan demikian, kami dari tim penulis ingin mengucapkan terima kasih yang sebesar-besarnya kepada:

1. Dr. Ika Alfina, S.Kom., M.Kom., selaku dosen pembimbing yang telah meluangkan waktu, tenaga, dan pikiran untuk memberikan bimbingan, arahan, serta diskusi yang mendalam selama pengerjaan skripsi ini.
2. Alden Luthfi dan Samuel Taniel Mulyadi, selaku tim pengembang *frontend*, terima kasih atas kerja sama dan juga dukungannya selama penyusunan proyek tugas akhir ini.
3. Pihak-pihak lain yang tidak dapat penulis sebutkan satu per satu, yang telah memberikan bantuan dalam penyusunan skripsi ini.

Secara khusus, Juan Dharmananda Khusuma ingin mengucapkan terima kasih kepada:

1. Ibu tercinta, yang senantiasa tiada henti memberikan cinta kasih, dukungan moral, material, dan doa, yang telah memberikan kekuatan kepada penulis, sehingga penulis dapat tetap semangat dalam menyelesaikan pendidikan ini.
2. Dr.rer.nat. Fariz Darari, S.Kom., M.Sc., selaku pembimbing akademik yang senantiasa membantu agar penulis dapat menjalani perkuliahannya dengan lancar.
3. Seluruh tim dosen dan anggota Laboratorium Reliable Software Engineering (RSE) Fasilkom UI, yang telah menyediakan lingkungan penelitian yang kondusif dan atas wawasan yang telah diberikan kepada tim penulis selama penulisan skripsi ini.
4. Wida Putri Kinasih, terima kasih atas persahabatan dan dukungan yang diberikan dalam masa-masa sulit hidup, perkuliahan, dan penyusunan skripsi ini.
5. Sahabat-sahabat Salam Justisia, yaitu Arvin, Ester, Vene, Lisa, Gilang, Vanya, Fajar, Maurice, Ella, WS, dan Edbert yang senantiasa memberikan dukungan, semangat, dan

pengalaman berharga yang tidak tergantikan selama masa perkuliahan.

6. Sahabat-sahabat dari KMB, yaitu Hendersen, Steffi, Yolanda, Takeshi, CJ, Ellen, Jessica, Viola, Debby, dan teman-teman lainnya yang tidak bisa saya sebutkan satu per satu. Terima kasih atas dukungan, semangat, dan pengalaman berharga yang tidak tergantikan selama masa perkuliahan.

Secara khusus, Lim Bodhi Wijaya ingin mengucapkan terima kasih kepada:

1. Keluarga tercinta, khususnya Papa, Mama, dan Cici yang selalu memberikan cinta kasih, dukungan moral, materil, dan doa selama perkuliahan dari awal semester hingga saat ini.
2. Imairi Eitiveni, S.Kom., M.Kom., Ph.D. selaku pembimbing akademik yang senantiasa membantu agar penulis dapat menjalani perkuliahan dengan lancar.
3. Seluruh teman-teman grup Baris Depan, KMB, TASLA, PPI dan teman-teman lainnya yang tidak bisa saya sebutkan satu per satu. Terima kasih atas dukungan, semangat, dan pengalaman berharga yang tidak tergantikan selama masa perkuliahan.

Penulis menyadari bahwa laporan skripsi ini masih jauh dari sempurna. Oleh karena itu, apabila terdapat kesalahan atau kekurangan dalam laporan ini, Penulis memohon agar kritik dan saran bisa disampaikan langsung melalui *e-mail* juan.dharmananda@ui.ac.id dan lim.bodhi@ui.ac.id.

Depok, 5 Januari 2026



Juan Dharmananda Khusuma



Lim Bodhi Wijaya

HALAMAN PERNYATAAN PERSETUJUAN PUBLIKASI TUGAS AKHIR UNTUK KEPENTINGAN AKADEMIS

Sebagai sivitas akademik Universitas Indonesia, kami yang bertanda tangan di bawah ini:

Penulis 1

Nama : Juan Dharmananda Khusuma
NPM : 2206081521
Program Studi : Ilmu Komputer

Penulis 2

Nama : Lim Bodhi Wijaya
NPM : 2206082410
Program Studi : Sistem Informasi
Jenis Karya : Skripsi

demikian pengembangan ilmu pengetahuan, menyetujui untuk memberikan kepada Universitas Indonesia **Hak Bebas Royalti Noneksklusif** (*Non-exclusive Royalty Free Right*) atas karya ilmiah kami yang berjudul:

Pengembangan *Backend* Aplikasi Anotasi *Dataset Grammatical Error Correction*
Berbasis Web dengan Metodologi *Agile*

beserta perangkat yang ada (jika diperlukan). Dengan Hak Bebas Royalti Noneksklusif ini Universitas Indonesia berhak menyimpan, mengalihmedia/formatkan, mengelola dalam bentuk pangkalan data (*database*), merawat, dan memublikasikan tugas akhir kami selama tetap mencantumkan nama kami sebagai penulis/pencipta dan sebagai pemilik Hak Cipta.

Demikian pernyataan ini kami buat dengan sebenarnya.

Dibuat di : Depok

Pada tanggal : 5 Januari 2026

Yang menyatakan

Penulis 1



(Juan Dharmananda Khusuma)

Penulis 2



(Lim Bodhi Wijaya)

ABSTRAK

Nama Penulis 1 / Program Studi : Juan Dharmananda Khusuma / Ilmu Komputer
Nama Penulis 2 / Program Studi : Lim Bodhi Wijaya / Sistem Informasi
Judul : Pengembangan *Backend* Aplikasi Anotasi *Dataset Grammatical Error Correction* Berbasis Web dengan Metodologi *Agile*
Pembimbing : Dr. Ika Alfina, S.Kom., M.Kom.

Pengembangan model *Natural Language Processing* (NLP) untuk *Grammatical Error Correction* (GEC) memerlukan *dataset* anotasi manual yang berkualitas tinggi, namun proses tersebut sulit dilakukan tanpa bantuan sistem. Penelitian ini mengembangkan ANOTA, *platform* anotasi berbasis web yang dirancang untuk memfasilitasi pembuatan *dataset* GEC secara terstruktur dan kolaboratif. Penelitian ini secara spesifik membahas pengembangan *backend* dari sistem ANOTA yang dilakukan menggunakan metodologi *Agile* dengan pendekatan *Kanban* untuk memastikan proses yang iteratif, fleksibel, dan berorientasi pada *continuous delivery*. Tahapan penelitian mencakup analisis kebutuhan, perancangan arsitektur *backend*, perancangan basis data, implementasi modul inti seperti autentikasi dan otorisasi berbasis *Role-Based Access Control* (RBAC), manajemen proyek, manajemen dokumen, pemrosesan anotasi dan *review*, *dashboard*, dan penyediaan API yang *RESTful* untuk mendukung *frontend*. Selain itu, *backend* dikonfigurasi dalam lingkungan terkontainerisasi menggunakan *Docker* dan ditambahkan *Prometheus* dan *Grafana* untuk pemantauan performa sistem. Pengujian melalui *unit testing* telah mencapai 100% *coverage* pada seluruh *business logic* utama, *load testing* menunjukkan sistem bekerja dengan baik ketika diakses oleh 50 pengguna secara bersamaan (median respons 63 ms, *error rate* 0%), serta tetap berfungsi hingga 150 pengguna. Proses *User Acceptance Test* (UAT) dilakukan secara khusus pada sisi *backend* dengan menggunakan emulasi klien berupa *Postman collection*. Seluruh hasil evaluasi menunjukkan kebutuhan sistem telah dipenuhi dan *backend* siap untuk diintegrasikan dengan *frontend*.

Kata kunci:

Anotasi teks, *Grammatical Error Correction* (GEC), sistem *backend*, aplikasi berbasis web, *Natural Language Processing* (NLP)

ABSTRACT

Writer 1 / Study Program : Juan Dharmananda Khusuma / Computer Science
Writer 2 / Study Program : Lim Bodhi Wijaya / Information System
Title : Development of a Web-Based Backend Application for Grammat-
ical Error Correction Dataset Annotation Using Agile Methodol-
ogy
Counselor : Dr. Ika Alfina, S.Kom., M.Kom.

The development of a Natural Language Processing (NLP) model for Grammatical Error Correction (GEC) requires a high-quality manually annotated dataset, but this process is difficult to do without the help of a system. This study develops ANOTA, a web-based annotation platform designed to facilitate the creation of structured and collaborative GEC datasets. This research specifically discusses the development of the ANOTA system backend using the Agile methodology with a Kanban approach to ensure an iterative, flexible, and continuous delivery-oriented process. The research stages included requirements analysis, backend architecture design, database design, implementation of core modules such as Role-Based Access Control (RBAC) based authentication and authorization, project management, document management, annotation and review processing, dashboard, and provision of a RESTful API to support the frontend. In addition, the backend is configured in a containerized environment using Docker and Prometheus and Grafana are added for system performance monitoring. Unit testing has achieved 100% coverage across all key business logic, and load testing shows that the system works well when accessed by 50 users simultaneously (median response 63 ms, error rate 0%) and continues to function with up to 150 users. User Acceptance Testing (UAT) was conducted exclusively on the backend using client emulation through a Postman collection. All evaluation results indicate that system requirements have been fulfilled and that the backend is ready to be integrated with the frontend.

Key words:

Text annotation, Grammatical Error Correction (GEC), backend system, web-based application, Natural Language Processing (NLP)

DAFTAR ISI

HALAMAN JUDUL	i
HALAMAN ORISINALITAS	ii
LEMBAR PENGESAHAN	iii
KATA PENGANTAR	iii
LEMBAR PERSETUJUAN KARYA ILMIAH	vi
ABSTRAK	viii
DAFTAR ISI	x
DAFTAR GAMBAR	xiii
DAFTAR TABEL	xv
DAFTAR KODE PROGRAM	xv
DAFTAR LAMPIRAN	xvi
1. PENDAHULUAN	2
1.1 Latar Belakang	2
1.2 Rumusan Masalah	3
1.3 Batasan Penelitian	4
1.4 Tujuan Penelitian	4
1.5 Manfaat Penelitian	5
1.6 Langkah Penelitian	5
1.7 Posisi Penelitian	6
1.8 Sistematika Penulisan	6
2. STUDI LITERATUR	8
2.1 <i>Grammatical Error Correction</i> (GEC)	8
2.1.1 Definisi <i>Grammatical Error Correction</i> dan Pentingnya <i>Dataset</i> Beranotasi	8
2.1.2 Dataset Sintetis	9
2.1.3 Metode Anotasi <i>Dataset</i> GEC	9
2.1.4 <i>Inter-Annotator Agreement</i>	10
2.1.5 Format <i>Dataset</i> GEC	11
2.2 Konsep Pengembangan Perangkat Lunak	13
2.2.1 Kanban	13
2.2.2 Agile	14
2.3 Testing dan Observability	15
2.3.1 Unit Testing	15
2.3.2 Load Testing	15
2.3.3 User Acceptance Testing	16
2.3.4 Prometheus dan Grafana	17
3. METODE PENELITIAN	19
3.1 Sistem ANOTA	19
3.2 Process Model	19
3.3 Tahapan Penelitian	21
3.3.1 Tahap Analisis	22
3.3.2 Tahap Perancangan	22

3.3.3	Tahap Implementasi	23
3.3.4	Tahap Evaluasi	23
4.	ANALISIS DAN PERANCANGAN	25
4.1	Analisis Gap Solusi	25
4.2	Kebutuhan Fungsional	25
4.2.1	Autentikasi dan Otorisasi Pengguna	26
4.2.2	Manajemen Akun	28
4.2.3	Manajemen Proyek Anotasi	29
4.2.4	Manajemen Dokumen	31
4.2.5	Anotasi Dokumen dan Review Anotasi	32
4.2.6	<i>Dashboard</i> Statistik	35
4.3	Kebutuhan Non-Fungsional	37
4.4	Analisis dan Perancangan Basis Data	37
4.4.1	Entity Relationship Diagram	38
4.4.2	Skema Relasional	40
4.5	Rancangan Modul-Modul Anotasi	42
4.5.1	Modul <i>User</i>	43
4.5.2	Modul <i>Project</i>	45
4.5.3	Modul <i>Document</i>	45
4.5.4	Modul <i>Annotation</i>	48
4.5.5	Modul <i>Review</i>	49
4.5.6	Modul <i>Dashboard</i>	52
4.6	Rancangan <i>Application Programming Interface</i> (API)	53
5.	IMPLEMENTASI	55
5.1	Pemilihan Teknologi	55
5.2	Arsitektur Sistem	58
5.3	Implementasi Logika Bisnis Inti Sistem	59
5.3.1	Perhitungan Skor <i>Inter-Annotator Agreement</i> (IAA)	59
5.3.2	Role-Based Access Control (RBAC)	60
5.3.3	Tokenisasi Kalimat secara Otomatis	62
5.3.4	<i>Document Deduplication</i> dengan <i>Hashing</i>	62
5.4	Konfigurasi <i>Deployment</i>	63
5.4.1	Layanan Inti Aplikasi	65
5.4.2	Layanan Penyimpanan Data dan Konektivitas	66
5.4.3	Layanan Monitoring	67
5.4.4	Volume dan Network	68
6.	EVALUASI	69
6.1	<i>Unit Test</i>	69
6.1.1	Cakupan Pengujian	69
6.1.2	Lingkungan Pengujian	71
6.1.3	Metode Pengujian	72
6.1.4	Hasil Pengujian	72
6.1.5	Kesimpulan <i>Unit Test</i>	73
6.2	<i>Load Test</i>	73

6.2.1	Lingkungan Uji Coba	73
6.2.2	Skenario Uji Coba	74
6.2.2.1	Skenario <i>Annotator</i>	75
6.2.2.2	Skenario <i>Reviewer</i>	76
6.2.2.3	Skenario <i>Admin</i> dan Kepala Riset	77
6.2.3	Hasil Uji Coba	78
6.2.3.1	Ringkasan Hasil Pengujian	79
6.2.3.2	Hasil Pengujian pada Skenario Normal Load	80
6.2.3.3	Hasil Pengujian pada Skenario Peak Load	81
6.2.3.4	Perbandingan Kinerja Antar Skenario	82
6.2.3.5	Analisis Konsumsi Sumber Daya Server	83
6.2.3.6	Analisis Bottleneck dan Temuan Teknis	84
6.2.3.7	Kesimpulan Evaluasi Performa	86
6.3	<i>User Acceptance Test (UAT)</i>	86
6.3.1	Tujuan UAT	87
6.3.2	Metodologi UAT	87
6.3.3	Skenario Pengujian	88
6.3.3.1	Skenario <i>Annotator</i>	88
6.3.3.2	Skenario <i>Reviewer</i>	88
6.3.3.3	Skenario <i>Admin</i>	89
6.3.3.4	Skenario Kepala Riset	89
6.3.4	Instrumen dan Kriteria Evaluasi	90
6.3.5	Hasil UAT	90
6.3.6	Pembahasan	91
6.3.7	Kesimpulan UAT	91
7.	PENUTUP	93
7.1	Kesimpulan	93
7.2	Saran	94
	DAFTAR REFERENSI	96

DAFTAR GAMBAR

Gambar 2.1.	Contoh format pasangan kalimat paralel pada korpus GEC	11
Gambar 2.2.	Contoh format M2 untuk satu kalimat pada korpus GEC	12
Gambar 3.1.	Contoh Kanban Board ANOTA	20
Gambar 3.2.	<i>Agile Methodology</i>	21
Gambar 4.1.	Use Case Diagram Modul Autentikasi	27
Gambar 4.2.	Use Case Diagram Modul Manajemen Akun	28
Gambar 4.3.	Use Case Diagram Modul Manajemen Proyek	30
Gambar 4.4.	Use Case Diagram Modul Manajemen Dokumen	31
Gambar 4.5.	Use Case Diagram Modul Anotasi Dokumen	33
Gambar 4.6.	Use Case Diagram Modul Dashboard	36
Gambar 4.7.	<i>Entity Relationship Diagram</i> Anota	38
Gambar 4.8.	Skema Relasional Anota	41
Gambar 4.9.	<i>Class Diagram</i> Modul <i>User</i>	44
Gambar 4.10.	<i>Class Diagram</i> Modul <i>Project</i>	46
Gambar 4.11.	<i>Class Diagram</i> Modul <i>Document</i>	47
Gambar 4.12.	<i>Class Diagram</i> Modul <i>Annotation</i>	49
Gambar 4.13.	<i>Class Diagram</i> Modul <i>Review</i>	51
Gambar 4.14.	<i>Class Diagram</i> Modul <i>Dashboard</i>	52
Gambar 5.1.	Arsitektur dari Django Rest Framework	58
Gambar 5.2.	Mekanisme Pengecekan <i>Role</i>	61
Gambar 5.3.	<i>Container Layer</i> dari Diagram C4 Arsitektur <i>Deployment Backend</i>	64
Gambar 6.1.	Skenario Uji Coba untuk <i>Annotator</i>	75
Gambar 6.2.	Skenario Uji Coba untuk <i>Reviewer</i>	76
Gambar 6.3.	Skenario Uji Coba untuk <i>Admin</i> dan Kepala Riset	78
Gambar 6.4.	Hasil <i>Load Test</i> pada Skenario <i>Normal Load</i>	81
Gambar 6.5.	Hasil <i>Load Test</i> pada Skenario <i>Peak Load</i>	82
Gambar 6.6.	Persentase Penggunaan CPU pada Skenario <i>Normal Load</i>	84
Gambar 6.7.	Persentase Penggunaan RAM pada Skenario <i>Normal Load</i>	84
Gambar 6.8.	Persentase Penggunaan CPU pada Skenario <i>Peak Load</i>	85
Gambar 6.9.	Persentase Penggunaan RAM pada Skenario <i>Peak Load</i>	85
Gambar 1.	Alur Autentikasi <i>Administrator</i>	162
Gambar 2.	Alur Manajemen <i>Project Administrator</i> Bagian 1	163
Gambar 3.	Alur Manajemen <i>Project Administrator</i> Bagian 2	164
Gambar 4.	Alur Manajemen <i>User Administrator</i> Bagian 1	165
Gambar 5.	Alur Manajemen <i>User Administrator</i> Bagian 2	166
Gambar 6.	Alur Manajemen Dokumen <i>Administrator</i>	167
Gambar 7.	Flow Export Data Administrator	168
Gambar 8.	Alur <i>Dashboard Administrator</i>	169
Gambar 9.	Alur Buka Kembali Dokumen	170
Gambar 10.	Alur Kelola Jenis <i>Error</i> Bagian 1	171

Gambar 11.	Alur Kelola Jenis <i>Error</i> Bagian 2	172
Gambar 12.	Alur	173
Gambar 13.	Alur Autentikasi <i>Editor</i> Bagian 2	174
Gambar 14.	Alur Penugasan Dokumen Bagian 1	175
Gambar 15.	Alur Penugasan Dokumen Bagian 2	176
Gambar 16.	Alur <i>Review</i> Anotasi Bagian 1	177
Gambar 17.	Alur <i>Review</i> dan Anotasi Bagian 2	178
Gambar 18.	Alur <i>Review</i> Anotasi Bagian 3	179
Gambar 19.	Alur <i>Submit</i> Dokumen	180
Gambar 20.	Alur Buka Kembali Dokumen	181
Gambar 21.	Alur Ubah Profil	182

DAFTAR TABEL

Tabel 4.1.	Ringkasan perbandingan kondisi <i>As-Is</i> dan <i>To-Be</i> pada sistem ANOTA	25
Tabel 4.2.	Mapping entity dan atribut pada basis data Anota	40
Tabel 6.1.	Spesifikasi VM untuk <i>Load Test</i>	73
Tabel 6.2.	Skenario Normal Load	74
Tabel 6.3.	Skenario Peak Load	74
Tabel 6.4.	<i>Request Statistics</i> untuk Skenario <i>Normal Load</i>	79
Tabel 6.5.	<i>Response Statistics</i> (in milliseconds) untuk Skenario <i>Normal Load</i> . .	79
Tabel 6.6.	<i>Request Statistics</i> untuk Skenario <i>Peak Load</i>	80
Tabel 6.7.	<i>Response Statistics</i> (in milliseconds) untuk Skenario <i>Peak Load</i>	80
Tabel 6.8.	Perbandingan Metrik Performa antara <i>Normal Load</i> dan <i>Peak Load</i> . .	83
Tabel 6.9.	Ringkasan Hasil UAT <i>Backend</i>	91
Tabel 2.	Pembagian Tugas Pengembangan Aplikasi	100
Tabel 4.	Pembagian Tugas Menulis Laporan TA	101
Tabel 5.	List <i>endpoint</i> ANOTA	110

DAFTAR KODE PROGRAM

Kode 4.1.	Login Endpoint Complete Request	53
Kode 5.1.	Implementasi Perhitungan Cohen's Kappa	60
Kode 5.2.	Metode Tokenisasi Kalimat dengan NLTK dan Fallback RegEx	62
Kode 5.3.	Contoh Implementasi Deduplication via Hashing	63
Kode 5.4.	Dockerfile untuk Layanan Aplikasi Anota	65
Kode 5.5.	Layanan Inti Aplikasi	65
Kode 5.6.	Layanan Database, Connection Pooling, dan Redis	66
Kode 5.7.	Layanan Monitoring: Prometheus, Node Exporter, Grafana	67
Kode 5.8.	Volume dan Network	68

DAFTAR LAMPIRAN

Lampiran 1.	Pembagian Tugas Pengembangan Aplikasi	100
Lampiran 2.	Pembagian Tugas Menulis Laporan TA	101
Lampiran 3.	List <i>endpoint</i> ANOTA	102
Lampiran 4.	List User Acceptance Test Backlog	111
Lampiran 5.	Konfigurasi JSON untuk Tes UAT pada Postman <i>Collection</i>	160
Lampiran 6.	<i>User Flow</i>	161

BAB 1

PENDAHULUAN

Pada bab ini, akan dijelaskan mengenai latar belakang, rumusan masalah, batasan penelitian, tujuan penelitian, manfaat penelitian, langkah penelitian, posisi penelitian, serta sistematika penulisan.

1.1 Latar Belakang

Grammatical Error Correction (GEC) adalah salah satu bentuk permasalahan pada bidang *Natural Language Processing* (NLP) yang bertujuan untuk memperbaiki ejaan, tata bahasa, dan kesalahan penulisan pada suatu teks (Musyafa et al., 2024). Dalam pengembangannya, GEC sangat bergantung pada ketersediaan *training data* berkualitas tinggi dalam jumlah yang besar (Bryant et al., 2023). Untuk sebagian bahasa asing seperti bahasa Inggris, kebutuhan data tersebut telah terpenuhi oleh berbagai korpus besar yang dianotasi secara manual, seperti korpus NUCLE oleh Dahlmeier et al. (2013), CoNLL-2013 oleh Ng et al. (2013), dan CoNLL-2014 oleh Ng et al. (2014).

Upaya untuk membangun *dataset* GEC berskala besar dalam bahasa Indonesia juga telah dilakukan melalui berbagai penelitian, seperti Marier et al. (2025), Lin et al. (2024), Musyafa et al. (2024), dan (Tadjudin, 2023). Namun, seluruh *dataset* yang dihasilkan pada penelitian tersebut masih bersifat sintetik, baik melalui penambahan *noise*, pembuatan kesalahan secara terprogram, maupun pemanfaatan *language model* untuk menghasilkan koreksi otomatis. Pendekatan tersebut memiliki keterbatasan karena pola kesalahan tata bahasa manusia sangat sulit direplikasi secara akurat oleh proses sintetik (Bryant et al., 2023), sehingga kualitas data yang dihasilkan bergantung pada model dan aturan yang digunakan.

Hingga saat ini, bahasa Indonesia belum memiliki *dataset* GEC berukuran besar yang dianotasi secara manual oleh ahli. Hal ini disebabkan oleh beberapa tantangan, seperti:

1. Anotator harus membedakan berbagai jenis kesalahan tata bahasa. Penelitian sebelumnya mencatat setidaknya 25 kategori (Tadjudin, 2023). Jika dilakukan tanpa bantuan alat, banyaknya kategori tersebut akan menambah beban kognitif karena anotator harus terus mengingat daftar kategori, kode kesalahannya, serta menjaga konsistensi penen-

tuan label di setiap anotasi.

2. Untuk keperluan pelatihan model, hasil anotasi harus dikonversi ke format yang dapat diproses mesin, seperti format *parrarel* dan M2 (Tadjudin, 2023). Konversi ini sulit dilakukan secara manual sehingga memerlukan proses otomasi.
3. Proses anotasi melibatkan banyak pihak, seperti anotator, *reviewer*, dan peneliti. Sehingga diperlukan sebuah *platform* terintegrasi untuk menjaga efisiensi, konsistensi, dan kualitas hasil anotasi.

Dengan demikian, untuk mengatasi tantangan tersebut, diperlukan sebuah *platform* anotasi *dataset* GEC berbasis web yang dikhususkan untuk bahasa Indonesia.

Dalam upaya memastikan efektivitas dari *platform* anotasi, proses pengembangan perlu bersifat fleksibel karena kebutuhan sistem belum sepenuhnya terdefinisi sejak awal. Oleh karena itu, penelitian ini menerapkan prinsip-prinsip *Agile Software Development*, yang mendukung iterasi singkat, sehingga mampu beradaptasi terhadap perubahan kebutuhan (Beck et al., 2001; Dingsøyr et al., 2012). Secara khusus, kerangka Kanban digunakan untuk memvisualisasikan alur kerja, membatasi pekerjaan yang sedang berlangsung, dan menjaga proses pengembangan tetap terstruktur (Ahmad et al., 2013; Ikonen et al., 2011). Dengan demikian, pengembangan *platform* anotasi GEC dapat dilakukan secara lebih terarah dan responsif terhadap kebutuhan riset.

1.2 Rumusan Masalah

Pada bagian ini, akan dijelaskan permasalahan-permasalahan yang telah diidentifikasi dari latar belakang sebelumnya. Rumusan masalah dalam penelitian ini adalah sebagai berikut

1. Bagaimana merancang suatu sistem yang dapat mendukung proses anotasi dataset GEC bahasa Indonesia secara terstruktur, efisien, dan sesuai dengan kebutuhan berbagai pemangku kepentingan?
2. Bagaimana mengimplementasikan rancangan sistem tersebut menjadi sebuah *platform* anotasi berbasis web yang mampu membantu anotator, *reviewer*, dan peneliti dalam menjalankan proses anotasi secara konsisten?
3. Bagaimana mengevaluasi sistem yang dikembangkan untuk memastikan bahwa *platform* anotasi tersebut telah memenuhi seluruh kebutuhan fungsional dan non-fungsional dari pengguna?

1.3 Batasan Penelitian

Pengembangan aplikasi ANOTA dilakukan oleh dua tim yang bekerja secara paralel, tim pertama menangani pengembangan *frontend* dan tim kedua, yaitu tim penulis berfokus pada pengembangan *backend*. Oleh karena itu, ruang lingkup penelitian ini dibatasi agar selaras dengan pembagian tugas tersebut. Adapun batasan penelitian ini adalah sebagai berikut:

1. Penelitian ini hanya berfokus pada perancangan dan implementasi *backend* aplikasi ANOTA, sementara seluruh pengembangan *frontend* dikerjakan oleh tim lain dan berada di luar cakupan penelitian.
2. Penelitian tidak melakukan proses *requirement gathering* dan juga *requirement validation*. Seluruh kebutuhan sistem telah ditentukan sebelumnya oleh tim *frontend*, sehingga implementasi *backend* sepenuhnya mengikuti *requirements* yang sudah diberikan.
3. Penelitian tidak melakukan proses *benchmarking*. Proses tersebut telah menjadi bagian dari *requirements gathering* oleh tim *frontend*.
4. Penelitian ini tidak mencakup pembuatan, pelatihan, maupun evaluasi model GEC. Fokus penelitian sepenuhnya berada pada pengembangan sistem untuk mendukung proses anotasi manual dalam pembangunan *dataset* GEC bahasa Indonesia.
5. Pengujian pada penelitian ini terbatas pada pengujian sisi *backend*, yang mencakup *unit test* untuk memverifikasi fungsi-fungsi internal, *load test* untuk mengukur kinerja API bila diakses secara bersamaan, dan *User Acceptance Testing* (UAT) yang dilakukan melalui emulasi klien menggunakan skenario Postman, sehingga tidak mencakup pengujian *frontend* maupun pengalaman pengguna akhir.

1.4 Tujuan Penelitian

Berdasarkan rumusan masalah yang telah dijelaskan, tujuan dari penelitian ini adalah sebagai berikut:

1. Merancang sebuah sistem yang mampu mendukung proses anotasi *dataset* GEC bahasa Indonesia.
2. Mengimplementasikan rancangan penelitian menjadi sebuah *platform* anotasi berbasis web yang dapat membantu anotator, *reviewer*, dan peneliti dalam melakukan proses

anotasi.

3. Melakukan evaluasi terhadap sistem yang dikembangkan untuk memastikan fungsionalitas, kegunaan, serta kesesuaiannya dalam mendukung proses pembangunan *dataset* GEC bahasa Indonesia.

1.5 Manfaat Penelitian

Berikut ini adalah merupakan manfaat dari penelitian yang dilakukan:

1. Penelitian ini berperan dalam menyediakan infrastruktur *backend* yang memfasilitasi proses anotasi dalam pembuatan korpus *Grammatical Error Correction* (GEC) Bahasa Indonesia berkualitas tinggi (*gold-standard*) secara lebih efisien. Hal ini diharapkan dapat mengatasi kelangkaan *dataset* pelatihan yang selama ini menjadi penghambat utama perkembangan teknologi GEC di Indonesia.
2. Sistem ini mempermudah kinerja *annotator* dan *reviewer* melalui otomatisasi alur kerja, seperti tokenisasi kalimat otomatis dan manajemen distribusi dokumen. Dengan adanya sistem terintegrasi, proses anotasi manual yang sebelumnya memakan waktu dan rentan kesalahan dapat dilakukan dengan lebih konsisten dan terstruktur. Selain itu, sistem ini memungkinkan pemrosesan anotasi kedalam format dataset standar secara otomatis, sehingga mengurangi beban kerja dan menurunkan potensi kesalahan translasi format.

1.6 Langkah Penelitian

Berikut adalah langkah-langkah penelitian yang dilakukan:

1. Studi Literatur
Mengkaji penelitian terdahulu terkait GEC, proses anotasi, pembangunan *dataset*, serta pendekatan pengembangan sistem untuk memahami kebutuhan dan tantangan utama.
2. Analisis Kebutuhan
Mengidentifikasi kebutuhan fungsional dan non-fungsional dengan meninjau alur kerja anotasi serta peran para pemangku kepentingan.
3. Perancangan Sistem
Menyusun rancangan arsitektur, alur proses, dan struktur data agar *platform* dapat mendukung anotasi secara terstruktur dan konsisten.
4. Implementasi Sistem

Mewujudkan rancangan penelitian menjadi *platform* anotasi berbasis web melalui pengembangan bertahap yang menyesuaikan kebutuhan pengguna.

5. Pengujian dan Evaluasi

Menguji dan mengevaluasi sistem untuk memastikan sistem berjalan dengan baik dan sesuai kebutuhan pengguna.

1.7 Posisi Penelitian

Penelitian ini merupakan bagian dari pengembangan sistem ANOTA, yaitu *platform* anotasi berbasis web yang dirancang untuk mendukung proses penyusunan *dataset Grammatical Error Correction* (GEC). Proyek ANOTA secara keseluruhan terdiri atas dua komponen utama, yaitu *frontend* sebagai antarmuka pengguna dan *backend* sebagai penyedia layanan inti, manajemen data, serta pengaturan alur kerja anotasi.

Posisi penelitian ini berada pada pengembangan *backend* aplikasi ANOTA. Lingkup pekerjaan yang dilakukan mencakup perancangan arsitektur sistem, penyusunan basis data, implementasi modul-modul utama, penyediaan API yang *RESTful*, serta integrasi dengan lingkungan kontainerisasi. Dengan demikian, penelitian ini berfokus pada perancangan dan pembangunan fondasi sistem yang akan digunakan oleh komponen *frontend* saat proses anotasi dan peninjauan dilakukan.

Kontribusi penelitian ini terletak pada penyediaan komponen *backend* yang stabil, terukur, dan siap diintegrasikan dengan *frontend*, sehingga keseluruhan *platform* ANOTA dapat berfungsi secara utuh sebagai alat bantu pembuatan *dataset* GEC Bahasa Indonesia.

1.8 Sistematika Penulisan

Subbab ini menjelaskan struktur penulisan laporan penelitian. Adapun sistematika penulisan laporan ini adalah sebagai berikut

- BAB 1 PENDAHULUAN

Bab ini memuat latar belakang, rumusan masalah, batasan penelitian, tujuan penelitian, manfaat penelitian, langkah penelitian, posisi penelitian, serta sistematika penulisan.

- BAB 2 STUDI LITERATUR

Bab ini berisi landasan teori dan kajian pustaka yang meliputi konsep-konsep dasar, metodologi yang digunakan, serta penelitian terkait yang menjadi dasar dalam pengem-

bangun sistem.

- **BAB 3 METODE PENELITIAN**

Bab ini menjelaskan metode penelitian yang digunakan, termasuk pendekatan pengembangan sistem, tahapan penelitian, serta aktivitas-aktivitas yang dilakukan dalam proses pengembangan.

- **BAB 4 ANALISIS DAN PERANCANGAN**

Bab ini menyajikan analisis kebutuhan sistem, baik kebutuhan fungsional maupun non-fungsional, serta analisis terhadap aktor dan alur kerja yang terlibat dalam sistem.

- **BAB 5 IMPLEMENTASI**

Bab ini membahas perancangan sistem, yang mencakup perancangan arsitektur, model data, serta komponen dan struktur utama yang mendukung proses pengembangan sistem.

- **BAB 6 EVALUASI**

Bab ini berisi implementasi dan pengujian sistem, mencakup proses realisasi perancangan menjadi sistem yang dapat dijalankan serta hasil evaluasi berdasarkan pengujian yang dilakukan.

- **BAB 7 PENUTUP**

Bab ini berisi kesimpulan dari hasil penelitian serta saran untuk pengembangan dan penelitian lebih lanjut.

BAB 2

STUDI LITERATUR

Bab ini menjelaskan mengenai beberapa terminologi dan konsep utama yang perlu dipahami dalam pengembangan aplikasi berbasis web untuk membantu proses anotasi pada *dataset* GEC.

2.1 *Grammatical Error Correction* (GEC)

Bagian ini akan menjelaskan mengenai definisi dari *Grammatical Error Correction* (GEC), metodologi anotasi *dataset* GEC, metrik pengukuran kualitas anotasi (*Inter-Annotator Agreement*), serta format *dataset* GEC.

2.1.1 Definisi *Grammatical Error Correction* dan Pentingnya *Dataset* Beranotasi

Menurut Bryant et al. (2023), *Grammatical Error Correction* (GEC) adalah salah satu tugas dalam bidang *Natural Language Processing* (NLP) yang bertujuan untuk mengoreksi kesalahan gramatikal pada teks secara otomatis. Proses ini tidak hanya terbatas pada perbaikan tata bahasa, seperti kesalahan preposisi dan ketidaksesuaian antara subjek dan kata kerja, tetapi juga mencakup perbaikan ortografi dan aspek semantik, misalnya kesalahan ejaan serta pemilihan diksi atau kata yang kurang tepat.

Dalam upaya memastikan model GEC dapat bekerja dengan baik, dibutuhkan korpus teks teranotasi yang terdiri atas pasangan kalimat salah dan kalimat benar untuk digunakan sebagai data *training* sekaligus sebagai acuan evaluasi. Perkembangan model GEC saat ini sangat dipengaruhi oleh ketersediaan dan kualitas *dataset* yang baik (Bryant et al., 2023). Dalam upaya menciptakan model GEC yang baik, telah dikembangkan beberapa korpus GEC yang memenuhi *gold standard*, seperti NUS Corpus of Learner English (NUCLE) (Dahlmeier et al., 2013), dan korpus yang digunakan dalam *shared task* CoNLL-2013 (Ng et al., 2013) dan CoNLL-2014 (Ng et al., 2014).

Meskipun demikian, dari ketiga korpus utama yang banyak digunakan tersebut, belum terdapat korpus standar untuk bahasa Indonesia. Upaya awal penyusunan *dataset* GEC beranotasi mulai muncul, misalnya korpus sintetik Gramatika yang membangkitkan kalimat salah dari kalimat benar dan memformat hasilnya menjadi pasangan kalimat paralel

serta format M2 untuk mendukung pelatihan dan evaluasi model GEC bahasa Indonesia (Tadjudin, 2023). Selain itu, terdapat juga beberapa upaya lanjutan yang dilakukan untuk mengembangkan *dataset* sintetis GEC Indonesia seperti yang dilakukan oleh Marier et al. (2025), Lin et al. (2024) dan Musyafa et al. (2024). Hal ini menegaskan pentingnya keberadaan sistem anotasi yang terstruktur guna mempercepat pembuatan korpus GEC dalam bahasa Indonesia dan menjembatani kesenjangan sumber daya antara bahasa Indonesia dan bahasa-bahasa lainnya yang sudah memiliki korpus GEC standar.

2.1.2 Dataset Sintetis

Dataset sintetis adalah data yang dibuat secara artifisial melalui proses komputasional seperti menggunakan algoritma atau model generatif untuk meniru karakteristik data nyata, dan dapat digunakan sebagai data pelatihan maupun evaluasi (Lu et al., 2025). Pada domain *Grammatical Error Correction* (GEC), salah satu pendekatan pembuatan dataset sintetis adalah *tagged corruption models*, yaitu model yang menerima kalimat bersih sebagai target beserta *tag* tipe kesalahan (misalnya skema ERRANT), kemudian menghasilkan versi kalimat yang sengaja dibuat salah, dengan tipe kesalahan sesuai *tag* tersebut. Hasil dari proses ini adalah pasangan paralel (kalimat *error* dan kalimat benar) sebagai data sintetis (Stahlberg and Kumar, 2021).

2.1.3 Metode Anotasi Dataset GEC

Proses anotasi *dataset* GEC umumnya dilakukan dalam beberapa tahapan utama, yaitu: (1) pengumpulan teks awal (misalnya esai mata kuliah bahasa), (2) penetapan panduan anotasi (*annotation guidelines*), (3) proses anotasi oleh anotator secara manual, (4) pemeriksaan kualitas anotasi (misalnya melalui proses *adjudication*), dan (5) transformasi hasil anotasi ke format standar untuk pelatihan dan evaluasi model (Dahlmeier et al., 2013; Ng et al., 2014,1).

Pada korpus NUCLE, teks esai dari mata kuliah bahasa Inggris dianotasi dengan fokus pada kesalahan tata bahasa yang relevan pada konteks akademik (Dahlmeier et al., 2013). anotator dibekali panduan anotasi yang rinci, termasuk prinsip pemilihan rentang teks yang sesingkat mungkin (*minimal edit span*) dan aturan kategorisasi jenis kesalahan, misalnya VERB : SVA untuk kesalahan kesesuaian subjek-verba, PREP untuk kesalahan preposisi, atau ARTICLE untuk penggunaan artikel yang keliru. Panduan yang serupa juga digunakan

dalam CoNLL-2013 dan CoNLL-2014, di mana peserta diberi *training data* yang telah dikoreksi secara manual dan diuji pada esai baru yang bersifat *blind test* (Ng et al., 2014,1).

Seiring berkembangnya pendekatan *neural network* dan kebutuhan data yang lebih besar, sebagian proses anotasi mulai dibantu oleh alat otomatis seperti ERRANT (*ERRor ANnotation Toolkit*). ERRANT dapat mengekstraksi dan mengklasifikasikan kesalahan dari pasangan kalimat salah-benar secara otomatis berdasarkan aturan linguistik dan informasi *alignment* (Bryant et al., 2017). *Toolkit* ini banyak digunakan untuk menstandarkan anotasi berbagai korpus GEC ke dalam satu skema jenis kesalahan dan format M2 yang seragam, serta menjadi komponen penting dalam BEA-2019 *Shared Task on Grammatical Error Correction* (Bryant et al., 2019). Dengan demikian, peneliti dapat membandingkan kinerja berbagai sistem GEC secara lebih adil karena *dataset* dan metrik evaluasi yang digunakan telah dinormalisasi.

Dalam konteks pengembangan korpus GEC baru, khususnya untuk bahasa selain bahasa Inggris, metodologi anotasi yang jelas dan terdokumentasi menjadi sangat penting untuk memastikan konsistensi keputusan anotator sehingga kualitas *dataset* tetap terjaga, memudahkan pelatihan anotator baru karena aturan anotasi terdokumentasi secara eksplisit, serta memungkinkan replikasi serta perluasan korpus di masa depan, baik oleh peneliti yang sama maupun komunitas peneliti lain.

Berbagai studi tentang GEC untuk bahasa dengan sumber daya terbatas menegaskan bahwa ketersediaan panduan anotasi yang terstruktur dan *platform* anotasi yang mendukung alur kerja ini merupakan prasyarat utama untuk mendapatkan *dataset* yang berkualitas tinggi (Lin et al., 2024; Marier et al., 2025; Musyafa et al., 2024).

2.1.4 Inter-Annotator Agreement

Kualitas *dataset* beranotasi sangat dipengaruhi oleh konsistensi keputusan antar anotator. Oleh karena itu, *inter-annotator agreement* (IAA) menjadi metrik penting dalam penilaian kualitas korpus GEC. IAA mengukur sejauh mana beberapa anotator memberikan label atau koreksi yang sama terhadap unit data yang sama, dengan potensi kesepakatan yang mungkin terjadi secara acak. Salah satu ukuran IAA yang paling sering digunakan adalah Cohen's kappa (κ), yang membandingkan kesepakatan yang diamati dengan kesepakatan yang diharapkan terjadi secara acak (Artstein and Poesio, 2008; Cohen, 1960). Nilai κ

didefinisikan dalam persamaan berikut:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Di mana:

- p_o adalah probabilitas kesepakatan yang diamati (*relative observed agreement*) di antara anotator.
- p_e adalah probabilitas kesepakatan yang diharapkan terjadi secara kebetulan (*hypothetical probability of chance agreement*).

Nilai koefisien ini berada dalam rentang $-1 \leq \kappa \leq 1$. Nilai $\kappa = 1$ menunjukkan kesepakatan sempurna, $\kappa = 0$ setara dengan kesepakatan acak, dan nilai negatif ($\kappa < 0$) menunjukkan ketidaksepakatan sistematis.

2.1.5 Format Dataset GEC

Terdapat dua format *dataset* yang sering digunakan pada domain GEC, yaitu *parallel format* dan *M2 format*. Pada representasi pasangan kalimat paralel, setiap contoh terdiri dari kalimat asli yang berisi kesalahan dan versi koreksinya. Format ini sangat sesuai dengan pendekatan berbasis *sequence-to-sequence* dan *neural machine translation*, di mana model dilatih untuk seolah-olah menerjemahkan kalimat salah menjadi kalimat benar (Bryant et al., 2023). Korpus seperti NUCLE dan data CoNLL-2013/2014 disebarkan dalam bentuk ini, sehingga banyak sistem GEC modern dilatih pada pasangan kalimat paralel (Dahlmeier et al., 2013; Ng et al., 2014).

Kalimat Asli	Kalimat Koreksi
People get certain disease because of genetic changes .	People get certain diseases because of genetic changes .

Gambar 2.1: Contoh format pasangan kalimat paralel pada korpus GEC

Pada contoh pada Gambar 2.1, struktur pasangan kalimat paralel dapat dijelaskan sebagai berikut:

- Kolom kiri berisi kalimat sumber yang ditulis oleh pembelajar bahasa dan masih men-

gandung kesalahan, misalnya penggunaan bentuk tunggal *disease* pada konteks yang seharusnya jamak.

- Kolom kanan berisi kalimat koreksi yang dianggap benar secara gramatikal oleh anotator manusia, dalam hal ini dengan memperbaiki *disease* menjadi *diseases*.
- Setiap baris merepresentasikan satu pasangan contoh yang dapat digunakan sebagai data latih maupun data uji untuk model GEC berbasis *sequence-to-sequence* atau *neural machine translation*, di mana model belajar memetakan kalimat sumber ke kalimat koreksi.

Namun, pasangan kalimat paralel saja tidak memuat informasi eksplisit tentang lokasi dan jenis setiap kesalahan. Untuk itu, diperkenalkan format M2 (*MaxMatch*), yaitu format yang merepresentasikan kalimat sumber diikuti oleh sekumpulan baris anotasi yang mendeskripsikan deret *edit* frasa yang perlu dilakukan untuk menghasilkan kalimat koreksi (Dahlmeier and Ng, 2012). Algoritme MaxMatch menghitung serangkaian *edit* frasa yang secara optimal memetakan kalimat sumber ke hipotesis koreksi, dan output ini selanjutnya digunakan untuk perhitungan metrik evaluasi seperti $F_{0.5}$ (Qorib and Ng, 2022). Toolkit ERRANT secara luas digunakan untuk mengubah pasangan kalimat paralel menjadi anotasi M2 sekaligus memetakan setiap perubahan ke dalam tipe kesalahan tertentu, dan telah diadopsi, misalnya, dalam BEA-2019 *shared task* untuk menstandarkan berbagai korpus GEC ke format M2 dan skema tipe kesalahan yang konsisten (Bryant et al., 2019,1).

```
S People get certain disease because of genetic changes .
A 3 4|||Nn|||diseases|||REQUIRED|||-NONE-|||0
```

Gambar 2.2: Contoh format M2 untuk satu kalimat pada korpus GEC

Secara umum, format M2 terdiri atas satu baris kalimat asal yang diawali dengan huruf S, diikuti satu atau beberapa baris anotasi yang diawali huruf A. Struktur baris anotasi pada contoh di Gambar 2.2 dapat dijelaskan sebagai berikut.

- S People get certain disease because of genetic changes .
Baris ini menyimpan kalimat sumber yang masih mengandung kesalahan. Setiap token biasanya sudah ditokenisasi (misalnya tanda titik dipisah menjadi token tersendiri).
- A 3 4|||Nn|||diseases|||REQUIRED|||-NONE-|||0

Baris ini menyimpan satu *edit* yang perlu dilakukan pada kalimat sumber. Komponen-komponennya dipisahkan oleh pemisah `|||`:

- A menandakan bahwa baris ini merupakan baris anotasi atau *edit*.
- 3 4 menunjukkan indeks token awal dan akhir dari span yang diubah. Pada contoh ini, indeks 3 dan 4 menunjuk pada token *disease*.
- Nn adalah tipe kesalahan menurut skema klasifikasi yang digunakan (misalnya Nn untuk kesalahan terkait kata benda).
- diseases adalah token pengganti (*replacement*) yang akan digunakan untuk memperbaiki span tersebut.
- REQUIRED menunjukkan bahwa koreksi ini bersifat wajib (bukan anotasi opsional).
- -NONE- merupakan kolom komentar, yang dapat diisi apabila anotator ingin menuliskan catatan tambahan terkait *edit* tersebut.
- 0 adalah ID anotator yang melakukan anotasi; nilai ini memungkinkan beberapa anotator berbeda dianalisis pada kalimat yang sama.

Dengan representasi seperti ini, kalimat sumber dan daftar *edit* dapat secara deterministik dikonversi kembali menjadi *dataset gold standard*, sekaligus menyediakan informasi lokasi dan tipe kesalahan yang diperlukan untuk evaluasi dan analisis kesalahan yang lebih rinci. Banyak korpus GEC modern, termasuk NUCLE dan data CoNLL-2013/2014, dikonversi ke format M2 untuk keperluan evaluasi menggunakan M^2 *scorer* dan toolkit ERRANT, sedangkan korpus GEC bahasa Indonesia seperti Gramatika dan IGED mengadopsi prinsip representasi serupa dalam bentuk pasangan kalimat paralel dan dukungan ekspor ke format M2 (Dahlmeier et al., 2013; Lin et al., 2024; Marier et al., 2025; Ng et al., 2014,1; Tadjudin, 2023).

2.2 Konsep Pengembangan Perangkat Lunak

Bagian ini menjelaskan dua pendekatan utama yang digunakan dalam pengembangan ANOTA, yaitu metode kanban dan agile *framework* dari *Software Development Life Cycle* (SDLC).

2.2.1 Kanban

Kanban merupakan salah satu metode manajemen kerja yang awalnya dikembangkan di dunia manufaktur, kemudian diadopsi secara luas dalam pengembangan perangkat lunak

(Ahmad et al., 2013; Ikonen et al., 2011). Tujuan utama Kanban adalah membantu tim memvisualisasikan pekerjaan, membatasi jumlah pekerjaan yang sedang dikerjakan (*work in progress/WIP*), serta mendorong perbaikan proses secara berkelanjutan (Ladas, 2009). Visualisasi umumnya dilakukan melalui papan Kanban (*Kanban board*) yang berisi kolom-kolom seperti *To Do*, *In Progress*, *In Review*, dan *Done*, dengan setiap tugas direpresentasikan sebagai *card*. Pendekatan ini memudahkan tim untuk melihat status terkini setiap pekerjaan dan mengidentifikasi *bottleneck* dalam alur kerja (Ahmad et al., 2013).

Dalam konteks pengembangan perangkat lunak, pembatasan WIP menjadi praktik penting karena mencegah tim mengerjakan terlalu banyak hal secara paralel (Ikonen et al., 2011). Dengan fokus menyelesaikan tugas yang sedang berjalan sebelum memulai tugas baru, risiko *multitasking* yang berlebihan dan penumpukan pekerjaan di satu tahap dapat dikurangi. Beberapa studi menunjukkan bahwa penggunaan Kanban dalam pengembangan perangkat lunak dapat meningkatkan transparansi proses, mempercepat alur penyelesaian tugas, serta membantu tim merespons perubahan kebutuhan dengan lebih cepat karena prioritas pekerjaan dapat diatur ulang secara dinamis pada papan Kanban¹ (Ahmad et al., 2013; Ikonen et al., 2011).

2.2.2 Agile

Agile Software Development adalah pendekatan pengembangan perangkat lunak yang menekankan iterasi singkat, kolaborasi intensif dengan pemangku kepentingan, serta kemampuan merespons perubahan kebutuhan secara cepat (Beck et al., 2001; Dingsøyr et al., 2012). Berbeda dengan model pengembangan tradisional yang linear dan kaku, Agile memecah pekerjaan menjadi siklus-siklus kecil yang berulang (iterasi). Pada setiap iterasi, tim merencanakan pekerjaan, mengimplementasikan fitur, melakukan pengujian, kemudian melakukan evaluasi terhadap hasil yang dicapai untuk menjadi masukan pada iterasi berikutnya (Dingsøyr et al., 2012).

Prinsip-prinsip Agile seperti *working software over comprehensive documentation* dan *responding to change over following a plan* menjadikannya cocok untuk proyek penelitian dan prototipe produk, di mana kebutuhan sistem belum sepenuhnya terdefinisi di awal dan dapat berkembang seiring temuan baru (Beck et al., 2001). Dalam konteks pengembangan

¹ Lihat pembahasan penggunaan Kanban pada proses pengembangan *backend* ANOTA pada Bab 3.

backend, pendekatan iteratif ini memungkinkan penyesuaian desain API, model data, dan mekanisme otorisasi berdasarkan temuan masalah teknis maupun masukan pengguna (Dingsøyr et al., 2012).

2.3 Testing dan Observability

Bagian ini menjelaskan mengenai tentang pengujian yang dilakukan dan pemantauan terhadap metrik-metrik utama pengukuran. Terdapat beberapa pengujian yang dilakukan yaitu, *unit testing*, *load testing* dan verifikasi akhir menggunakan UAT. Selanjutnya, hasil dari pengujian di proses oleh Prometheus sebelum divisualisasikan oleh Grafana.

2.3.1 Unit Testing

Unit testing adalah metode pengujian perangkat lunak yang berfokus pada pengujian unit terkecil dari kode program seperti fungsi, metode, atau kelas secara terisolasi untuk memastikan bahwa unit tersebut berperilaku sesuai dengan spesifikasi (Runeson, 2006). Pendekatan ini penting untuk mendeteksi kesalahan secepat mungkin pada level logika bisnis, sebelum komponen diintegrasikan dengan bagian sistem lain (Myers et al., 2011). Dalam praktik modern, *unit test* biasanya dijalankan otomatis secara berkala, misalnya setiap kali terjadi perubahan kode (Beller et al., 2015).

Dalam ekosistem Python, berbagai kerangka kerja (*testing framework*) seperti Pytest banyak digunakan karena mendukung penulisan skrip pengujian yang ringkas, penggunaan *fixture* untuk menyiapkan data uji, dan integrasi dengan laporan berupa *code coverage* (Zhang and Mesbah, 2015). *Code coverage* menjadi indikator seberapa banyak jalur logika, termasuk penanganan *edge case* dan *exception*, yang telah tervalidasi oleh *unit test* (Runeson, 2006). Dengan adanya *unit test* otomatis, tim dapat melakukan perubahan atau *refactoring* kode *backend* dengan risiko regresi yang lebih rendah, karena perilaku penting sistem telah terlindungi oleh serangkaian pengujian (Beller et al., 2015).

2.3.2 Load Testing

Load testing termasuk dalam kategori pengujian performa non-fungsional yang bertujuan mengevaluasi bagaimana sistem berperilaku ketika dihadapkan pada beban kerja tertentu (Elberzhager et al., 2012; Legramante et al., 2020). Fokus utamanya adalah mengukur metrik kinerja seperti *response time*, *throughput*, *error rate*, serta pemakaian sumber daya

(CPU, memori, dan I/O) ketika sistem diakses secara bersamaan oleh banyak pengguna (Jain, 1991). Hasil *load testing* membantu menentukan batas kapasitas sistem dan mengidentifikasi *bottleneck* sebelum aplikasi digunakan dalam *production environment* (Meier et al., 2007).

Dalam pengembangan sistem berbasis web, *load testing* dapat dilakukan dengan alat seperti Locust, JMeter, atau k6 yang mensimulasikan sejumlah *virtual user* yang menjalankan skenario akses API tertentu (Legramante et al., 2020). Skenario biasanya dirancang agar mencerminkan pola penggunaan nyata, misalnya kombinasi operasi *read* dan *write*, akses *endpoint*, serta skenario *peak load*.

Németh and Sótér (2021) menekankan bahwa pengujian performa yang efektif harus dibuat dengan skenario yang menunjukkan pola interaksi pengguna nyata bukan hanya menjalankan pengujian secara mekanis. Mereka juga menekankan bahwa interpretasi hasil pengujian seperti analisis *bottleneck*, tren *throughput*, dan perilaku sistem di dalam pengujian, sama pentingnya dengan pelaksanaan uji itu sendiri. Metode ini memastikan bahwa hasil pengujian tidak hanya mengukur ketahanan sistem, tetapi juga memberikan wawasan terkait dampak beban terhadap pengalaman pengguna.

2.3.3 User Acceptance Testing

User Acceptance Testing (UAT) merupakan tahap akhir dalam siklus pengujian perangkat lunak yang berfokus pada verifikasi apakah sistem yang dibangun telah memenuhi kebutuhan dan skenario penggunaan yang diharapkan oleh pengguna akhir atau pemilik proses bisnis (Aladwani and Palvia, 2002; Davis and Venkatesh, 2004). Berbeda dari pengujian teknis yang menekankan aspek internal seperti struktur kode atau performa sistem, UAT menilai kesesuaian alur kerja dari sudut pandang pengguna terkait apakah langkah-langkah yang harus dilakukan masuk akal, mudah diikuti, dan menghasilkan keluaran yang benar sesuai kebutuhan operasional (Venkatesh et al., 2003).

Pada sistem *backend* yang tidak memiliki antarmuka visual langsung, UAT tetap dapat dilakukan melalui emulasi klien menggunakan alat seperti Postman atau koleksi skrip uji API lainnya. Dalam pendekatan ini, skenario bisnis diterjemahkan menjadi rangkaian permintaan HTTP lengkap dengan *payload*, *header* otentikasi, dan urutan eksekusi tertentu. Hasil respons dievaluasi berdasarkan kode status, struktur dan isi data, serta efek samping

di basis data, misalnya perubahan status dokumen atau hak akses pengguna (Aladwani and Palvia, 2002). Hasil UAT menunjukkan apakah kombinasi *endpoint* yang telah diimplementasikan benar-benar mampu mewujudkan proses anotasi kolaboratif yang diharapkan dan siap digunakan dalam pelaksanaan proyek anotasi GEC (Davis and Venkatesh, 2004; Venkatesh et al., 2003).

2.3.4 Prometheus dan Grafana

Konsep *observability* dalam pengembangan perangkat lunak modern menekankan pentingnya kemampuan sistem untuk diamati dari luar melalui metrik, log, dan *tracing*, sehingga tim pengembang dapat memahami keadaan internal sistem tanpa harus melakukan instrumen manual secara terus-menerus (Sigelman et al., 2010; Wilkinson, 2019). Untuk mendukung hal ini, banyak praktik industri yang mengkombinasikan Prometheus dan Grafana sebagai solusi *monitoring* utama di *production environment* (Baskaran, 2020).

Prometheus adalah sistem *monitoring* dan basis data *time-series open source* yang dirancang untuk mengumpulkan metrik dari layanan maupun *host* secara berkala Baskaran (2020). Cara kerjanya umumnya dengan *men-scrape endpoint* metrik (misalnya */metrics*) pada interval tertentu, lalu menyimpan data seperti penggunaan CPU, memori, jumlah permintaan per detik, dan durasi respons permintaan HTTP. Data yang tersimpan dapat digunakan untuk melakukan analisis performa historis, menetapkan *threshold alert*, maupun mengevaluasi dampak perubahan konfigurasi (West and Harrison, 1997).

Grafana adalah *platform* analitik dan visualisasi yang terintegrasi dengan berbagai sumber data, termasuk Prometheus. Melalui *dashboard* Grafana, metrik yang dikumpulkan dapat ditampilkan dalam bentuk grafik interaktif, panel numerik, dan *alert* yang mudah dipahami oleh tim pengembang maupun tim operasi (Baskaran, 2020). Integrasi PrometheusGrafana memungkinkan pemantauan sistem *backend* secara *realtime* serta deteksi dini terhadap anomali, misalnya lonjakan latensi API atau peningkatan penggunaan memori yang tidak wajar (Wilkinson, 2019).

Pada penelitian ini, Prometheus dan Grafana digunakan untuk memantau performa *back-end* ANOTA dalam *testing environment* dan selama pelaksanaan *load test*. Prometheus mengumpulkan metrik dari aplikasi Django, basis data, dan server *host* melalui beberapa layanan seperti Node Exporter, sementara Grafana menyediakan *dashboard* untuk meman-

tau tren metrik selama pengujian. Informasi ini digunakan untuk menganalisis hasil *load testing*, mengidentifikasi *bottleneck* seperti penggunaan CPU yang tinggi pada *endpoint* tertentu, dan mengevaluasi kestabilan sistem di bawah beban. Dengan demikian, kombinasi Prometheus dan Grafana tidak hanya berperan sebagai alat pemantau, tetapi juga sebagai dasar pengambilan keputusan teknis dalam optimasi performa *backend* ANOTA (Baskaran, 2020; Wilkinson, 2019).

BAB 3

METODE PENELITIAN

Bab ini menjelaskan mengenai metodologi penelitian yang digunakan oleh tim pengembangan dalam pembuatan *backend* ANOTA. Pada bab ini akan dijelaskan secara spesifik mengenai metodologi penelitian yang meliputi penggunaan metode Kanban pada Agile *framework* dalam *Software Development Life Cycle* (SDLC), serta tahapan penelitian yang dilakukan dalam pengembangan aplikasi ANOTA.

3.1 Sistem ANOTA

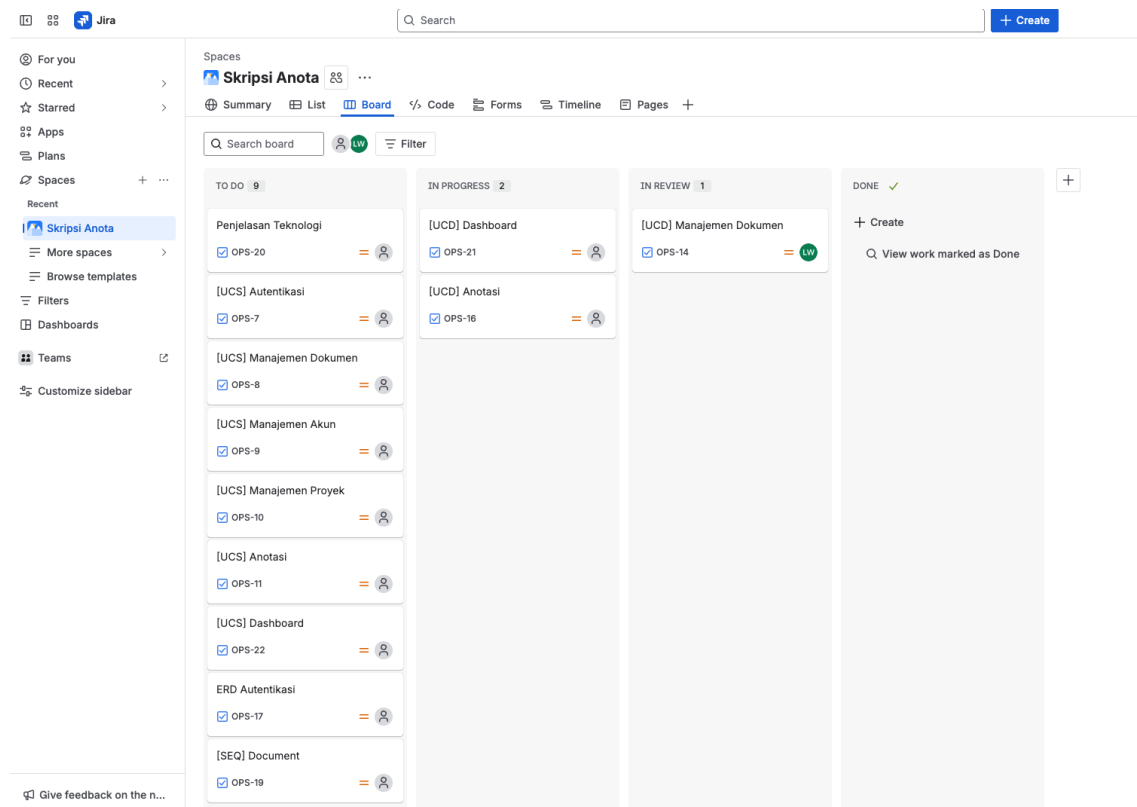
Sistem ANOTA dikembangkan sebagai aplikasi berbasis web untuk mendukung proses pembuatan *dataset Grammatical Error Correction* (GEC) dalam bahasa Indonesia. Sistem ini dirancang agar para pengguna dapat berkolaborasi dalam melakukan proses anotasi serta menghasilkan *dataset* dalam dua format standar, yaitu format *parallel sentences* dan M2.

Pengembangan sistem ANOTA dilakukan secara berkelompok oleh dua tim, yaitu tim *front-end* dan tim *backend*. Bagian tugas akhir ini secara khusus berfokus pada pengembangan sistem *backend* dari aplikasi. Hal ini mencakup analisis, perancangan, implementasi, dan pengujian seluruh komponen *server* dan layanan API.

3.2 Process Model

Proses pengembangan ini dilakukan menggunakan metodologi Agile yang menekankan proses pengembangan yang iteratif, adaptif, dan kolaboratif. Metode Agile dipilih karena sifat proyek yang memiliki kebutuhan awal yang belum sepenuhnya lengkap dan dapat berubah seiring waktu.

Dalam konteks penelitian ini, kerangka Agile yang digunakan lebih spesifik mengacu pada Kanban, yakni pendekatan yang menekankan visualisasi alur kerja, pembatasan pekerjaan yang sedang berlangsung (*work in progress/WIP*), serta proses pengembangan yang berkelanjutan sebagaimana telah dibahas lebih jauh pada Subbab 2.2.2 dan Subbab 2.2.1. Setiap pekerjaan pengembangan *backend* direpresentasikan sebagai *card* yang berpindah di papan Kanban melalui tahapan seperti *To Do*, *In Progress*, *In Review*,

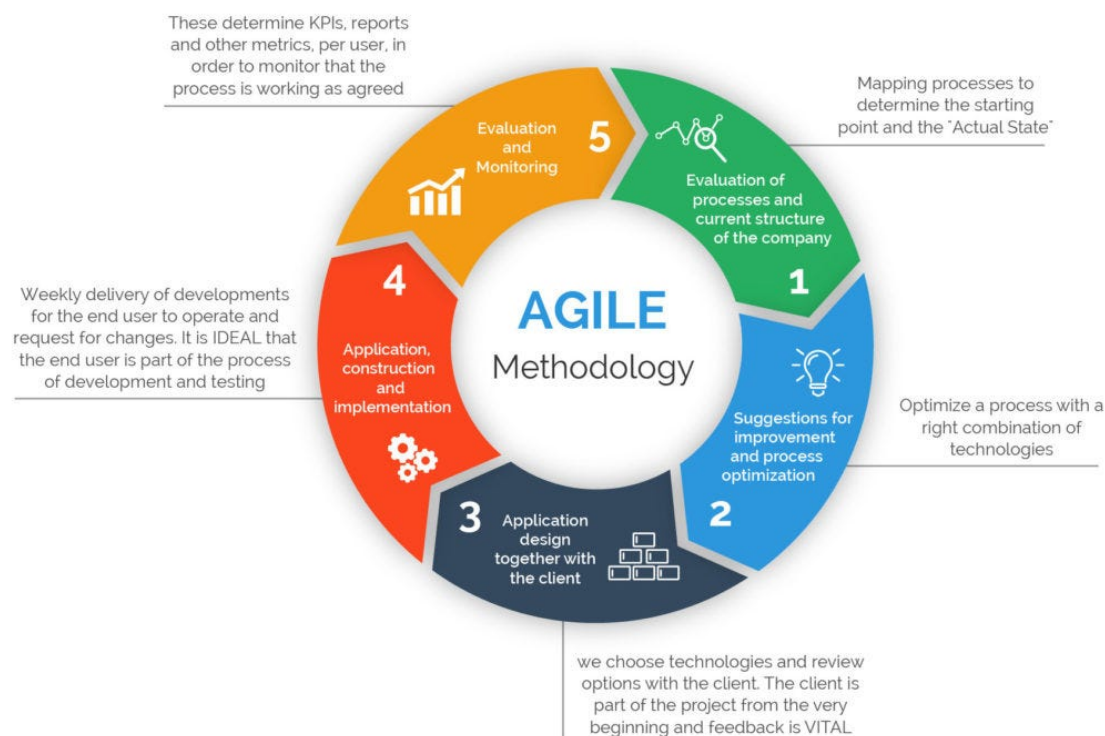


Gambar 3.1: Contoh Kanban Board ANOTA

hingga *Done* seperti pada Gambar 3.1. Dengan batasan WIP pada tiap fase, tim terdorong untuk menyelesaikan tugas yang sedang dikerjakan sebelum memulai tugas baru, sehingga beban kerja lebih terkontrol dan hambatan (*bottleneck*) dalam pengembangan lebih mudah diidentifikasi.

Dengan pendekatan Kanban ini, pengembangan *backend* dilakukan secara bertahap mulai dari kebutuhan dasar hingga fitur tambahan yang disesuaikan berdasarkan hasil evaluasi serta umpan balik pengguna. Proses pengembangan tidak dilakukan secara linier, melainkan dalam tahap iterasi yang terus berulang. Setiap siklus dimulai dengan analisis kebutuhan terhadap fitur yang akan dikembangkan, dilanjutkan dengan proses perancangan dan implementasi, kemudian dilakukan pengujian serta evaluasi hasil. Umpan balik dari hasil pengujian maupun dari pengguna digunakan untuk menyusun prioritas perbaikan atau penambahan fitur pada siklus berikutnya, yang kemudian direfleksikan kembali dalam papan Kanban melalui pengaturan ulang prioritas dan pemindahan *card* antar kolom.

Struktur tahapan penelitian secara umum dapat dilihat pada Gambar 3.2. Diagram tersebut menggambarkan hubungan iteratif antar tahapan dalam pengembangan *backend*.



Gambar 3.2: *Agile Methodology*

Sumber: <https://medium.com/@sudarhtc/>

agile-project-management-methodology-manifesto-frameworks-and-process-f4c332ddb779

Pendekatan Agile dengan kerangka Kanban ini memungkinkan sistem *backend* berkembang secara bertahap dan adaptif terhadap hasil evaluasi serta kebutuhan pengguna yang berubah. Penelitian ini berfokus pada proses perancangan, pengembangan, dan pengujian sistem *backend* ANOTA dengan mengimplementasikan pendekatan Agile berbasis Kanban yang umum digunakan dalam *Software Development Life Cycle* (SDLC). Pendekatan ini membantu mengelola prioritas pekerjaan, memantau progres secara visual, serta memastikan bahwa fitur yang dikembangkan senantiasa selaras dengan kebutuhan dan tantangan nyata yang dihadapi pengguna.

3.3 Tahapan Penelitian

Tahapan penelitian ini secara umum dibagi menjadi empat tahap utama yaitu, tahap analisis, tahap perancangan, tahap implementasi, dan tahap evaluasi. Seluruh tahapan-tahapan ini dilaksanakan mengikuti kaidah pengembangan *agile* guna memastikan kelancaran dari pembuatan *backend* aplikasi ANOTA.

3.3.1 Tahap Analisis

Pada tahap ini akan dilakukan analisis kebutuhan sistem *backend* ANOTA. Kegiatan yang direncanakan meliputi identifikasi pemangku kepentingan dan aktor utama (misalnya anotator, *reviewer*, kepala riset, dan admin), pemahaman alur bisnis proses anotasi dan peninjauan dokumen, serta perumusan kebutuhan fungsional dan non-fungsional sistem. Kebutuhan tersebut kemudian akan dipetakan menjadi *user story* atau *backlog* yang akan dikelola pada papan Kanban.

Secara lebih spesifik, proses analisis kebutuhan tidak hanya mencatat fitur apa saja yang dibutuhkan, tetapi juga mendalami konteks penggunaan sistem dalam skenario di lapangan. Pada tahap ini akan dilakukan identifikasi awal terhadap hak akses, batasan, serta interaksi antar aktor, misalnya bagaimana alur kerja anotator ketika mengerjakan tugas anotasi, bagaimana *reviewer* melakukan peninjauan dan memberikan umpan balik, serta bagaimana kepala riset memantau progres anotasi dan kualitas data. Selain itu, akan dianalisis pula kebutuhan non-fungsional khususnya terkait aspek performa.

Deliverable dari tahap analisis kebutuhan ini meliputi:

- Daftar kebutuhan fungsional sistem *backend* ANOTA;
- Daftar kebutuhan non-fungsional sistem *backend* ANOTA;

3.3.2 Tahap Perancangan

Tahap ini berfokus pada perancangan arsitektur dan komponen utama sistem *backend* berdasarkan data dari tahap analisis yang telah dilakukan. Pada fase perancangan, terdapat beberapa *deliverables* yang meliputi perancangan arsitektur *backend* seperti pemisahan modul autentikasi, manajemen pengguna, pengelolaan dokumen, anotasi, dan review, perancangan model data, serta perancangan antarmuka layanan. Selain itu, pada tahap ini juga akan ditentukan alur kerja pengembangan di papan Kanban, seperti definisi kolom (*To Do*, *In Progress*, *In Review*, *Done*) dan kriteria penyelesaian tugas (*definition of done*).

Deliverable yang dihasilkan pada tahap ini meliputi:

- Rancangan arsitektur *backend*;
- *Entity Relationship Diagram* (ERD);

- Tabel relasi diagram;
- *Class diagram* untuk menggambarkan struktur kelas dan relasi antar komponen;
- Rancangan *Application Programming Interface* (API) dalam bentuk spesifikasi *endpoint* berupa struktur *request* dan *response*, kode status, dan aturan otorisasi.

3.3.3 Tahap Implementasi

Tahap implementasi berfokus pada realisasi rancangan menjadi kode program *backend* yang dapat dijalankan. Kegiatan yang direncanakan mencakup penyiapan *environment* pengembangan, konfigurasi basis data, serta implementasi modul-modul utama sesuai dengan desain yang telah disusun pada tahap perancangan. Setiap *user story* atau tugas yang telah dirumuskan akan direpresentasikan sebagai *card* pada papan Kanban dan dikerjakan secara bertahap dengan memperhatikan batasan *work in progress* (WIP). Selain implementasi fungsional, pada tahap ini juga akan dilakukan inisialisasi awal untuk *environment* dan *setup deployment*.

3.3.4 Tahap Evaluasi

Tahap evaluasi bertujuan untuk menilai sejauh mana implementasi *backend* Anota telah memenuhi kebutuhan yang telah dirumuskan pada tahap analisis. Secara garis besar, pada tahap ini akan dilakukan serangkaian pengujian untuk menilai pemenuhan kebutuhan fungsional dan non-fungsional sistem, serta *User Acceptance Test* (UAT) guna memastikan kesesuaian implementasi dengan kebutuhan nyata di lapangan.

Pengujian fungsional difokuskan pada verifikasi perilaku setiap *endpoint* terhadap skenario penggunaan yang telah didefinisikan, seperti *user management*, *document management*, proses anotasi, dan proses review. Pengujian ini memastikan bahwa setiap fitur berjalan sesuai dengan alur bisnis yang dirancang dan menghasilkan respons yang benar pada berbagai kondisi input, termasuk skenario abnormal. Selain itu, pengujian berikutnya akan dilakukan melalui uji coba terbatas bersama calon pengguna, misalnya *anotator* atau *reviewer*, untuk memperoleh umpan balik terkait kemudahan penggunaan dan kesesuaian fungsi sistem terhadap kebutuhan mereka. Pada tahap akhir, akan dilakukan *User Acceptance Testing* (UAT) secara menyeluruh terhadap seluruh *endpoint* yang digunakan, guna memastikan sistem layak untuk digunakan pada *environment production*.

Pengujian non-fungsional berfokus pada aspek kinerja dan kualitas layanan sistem *back-*

end. Pada tahap ini dilakukan pengukuran waktu respon layanan untuk memastikan bahwa permintaan pengguna dapat diproses dalam batas waktu yang dapat diterima. Selain itu, dilakukan juga *load testing* untuk mengevaluasi perilaku sistem ketika menerima permintaan secara bersamaan dari sejumlah pengguna, sehingga dapat diketahui apakah performa dan stabilitas sistem masih terjaga pada beban tertentu.

Secara ringkas, jenis pengujian yang dilakukan pada tahap evaluasi ini meliputi:

- *Unit test* terhadap setiap *endpoint* berdasarkan skenario penggunaan seperti *user management*, *document management*, proses anotasi, dan proses *review*;
- *Integration test* dilakukan sehari sebelum *sprint review* untuk memastikan sistem *backend* telah terintegrasi penuh dengan *frontend*.
- *User acceptance testing* (UAT) untuk menilai apakah sistem *backend* telah memenuhi kebutuhan yang ada, pengujian hanya dilakukan pada sistem *backend* saja menggunakan Postman Collection dan *test script* (dalam bentuk *script* JavaScript) dari Postman.
- *Load testing* untuk mengevaluasi stabilitas dan performa sistem ketika diakses oleh banyak pengguna secara bersamaan.

BAB 4

ANALISIS DAN PERANCANGAN

Bagian ini membahas mengenai analisis gap solusi, analisis fungsional, non-fungsional, serta perancangan basis data, mekanisme pengujian, hingga perancangan modul-modul pada sistem ANOTA.

4.1 Analisis Gap Solusi

Bagian ini menjelaskan mengenai gap yang ada pada proses anotasi manual, dengan proses anotasi setelah ada sistem ANOTA sesuai dengan Tabel 4.1.

Tabel 4.1: Ringkasan perbandingan kondisi *As-Is* dan *To-Be* pada sistem ANOTA

Aspek	As-Is (Sebelum Sistem)	To-Be (Setelah Sistem)
Distribusi dokumen (<i>assignment</i>)	Penugasan dilakukan secara manual/terfragmentasi yang mengakibatkan rawan terjadi salah penugasan dan sulit dilacak riwayatnya.	Tersedia manajemen distribusi dokumen (<i>assign/unassign</i>) dan riwayat penugasan yang tercatat sehingga proses lebih terkontrol.
Beban kognitif pemilihan jenis <i>error</i>	Annotator cenderung mengandalkan ingatan atau merujuk <i>guideline</i> secara manual saat memilih kategori kesalahan.	Sistem menyediakan daftar <i>ErrorType</i> agar pemilihan kategori lebih konsisten dan mengurangi kesalahan input.
Konversi hasil anotasi ke format dataset	Konversi manual ke format dataset standar rawan tidak seragam dan berpotensi terjadi kesalahan translasi format.	Sistem menyediakan mekanisme ekspor otomatis ke format <i>parallel</i> dan M2 sehingga hasil lebih seragam dan lebih <i>robust</i> terhadap perubahan.
Keterukuran kualitas Anotasi	Kualitas anotasi sulit diukur secara objektif karena tidak ada mekanisme/metrik baku.	Sistem mendukung pengukuran kualitas anotasi dengan metrik IAA untuk memastikan bagaimana kesepakatan antara Annotator dengan Reviewer.

4.2 Kebutuhan Fungsional

Kebutuhan fungsional merupakan layanan dan fitur apa saja yang harus disediakan oleh sistem *backend* agar dapat memenuhi kebutuhan pengguna. Berdasarkan hasil analisis,

sistem *backend* ANOTA terdiri atas enam bagian utama, yaitu:

1. Autentikasi dan Otorisasi;
2. Manajemen Akun;
3. Manajemen Proyek;
4. Manajemen Dokumen;
5. Anotasi dan *Review*; dan
6. *Dashboard* Statistik.

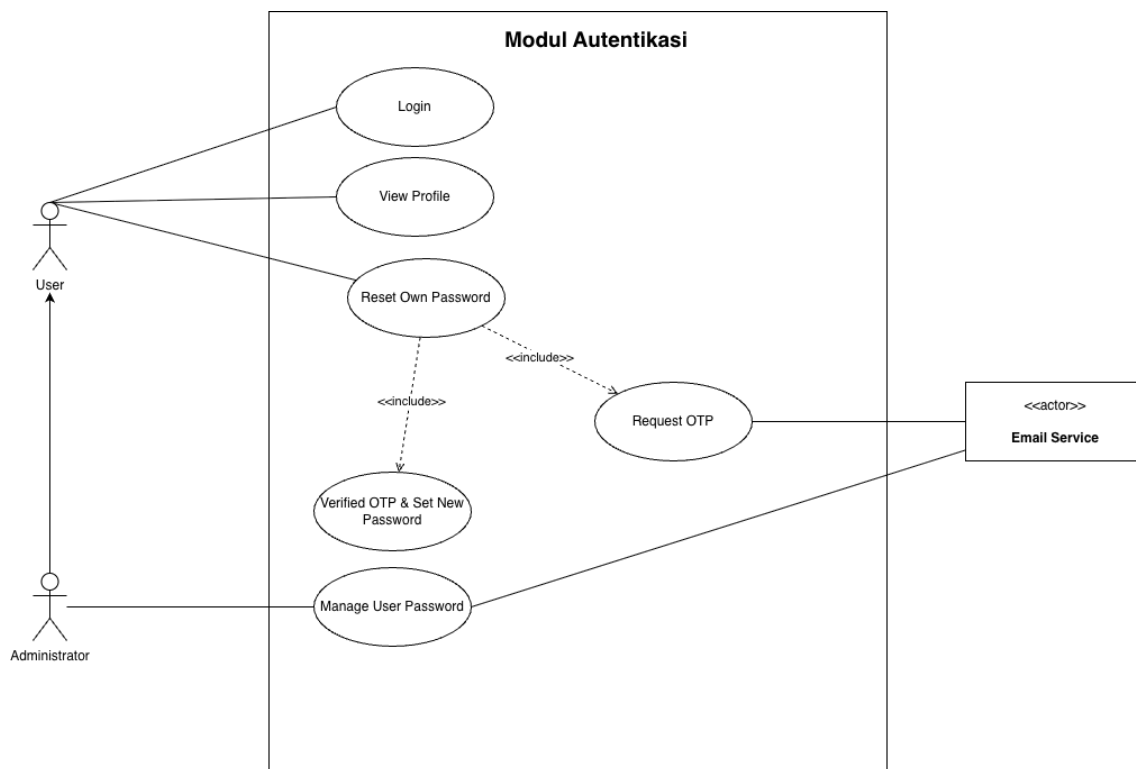
Selain pembagian modul, terdapat beberapa aktor yang berinteraksi dengan sistem. Berikut adalah daftar aktor beserta tanggung jawab utamanya:

- **User:** Aktor yang memiliki kewenangan umum, khususnya terkait fitur autentikasi. Aktor ini dapat berupa Kepala Riset, Admin, Annotator, dan *Reviewer*.
- **Administrator:** Aktor yang berperan dalam melakukan manajemen pada modul-modul utama seperti manajemen akun, *project*, *document*, dan *error*. Aktor ini terdiri atas *role* Admin yang secara spesifik mengatur pada *scope project* yang dikelolanya, serta Kepala Riset yang berfokus pada seluruh sistem.
- **Editor:** Aktor yang berperan dalam proses utama sistem, yaitu proses anotasi dan review. Aktor ini terdiri atas *Annotator* dan *Reviewer*.
- **Annotator:** Aktor yang melakukan proses anotasi pada dokumen.
- **Reviewer:** Aktor yang melakukan validasi dan tinjauan terhadap hasil anotasi.
- **Email Service:** Sistem eksternal untuk pengiriman kredensial dan OTP.

Berdasarkan hasil analisis dari tim penulis, sistem *backend* harus mampu menangani proses-proses berikut:

4.2.1 Autentikasi dan Otorisasi Pengguna

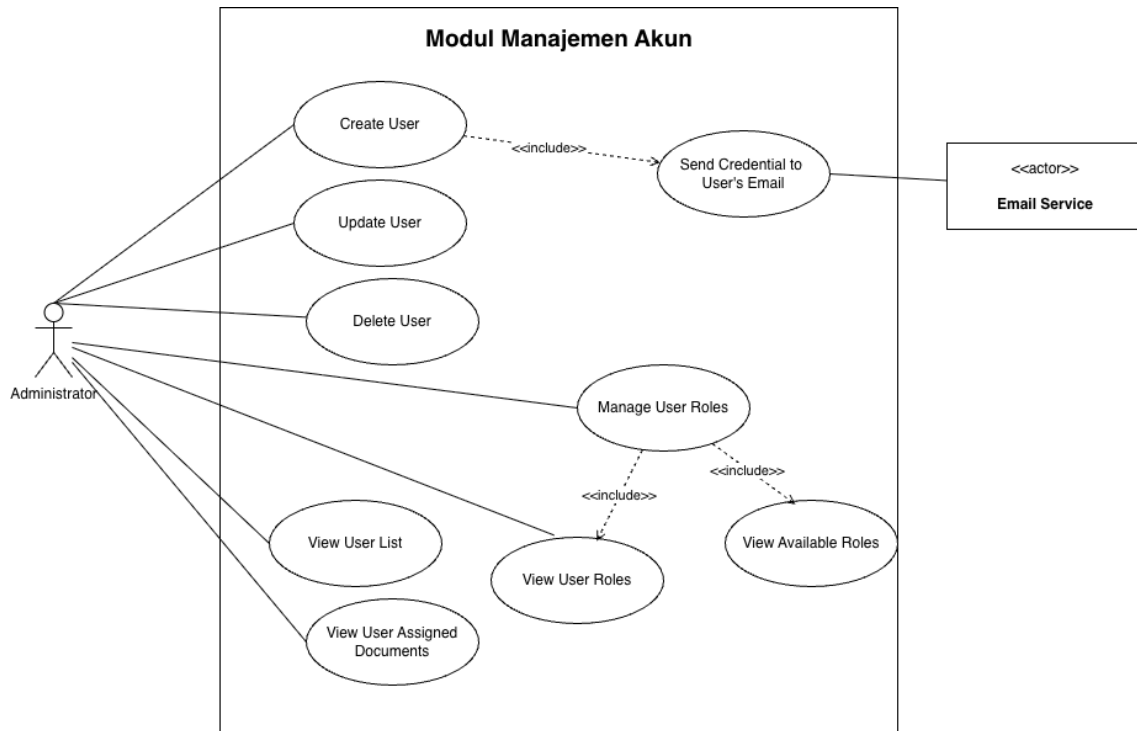
Modul Autentikasi menangani beberapa proses utama seperti *login*, *view profile*, serta *reset* kata sandi secara mandiri melalui OTP yang dikirim oleh *Email Service*, dan menyediakan kewenangan khusus bagi Administrator untuk mengelola kata sandi pengguna lain seperti yang ditunjukkan oleh Gambar 4.1. Pada modul ini terdapat beberapa aktor yang terlibat seperti User yang mencakup *Annotator* dan *Reviewer* dengan hak dasar berupa *login*, *view profile*, dan *reset own password* serta **Administrator sistem** yakni *Admin* dan *Kepala Riset* yang dapat melakukan fitur tambahan berupa *manage user password* dan *Email Service* sebagai aktor eksternal



Gambar 4.1: Use Case Diagram Modul Autentikasi

yang mengirimkan OTP pada alur *reset password*. Dalam upaya memastikan proses autentikasi dan otorisasi yang baik, sistem harus menyediakan mekanisme yang aman untuk mengontrol akses pengguna ke dalam sistem. Fitur-fitur yang harus tersedia meliputi:

- **Reset Password berbasis OTP:** Sistem harus mampu mengirimkan kode OTP (*One-Time Password*) berupa 6 digit angka via email dan hanya berlaku selama 15 menit untuk proses *reset password* secara mandiri oleh pengguna.
- **Verifikasi OTP:** Sistem harus dapat memvalidasi kode OTP yang dimasukkan pengguna dengan maksimal 3 kali percobaan per OTP, dan melakukan pembersihan untuk OTP yang sudah kedaluwarsa sehingga tidak membebani *database*.
- **Reset Password oleh Administrator:** Admin dan Kepala Riset harus dapat mereset *password* pengguna lain dan sistem membuat *password* baru secara otomatis yang memenuhi persyaratan kombinasi 12 karakter yang aman dan mengirimkan notifikasi email kepada pengguna.
- **Informasi Pengguna Aktif:** Sistem harus menyediakan *endpoint* untuk mendapatkan informasi profil pengguna yang sedang login, termasuk *roles*, email, dan data lainnya untuk keperluan validasi pengguna.



Gambar 4.2: Use Case Diagram Modul Manajemen Akun

- **Role-Based Access Control (RBAC):** Sistem harus menerapkan kontrol akses berbasis peran dengan 4 tingkat, yaitu, Kepala Riset (*full authority*), Admin (manajemen terbatas), Annotator (anotasi dokumen), dan Reviewer (*review* anotasi).

4.2.2 Manajemen Akun

Modul manajemen akun memungkinkan Admin/Kepala Riset untuk mengelola akun pengguna seperti membuat, memperbarui, menghapus pengguna, melihat daftar pengguna, dokumen yang ditugaskan ke tiap pengguna, serta mengelola peran pengguna melalui *Manage User Roles* seperti yang ditunjukkan pada Gambar 4.2. Saat membuat pengguna baru, sistem mengirim kredensial ke email pengguna melalui aktor eksternal *Email Service*.

Dalam upaya memastikan manajemen akun yang baik, sistem harus menyediakan fitur-fitur untuk mengelola akun pengguna dalam sistem dengan pembatasan akses sesuai *role*. Fitur-fitur yang diperlukan meliputi:

- **Registrasi Pengguna Baru:** Admin dan Kepala Riset harus dapat membuat akun pengguna baru dengan *password* yang di *generate* otomatis (12 karakter kombinasi huruf, angka, dan simbol) dan sistem mengirimkan email kredensial secara otomatis ke pengguna baru.

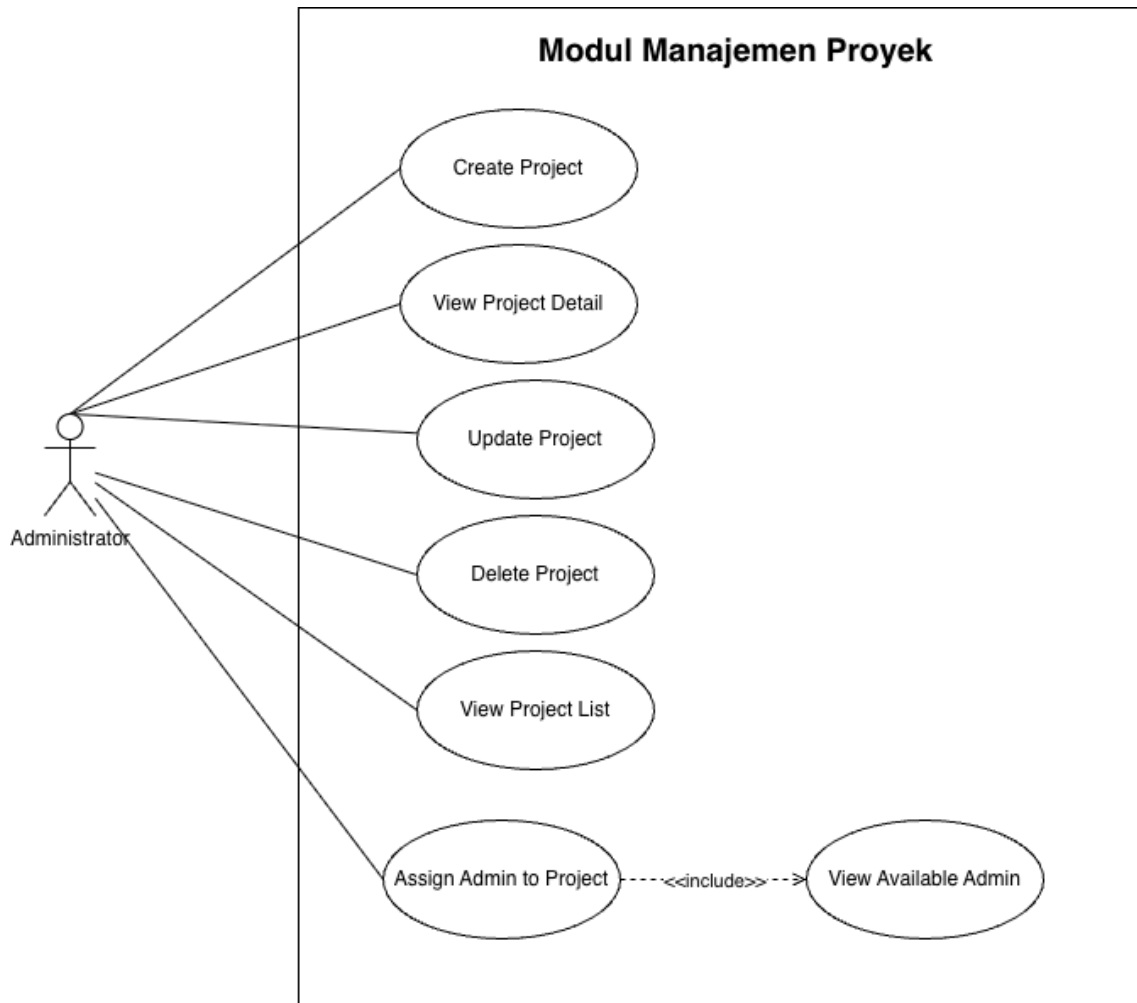
- **Pembaruan Data Pengguna:** Sistem harus memungkinkan Admin dan Kepala Riset untuk mengubah data pengguna meliputi *username*, email, *full name*, dan institusi dengan validasi *uniqueness* untuk *username* dan email.
- **Penghapusan Pengguna (*Soft Delete*):** Sistem harus mendukung *soft delete* yang mana pengguna ditandai sebagai *deleted* dan di *deactivate* tetapi data *annotations* dan aktivitasnya tetap tersimpan untuk integritas *historical data*.
- **Manajemen Role Pengguna:** Sistem harus menyediakan fungsi untuk menambah atau menghapus *role* dari pengguna dengan *constraint*: Kepala Riset dapat mengelola semua *role*, Admin hanya dapat mengelola *role* Annotator dan *Reviewer*, dan sistem harus memiliki minimal satu Kepala Riset aktif.
- **Audit Trail:** Sistem harus mencatat daftar pengguna yang sudah di *soft delete* untuk keperluan audit dan *monitoring* dengan informasi *timestamp* penghapusan.
- **Filter Berdasarkan Role:** Sistem harus menampilkan daftar pengguna yang sesuai dengan *role user* yang login dengan batasan berupa Admin hanya melihat *Annotator* dan *Reviewer*, sedangkan Kepala Riset dapat melihat semua pengguna.
- **Monitoring Assignment:** Sistem harus menyediakan fitur untuk melihat daftar dokumen yang diassign kepada pengguna tertentu untuk *monitoring workload*.

4.2.3 Manajemen Proyek Anotasi

Modul ini memberi Admin kemampuan membuat, melihat detail, memperbarui, menghapus, dan menampilkan daftar proyek seperti yang ditampilkan pada Gambar 4.3. Selain itu, Kepala Riset dapat menetapkan Admin ke sebuah proyek (*Assign Admin to Project*) dengan dukungan *use case* pendukung untuk melihat daftar admin yang tersedia, sehingga struktur pengelolaan proyek terjaga dan hak akses tersalurkan dengan tepat.

Dalam upaya memastikan manajemen proyek yang baik, sistem harus mampu untuk mengelola proyek-proyek riset anotasi. Fitur-fitur yang harus tersedia meliputi:

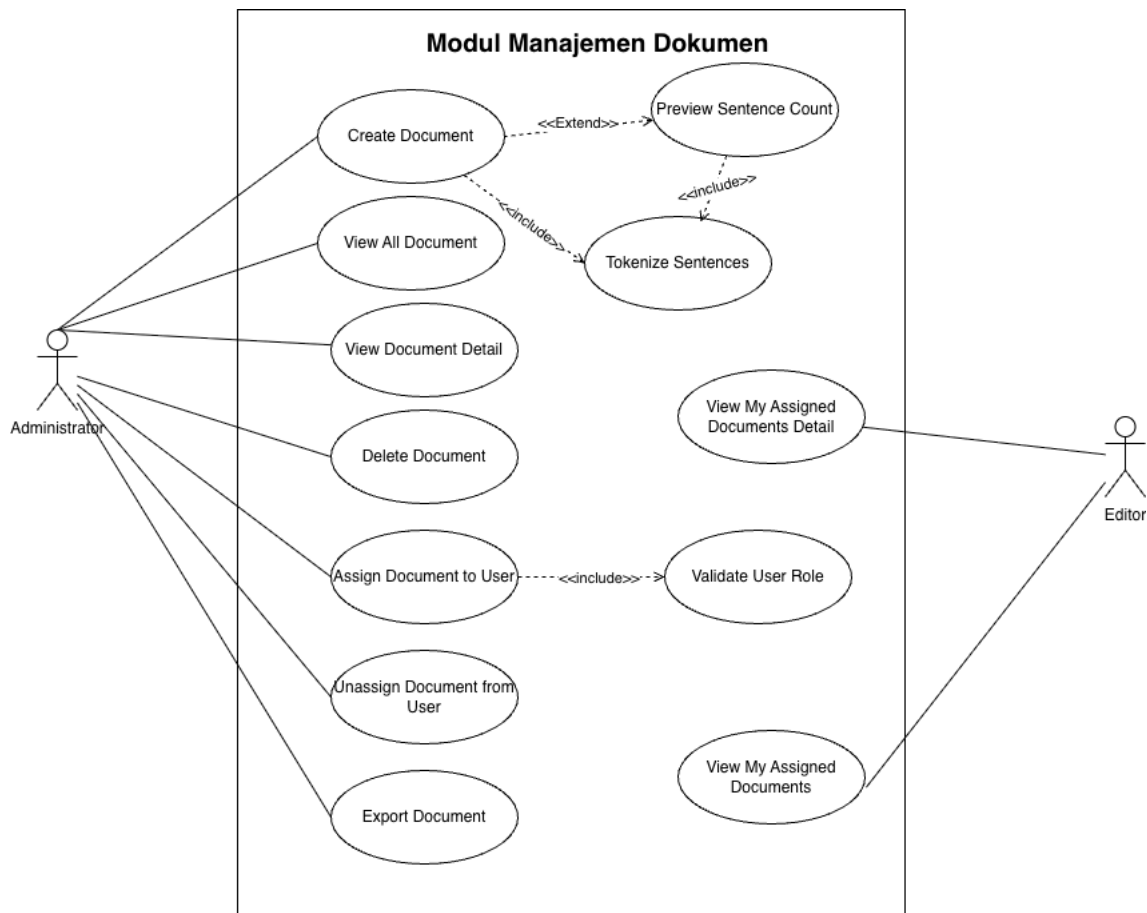
- **CRUD Proyek:** Sistem harus mendukung operasi *Create*, *Read*, *Update*, dan *Delete* untuk proyek dengan validasi *uniqueness* pada nama proyek.
- **Assignment Admin ke Proyek:** Kepala Riset harus dapat menugaskan satu Admin ke setiap proyek dengan *constraint* maksimal 1 admin per proyek, dan sistem harus memvalidasi bahwa *user target* memiliki *role* Admin.
- **Filter Akses Berbasis Role:** Sistem harus menerapkan *automatic filtering* dimana Admin hanya dapat melihat dan mengelola proyek yang diassign kepadanya, sedan-



Gambar 4.3: Use Case Diagram Modul Manajemen Proyek

gkan Kepala Riset dapat mengakses semua proyek.

- ***Cascade Delete***: Sistem harus mampu melakukan *cascade delete* ketika proyek dihapus, yang meliputi penghapusan pada semua dokumen, *sentences*, *annotations*, *reviews*, dan *document assignments* terkait dengan melakukan konfirmasi yang jelas kepada *user*.
- ***Monitoring Proyek***: Sistem harus menyediakan fitur untuk melihat detail proyek termasuk daftar dokumen dalam proyek dan daftar *user* yang diassign ke dokumen-dokumen dalam proyek tersebut.
- ***Project Specific Error Types***: Sistem harus mendukung *error types* yang spesifik untuk setiap proyek selain *error types default* yang berlaku global.



Gambar 4.4: Use Case Diagram Modul Manajemen Dokumen

4.2.4 Manajemen Dokumen

Modul ini berfungsi untuk mengatur dokumen proyek yang memungkinkan sistem untuk membuat dokumen, *Preview Sentence Count*, dan proses tokenisasi, melihat semua dokumen dan detailnya, menghapus, serta mengekspor dokumen seperti yang ditampilkan pada gambar 4.4. Admin/Kepala Riset dapat menugaskan atau membatalkan penugasan dokumen ke pengguna. Bagi Annotator/*Reviewer*, tersedia akses untuk melihat daftar dokumen yang ditugaskan dan detail dokumen yang menjadi tanggung jawabnya.

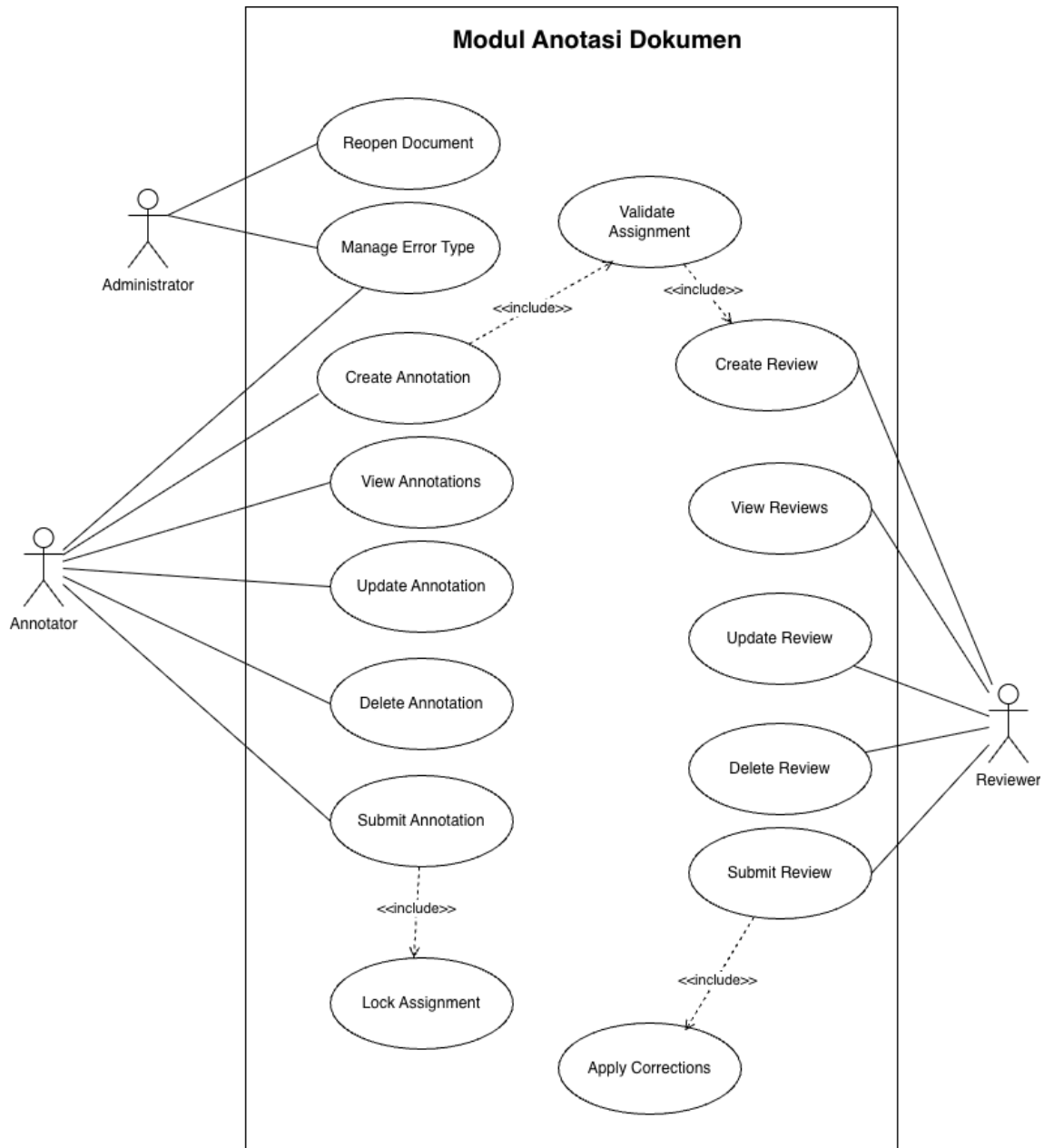
Sistem harus mampu mengakomodasi fungsi-fungsi untuk mengelola dokumen yang akan dianotasi. Fitur-fitur yang diperlukan:

- **Pembuatan Dokumen dengan *Auto-Tokenization*:** Sistem harus dapat membuat dokumen baru dan secara otomatis memecah teks menjadi *sentences* menggunakan *NLTK tokenizer* dengan *fallback* ke *regex* jika *NLTK* tidak tersedia, serta melakukan *filtering* untuk menghilangkan *sentences* kosong atau hanya *whitespace*.

- **Duplicate Detection:** Sistem harus mampu mendeteksi duplikasi dan mencegah pembuatan dokumen dengan teks yang sama kecuali *user* secara eksplisit mengizinkan dengan *flag* `allow_duplicate=true`.
- **Preview Sentence Count:** Sistem harus menyediakan *endpoint* untuk *preview* jumlah kalimat yang akan dihasilkan dari teks tanpa menyimpan ke *database*, menggunakan *logic tokenization* yang sama dengan proses *create document* untuk estimasi *workload*.
- **Assignment Dokumen:** Admin harus dapat melakukan *assignment* dokumen kepada Annotator atau Reviewer dengan *tracking* lengkap yang mencatat `assigned_by`, `assigned_at`, `notes`, dan `status`. Sistem harus memvalidasi bahwa *user* target memiliki *role* Annotator atau Reviewer, dan mendukung *multiple Annotators* per dokumen dengan maksimal satu Reviewer aktif per dokumen.
- **Unassignment Dokumen:** Sistem harus dapat menghapus *assignment* dokumen dari *user* dengan *soft deactivation* (set `is_active=False`) dimana *assignment record* tetap ada untuk audit tetapi *user* akan kehilangan akses, dan *annotations* yang sudah dibuat tetap tersimpan.
- **Export Dokumen:** Sistem harus mendukung *export* dokumen dalam 2 format yaitu *parallel sentences* dan M2 format dengan encoding UTF-8 untuk mendukung karakter internasional.
- **Cascade Delete Dokumen:** Sistem harus melakukan *hard delete permanent* dengan *cascade* ke semua *sentences*, *annotations*, *reviews*, dan *document assignments* dengan konfirmasi yang menampilkan jumlah data yang akan terhapus.
- **View Assignment dengan Status:** Annotator dan Reviewer harus dapat melihat daftar dokumen yang diassign kepada mereka dengan status *realtime* (`belum_dianotasi`, `sedang_dianotasi`, `sudah_dianotasi`, `belum_direview`, `sedang_direview`, `sudah_direview`) dan informasi `assigned_by` serta jumlah *sentences*.
- **Built-in Authorization:** Sistem harus secara otomatis memfilter dokumen berdasarkan *assignment* sehingga *user* hanya dapat mengakses detail dokumen yang diassign kepadanya dengan 404 error jika mencoba akses dokumen yang tidak diassign untuk mencegah *information leakage*.

4.2.5 Anotasi Dokumen dan Review Anotasi

Modul ini memfasilitasi proses anotasi dan review dokumen dimana Annotator dapat membuat, melihat, memperbarui, menghapus, dan men submit anotasi. Saat Submit



Gambar 4.5: Use Case Diagram Modul Anotasi Dokumen

Annotation sistem akan mengunci penugasan dan sebelumnya memastikan penugasan valid. Proses dilanjutkan dengan Reviewer membuat, melihat, memperbaiki, menghapus, dan men submit review. Saat men submit, koreksi diterapkan setelah validasi penugasan. Admin/Kepala Riset dapat membuka kembali dokumen (*Reopen Document*) dan mengelola tipe kesalahan (*Manage Error Type*) sebagai standar anotasi. Use case ini menjadi bagian utama dari sistem ANOTA yang dikembangkan, seperti yang ditampilkan pada Gambar 4.5.

Dalam upaya memastikan proses anotasi dan review yang lancar, sistem harus menye-

diadakan fitur-fitur untuk proses anotasi kesalahan gramatikal dan *review*. Fitur-fitur yang harus tersedia meliputi:

- **Manajemen *Error Types*:** Sistem harus mendukung 2 jenis error yaitu *default error types* yang tersedia untuk semua proyek dan *project specific error types* yang hanya tersedia untuk proyek tertentu. Kepala Riset dapat membuat *default error types*, sedangkan Admin yang diassign ke proyek dapat membuat *project specific error types* dengan validasi *uniqueness* dalam *scope project*.
- **CRUD Anotasi:** Annotator harus dapat membuat, membaca, mengubah, dan menghapus anotasi dengan informasi *start_index*, *end_index*, *error_type*, *correction*, *is_required*, dan *comments*. Sistem harus memvalidasi bahwa sentence yang dianotasi berada dalam dokumen yang diassign dan *assignment* harus aktif.
- **Submit Anotasi:** Annotator harus dapat melakukan submit pekerjaan anotasi pada dokumen yang akan mengubah semua *annotations* menjadi *is_submitted=true*, set *DocumentAssignment is_submitted=true*, dan mengunci dokumen sehingga tidak dapat diubah lagi.
- **Reopen Anotasi:** Annotator harus dapat membuka kembali dokumen yang sudah disubmit dengan *constraint* hanya jika belum ada review yang dibuat, set status kembali ke draft (*is_submitted=false*) dengan alasan *reopen* dicatat untuk *audit trail*.
- **CRUD Review:** Reviewer harus dapat membuat, membaca, mengubah, dan menghapus review terhadap anotasi yang sudah disubmit. Sistem harus mendukung dua mode yaitu *review existing annotation* dengan referensi *annotation_id* dan *new issue mode* tanpa referensi *annotation* untuk *error* yang tidak tertangkap annotator dengan validasi bahwa *annotation* yang direview sudah disubmit oleh annotator.
- **Submit Review:** Reviewer harus dapat melakukan submit review yang akan mengubah semua reviews menjadi *is_submitted=true*, mengaplikasikan *corrections* ke *field corrected_text* pada *sentences* hanya review dengan *is_required=true*.
- **Review Queue:** Sistem harus menyediakan *endpoint* khusus untuk Reviewer yang menampilkan daftar *sentences* dalam dokumen yang memiliki *annotation* yang sudah disubmit dan perlu direview, termasuk informasi apakah reviewer sudah memberikan review, *next_sentence_id* untuk workflow, dan support parameter *include_unannotated* untuk mode *new_issue*.
- **Admin Reopen:** Admin dan Kepala Riset harus dapat membuka kembali pekerjaan

annotator atau reviewer pada dokumen tertentu dengan constraint tidak dapat *reopen* annotator jika reviewer sudah submit, set status kembali ke draft, dan catat alasan *reopen* oleh admin untuk *accountability*.

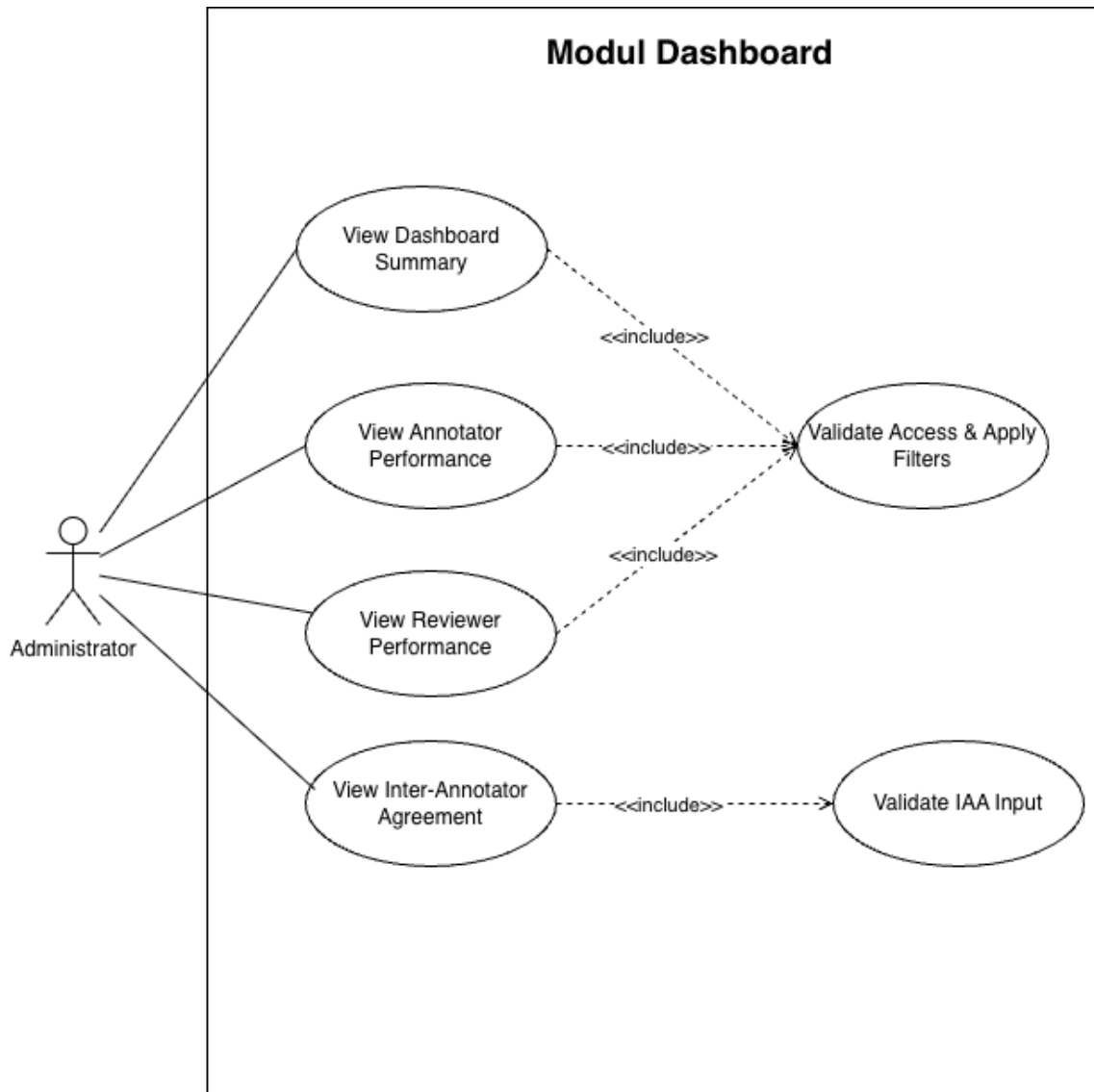
- **Assignment Status Validation:** Sistem harus memvalidasi setiap operasi anotasi/review terhadap status DocumentAssignment. *Assignment* harus aktif (*is_active=true*), jika sudah submit tidak dapat melakukan perubahan kecuali *direopen*, dan validasi bahwa dokumen diassign kepada *user* yang melakukan operasi.

4.2.6 Dashboard Statistik

Modul ini menyajikan ringkasan kinerja dalam bentuk dashboard, yang memiliki beberapa metrik utama yaitu performa Annotator, performa Reviewer, serta metrik *Inter-Annotator Agreement* (IAA). Setiap tampilan wajib melewati proses validasi akses dan penerapan filter, sementara perhitungan/tampilan IAA juga memvalidasi input IAA. Aktor utama yang memanfaatkan modul ini adalah Admin dan Kepala Riset untuk monitoring dan pengambilan keputusan. Gambar 4.6 mengilustrasikan seluruh *use case* pada modul *Dashboard*.

Dalam upaya memastikan informasi statistik yang relevan dan bermanfaat, sistem harus menyediakan *dashboard* untuk *monitoring* dan analisis performa anotasi. Fitur-fitur yang harus tersedia:

- **Dashboard Summary:** Sistem harus menampilkan ringkasan statistik keseluruhan meliputi total annotations, total reviews, per-annotator statistics (jumlah anotasi dan dokumen per annotator), per-reviewer statistics (jumlah reviews dan dokumen per reviewer), per-document statistics (jumlah anotasi per dokumen), dan inter-annotator agreement metrics dengan *support filtering* berdasarkan *project_id* dan *document_id*.
- **Annotator Performance Dashboard:** Admin dan Kepala Riset harus dapat melihat performa individual annotator tertentu termasuk total anotasi yang dibuat, *breakdown* per dokumen berupa jumlah anotasi per dokumen yang dikerjakan, dengan *filtering* berdasarkan *project* dan dokumen.
- **Reviewer Performance Dashboard:** Admin dan Kepala Riset harus dapat melihat performa individual reviewer tertentu termasuk total reviews yang dibuat, *breakdown* per dokumen, dengan *filtering* berdasarkan *project* dan dokumen.
- **Inter-Annotator Agreement (IAA):** Sistem harus menghitung Cohen's Kappa antara annotator dan reviewer untuk quality assurance dengan metodologi: menggu-



Gambar 4.6: Use Case Diagram Modul Dashboard

nakan span-based comparison (sentence_id, start_index, end_index, error_type_id), menghitung *agreement* pada *presence of error* dan *type of error*, menyediakan *detailed analysis* termasuk *perfect agreement spans*, *annotator only spans*, dan *reviewer-only spans*, serta menampilkan accuracy dan span agreement ratio.

- **Role Based Dashboard Access:** Admin hanya dapat mengakses *dashboard* untuk *project* yang diassign kepadanya, sedangkan Kepala Riset dapat mengakses semua *dashboard* tanpa filter dengan *automatic filtering* diterapkan di *query level*.
- **Real Time Metrics:** Semua metrics harus dihitung *real time* berdasarkan data terkini dalam *database* tanpa *caching* untuk memastikan akurasi data monitoring.

4.3 Kebutuhan Non-Fungsional

Kebutuhan non-fungsional merupakan syarat-syarat teknis yang harus dipenuhi oleh sistem *backend* agar dapat berfungsi dengan efisien dan lancar. Berikut adalah kebutuhan-kebutuhan non-fungsional yang telah diidentifikasi oleh tim penulis:

1. Keamanan

Sistem harus menjamin keamanan data dan akses pengguna. Untuk itu, *backend* menggunakan mekanisme *JSON Web Token* (JWT) untuk memverifikasi identitas pengguna. Setiap endpoint API dibatasi dengan *Role-Based Access Control* (RBAC) untuk memastikan hanya pengguna yang berwenang yang dapat mengakses sumber daya tertentu. Selain itu, seluruh komunikasi antara klien dan server dilakukan melalui HTTPS untuk melindungi data dari serangan *man-in-the-middle*.

2. Performa

Sistem harus mampu merespons permintaan dengan cepat dan efisien, bahkan ketika diakses secara bersamaan oleh banyak pengguna. Pengujian performa dilakukan untuk memastikan waktu respons tetap rendah dan stabil pada berbagai tingkat beban.

3. Maintainability

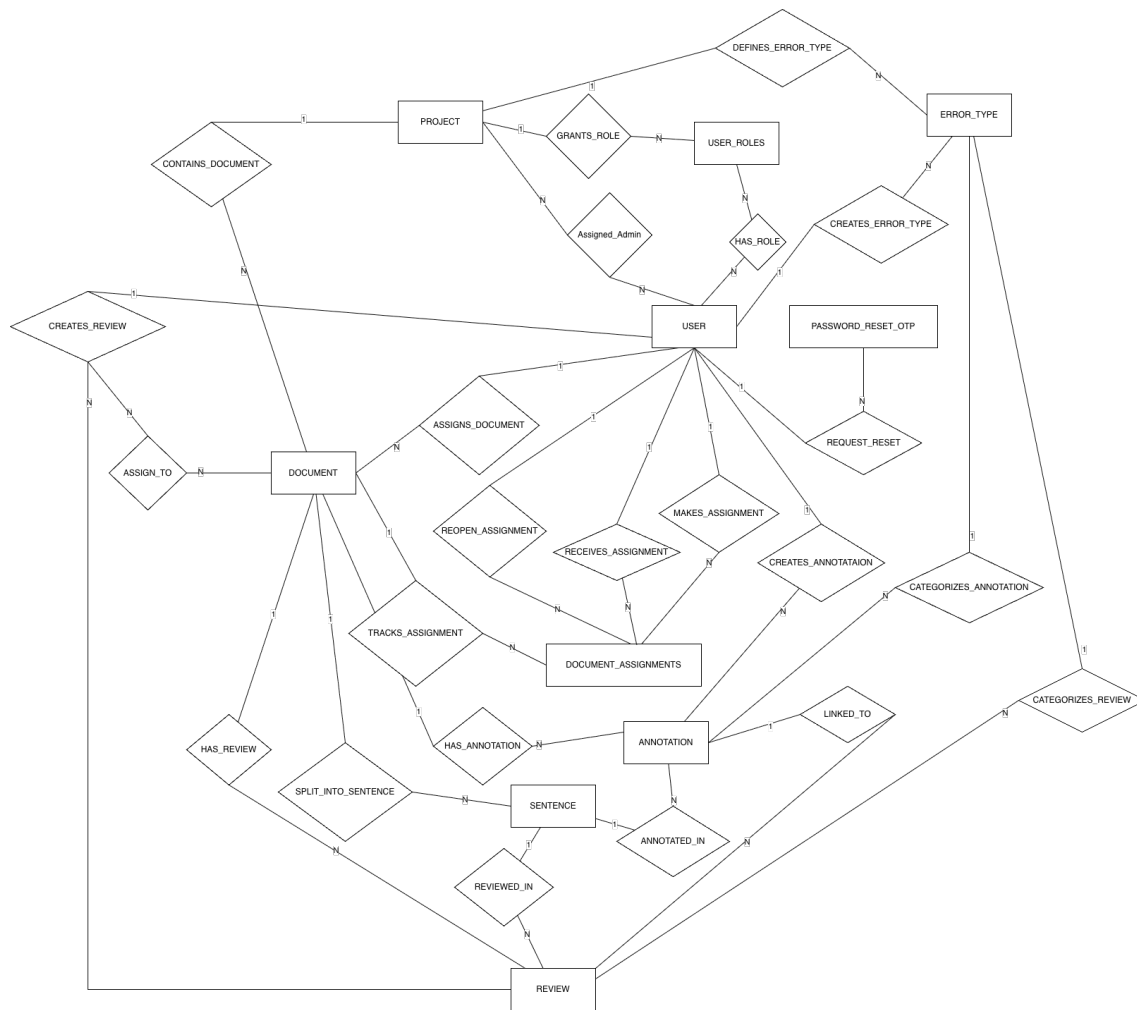
Sistem harus mudah dipelihara dan dikembangkan di kemudian hari. Kode *backend* ditulis mengikuti prinsip clean architecture dengan pemisahan tanggung jawab yang jelas antara Model, View, dan Serializer. Dokumentasi API disediakan menggunakan Swagger agar proses pengembangan lanjutan dapat dilakukan dengan cepat dan terarah.

4. Observability

Sistem harus memiliki kemampuan untuk dipantau dan dievaluasi kinerjanya secara *real-time*. Untuk itu, diterapkan integrasi antara Prometheus sebagai alat pengumpul metrik dan Grafana sebagai alat visualisasi data. Melalui observabilitas ini, performa sistem dapat dipantau secara berkelanjutan, sehingga tim pengembang dapat dengan cepat mendeteksi dan menangani anomali yang terjadi selama operasi sistem.

4.4 Analisis dan Perancangan Basis Data

Perancangan basis data pada sistem Anota akan digambarkan melalui *entity relationship diagram* pada Gambar 4.7. Terdapat sepuluh *Entity* dalam sistem Anota seperti USER, PROJECT, DOCUMENT, ANNOTATION, REVIEW, USER_ROLES, ERROR_TYPE, DOCUMENT_ASSIGNMENT, SENTENCE, dan PASSWORD_RESET_OTP. Selain



Gambar 4.7: Entity Relationship Diagram Anota

itu, Gambar 4.7 turut mengilustrasikan relasi-relasi yang tersedia serta kardinalitas dari relasi tersebut. Pada bagian ini, tidak digambarkan *attribute-attribute* dari masing-masing entitas dikarenakan adanya masalah keterbacaan. Penjelasan lengkap terkait masing-masing *attribute* dapat dilihat pada Tabel 4.2.

4.4.1 Entity Relationship Diagram

Secara keseluruhan, ERD pada Gambar 4.7 menunjukkan bahwa USER menjadi entitas utama dari hampir semua proses di Anota. Seorang pengguna dapat memiliki beberapa USER_ROLES seperti *annotator*, *reviewer*, atau *kepala riset* yang mengatur hak akses setiap pengguna di dalam sistem. Pengguna juga dapat membuat PROJECT, dan setiap proyek dapat berisi banyak DOCUMENT yang dikelola melalui relasi *CONTAINS_DOCUMENT*.

Setiap DOCUMENT dipecah menjadi banyak SENTENCE melalui relasi *SPLIT_INTO_SENTENCE*. Pada tingkat kalimat inilah proses anotasi dan peninjauan dilakukan: kalimat dapat memiliki satu atau lebih ANNOTATION (relasi *ANNOTATED_IN*) dan juga satu atau lebih REVIEW (relasi *REVIEWED_IN*). Baik anotasi maupun ulasan menggunakan indeks karakter (*start_index*, *end_index*) untuk menunjuk bagian teks yang dikoreksi, serta menyimpan informasi koreksi, komentar, dan status pengumpulan (*is_submitted*).

Tipe kesalahan yang digunakan ketika mengisi anotasi dimodelkan sebagai entity *ERROR_TYPE*, yang didefinisikan oleh pengguna tertentu dan kemudian dikaitkan dengan ANNOTATION melalui relasi *CATEGORIZES_ANNOTATION*. Penugasan dokumen ke annotator atau reviewer dikelola oleh entity *DOCUMENT_ASSIGNMENTS*, yang menyimpan status penugasan, tenggat waktu, catatan, serta riwayat pembukaan kembali (*reopened_at*, *reopen_reason*). Untuk mendukung keamanan akun, proses permintaan reset kata sandi dimodelkan sebagai *PASSWORD_RESET_OTP* yang menyimpan kode OTP, jumlah percobaan, status penggunaan, dan jejak waktu. Dengan demikian, seluruh skema basis data tidak hanya menangkap data teks dan anotasi, tetapi juga alur kerja penugasan, kontrol akses, dan mekanisme keamanan pengguna.

Tabel 4.2 merangkum pemetaan antara setiap *entity* utama pada basis data ANOTA dengan atribut atribut yang dimilikinya, seperti yang telah digambarkan pada ERD. Entity *USER* menyimpan informasi identitas dan status akun pengguna, *PROJECT* dan *DOCUMENT* merepresentasikan struktur proyek beserta dokumen yang dikelola, sedangkan *SENTENCE* mengurus pemecahan dokumen menjadi kalimat sebagai unit dasar anotasi. *ANNOTATION* dan *REVIEW* menyimpan detail proses anotasi dan peninjauan, termasuk posisi kesalahan, koreksi, dan metadata status, sementara *USER_ROLES* mengatur peran pengguna di dalam proyek. *ERROR_TYPE* mendefinisikan skema tipe kesalahan yang digunakan pada proses anotasi. *DOCUMENT_ASSIGNMENTS* merekam penugasan dokumen kepada annotator atau reviewer beserta status dan riwayatnya, dan *PASSWORD_RESET_OTP* menyimpan data terkait mekanisme *reset password* yang aman. Dengan demikian, tabel ini memberikan gambaran ringkas tentang struktur data yang digunakan untuk mendukung seluruh alur kerja anotasi dan peninjauan di sistem ANOTA.

Tabel 4.2: Mapping entity dan atribut pada basis data Anota

Entity	Atribut
USER	id, username, email, full_name, password, institusi, date_joined, is_deleted, deleted_at, is_kepala_riset, is_staff, last_login, is_superuser, is_active
PROJECT	id, name, description, created_at, updated_at
DOCUMENT	id, title, institution, text, hash_string, created_at, updated_at
SENTENCE	id, text, corrected_text, m2_text, has_error, created_at, updated_at
ANNOTATION	id, start_index, end_index, correction, is_required, comments, is_submitted, created_at, updated_at
REVIEW	id, start_index, end_index, correction, is_required, comments, is_submitted, created_at, updated_at
USER_ROLES	id, role
ERROR_TYPE	id, error_code, description, is_default, created_at, updated_at
DOCUMENT_ASSIGNMENTS	id, deadline, notes, assigned_at, is_active, is_submitted, submitted_at, reopened_at, reopen_reason
PASSWORD_RESET_OTP	id, otp_code, attempt_count, is_used, used_at, created_at, updated_at

4.4.2 Skema Relasional

Dalam upaya memperjelas skema basis data yang ada, selanjutnya dilakukan perancangan skema relational dari ERD yang telah dibuat.

Gambar 4.8 merupakan pemetaan dari ERD ke dalam tabel relasional. Entitas utama pada sistem Anota terdiri atas tabel **USER**, **PROJECT**, dan **DOCUMENT**. Tabel **USER** menyimpan informasi akun seperti username, email, password, nama lengkap, institusi, serta berbagai *flag status* (*is_kepala_riset*, *is_staff*, *is_active*, dan *is_superuser*) beserta *timestamp* waktu seperti (*date_joined*, *last_login*, dan *deleted_at*). Setiap pengguna dapat terlibat pada banyak proyek melalui tabel **USERROLES** yang menghubungkan **USER** dan **PROJECT** dengan atribut *role*, sehingga satu pengguna dapat memiliki peran yang berbeda pada proyek yang berbeda. Selain itu, tabel **PROJECT_ASSIGNED_ADMINS** memetakan relasi many-to-many antara **PROJECT** dan **USER** untuk menandai siapa saja yang berperan sebagai admin pada proyek tersebut. Tabel **PASSWORDRESETOTP** menampung data permintaan *reset* kata sandi berupa kode OTP, waktu kedaluwarsa, status penggunaan, serta

dan *description*) beserta informasi waktu pembuatan dan pembaruan. Setiap proyek dapat berisi banyak dokumen yang direpresentasikan oleh tabel `DOCUMENT`. Tabel ini menyimpan metadata dokumen seperti `hash_string`, `title`, `institution`, *raw text* (`text`), serta *foreign key* ke `PROJECT` dan `USER` yang melakukan *assignment* melalui variabel *assigned by*. Relasi penugasan dokumen kepada pengguna dikelola melalui tabel penghubung `DOCUMENT_ASSIGNED_TO` dan `USER_ASSIGNED_DOCUMENTS` yang masing-masing menghubungkan dokumen ke pengguna dan sebaliknya, sehingga sistem dapat melacak dokumen apa saja yang dimiliki seorang pengguna dan kepada siapa suatu dokumen ditugaskan. Riwayat dan status penugasan tersebut direkam lebih rinci pada tabel `ASSIGNMENT_TRACKING` yang mencatat siapa yang ditugaskan dan siapa yang menugaskan, waktu penugasan, status aktif atau sudah disubmit, *deadline*, proses *reopen*, serta alasan perubahan. Di sisi lain, isi dokumen dipecah menjadi unit yang lebih kecil berupa kalimat dan disimpan pada tabel `SENTENCE` yang memiliki *foreign key* ke `DOCUMENT` dan atribut seperti `text`, `m2_text`, `corrected_text`, dan `has_error`.

Alur anotasi dan peninjauan kualitas data dimodelkan melalui tabel `ERRORTYPE`, `ANNOTATION`, dan `REVIEW`. Tabel `ERRORTYPE` menyimpan daftar jenis kesalahan yang didefinisikan per proyek (`project_id`) dengan kode error, deskripsi, dan penanda apakah jenis tersebut merupakan *default*, serta *foreign key* ke `USER` yang membuatnya. Proses anotasi dilakukan pada tabel `ANNOTATION` yang menghubungkan `USER` anotator, `DOCUMENT`, dan `SENTENCE` dengan informasi posisi kesalahan (`start_index`, `end_index`), jenis kesalahan (`error_type_id`), teks perbaikan, komentar, serta status submit. Proses peninjauan berikutnya direpresentasikan oleh tabel `REVIEW` yang memiliki struktur serupa, namun mengaitkan `USER` sebagai reviewer dengan anotasi yang diperiksa (`annotation_id`) sehingga sistem dapat merekam koreksi, persetujuan, ataupun perbedaan pendapat terhadap anotasi awal.

4.5 Rancangan Modul-Modul Anota

Bagian ini membahas mengenai rancangan dari modul-modul pada sistem ANOTA yang akan dijelaskan melalui rancangan *class diagram* dari sistem yang telah dibuat. Salah satu bagian utama dari perancangan sistem pada SDLC adalah merancang bagaimana masing-masing *class* pada sistem saling terhubung. Gambar 4.9, Gambar 4.10, Gambar 4.11, Gambar 4.12, Gambar 4.13, dan Gambar 4.14 mengilustrasikan contoh implementasi

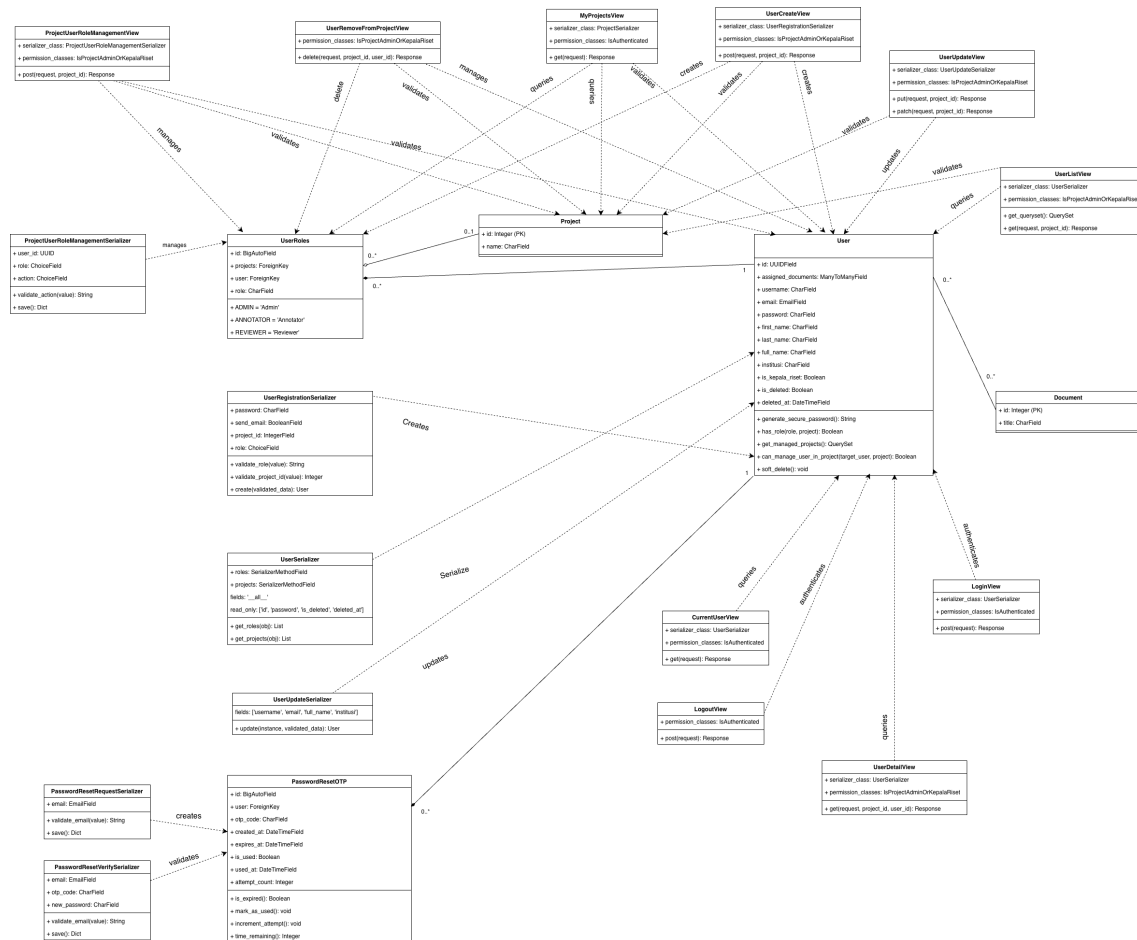
class-class yang ada pada sistem Anota. Masing-masing *class* berperan dalam berbagai modul seperti berikut:

4.5.1 Modul *User*

Gambar 4.9 menunjukkan implementasi yang ada pada modul *user* yang berfokus dalam menjalankan *authentikasi* dan *otorisasi*, serta berperan sebagai entitas utama yang akan mengelola keseluruhan sistem. Pada modul ini terdapat kelas *User* yang merepresentasikan entitas pengguna sistem. Kelas ini menyimpan atribut utama seperti *username*, *email*, *password*, *nama lengkap*, serta informasi institusi pengguna. Selain itu, terdapat sejumlah atribut logika seperti *is_kepala_riset*, *is_deleted*, dan *deleted_at* yang digunakan untuk membedakan peran khusus Kepala Riset dan mengimplementasikan mekanisme *soft delete* terhadap akun pengguna. Relasi antara pengguna dan proyek dimodelkan melalui kelas *Project* dan *UserRoles*. Kelas *UserRoles* bertindak sebagai *association class* yang menghubungkan satu *User* dengan satu *Project* dengan atribut *role* yang dapat berupa *ADMIN*, *ANNOTATOR*, atau *REVIEWER*. Sistem Anota dirancang untuk mendukung *multiple role* bagi pengguna di berbagai proyek yang berbeda. Modul ini juga menyediakan relasi terhadap kelas *Document* untuk mengatur dokumen anotasi yang ditugaskan kepada pengguna, serta kelas *PasswordResetOTP* yang menyimpan kode OTP, waktu kedaluwarsa, dan status penggunaan.

Bagian selanjutnya adalah *serializer* yang berfungsi untuk menjembatani representasi data pada model dengan API. *UserRegistrationSerializer* digunakan saat pembuatan akun baru di dalam suatu proyek dengan menerima data seperti *password*, *project_id*, dan *role*, kemudian melakukan validasi sebelum membuat objek *User* beserta entri *UserRoles* yang sesuai. Proses pembacaan data ditangani oleh *UserSerializer* yang mengembalikan detail akun pengguna sekaligus daftar peran dan proyek yang dimilikinya, sementara *UserUpdateSerializer* membatasi atribut yang boleh diubah saat pengguna memperbarui profil. Manajemen peran pada suatu proyek diatur melalui *ProjectUserRoleManagementSerializer*, sedangkan alur permintaan dan verifikasi OTP pada fitur *forgot password* ditangani oleh *PasswordResetRequestSerializer* dan *PasswordResetVerifySerializer*.

Pada bagian *view*, modul ini mengimplementasikan beberapa kelas *class-based view* Django REST Framework yang memanfaatkan pola MVT. *UserCreateView*,



Gambar 4.9: Class Diagram Modul User

UserListView, UserDetailView, dan UserUpdateView bertanggung jawab terhadap operasi pembuatan, pembacaan, dan pembaruan data pengguna dalam konteks suatu proyek, dengan *permission class* seperti *IsAuthenticated* dan *IsProjectAdminOrKepalaRiset* yang membatasi hak akses *user*. ProjectUserRoleManagementView dan UserRemoveFromProjectView mengatur penambahan dan penghapusan peran pengguna dari proyek. Sementara itu, LoginView, LogoutView, dan CurrentUserView menangani proses autentikasi serta pengambilan informasi akun yang sedang aktif. Kombinasi antara model, serializer, dan view ini memastikan bahwa proses *autentikasi* dan *otorisasi* pada modul *user* berjalan konsisten, terstruktur, dan selaras dengan arsitektur MVT pada Django REST Framework.

4.5.2 Modul *Project*

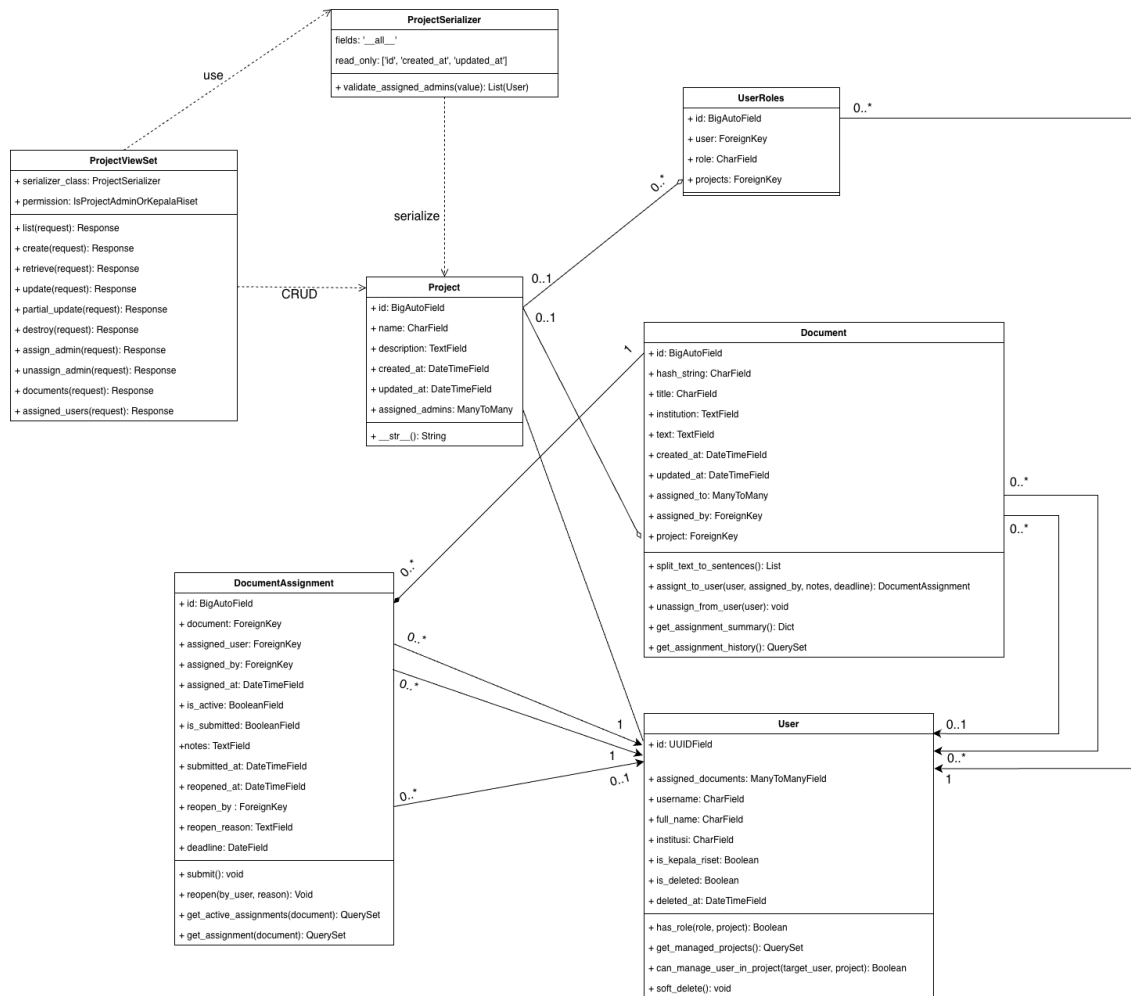
Gambar 4.10 menunjukkan modul *project* yang berperan dalam mengelola entitas proyek dan mengelompokkan dokumen dan peran pengguna pada sistem Anota. Kelas `Project` menyimpan informasi identitas proyek seperti `name`, `description`, serta atribut waktu `created_at` dan `updated_at`. Setiap proyek dapat memiliki banyak dokumen melalui relasi `ForeignKey` dari kelas `Document` ke `Project`. Selain itu, `Project` juga memiliki relasi `ManyToMany` ke `User` melalui atribut `assigned_admins`, yang merepresentasikan daftar pengguna yang dapat mengelola proyek tersebut. Kelas `UserRoles` menghubungkan `User` dengan `Project` melalui atribut `role`, sehingga satu pengguna bisa berperan sebagai `ADMIN`, `ANNOTATOR`, maupun `REVIEWER` pada proyek yang berbeda. Selain itu, terdapat `DocumentAssignment` yang berperan dalam memastikan setiap penugasan dokumen selalu berada dalam konteks satu proyek tertentu.

Pada bagian *serializer*, modul ini menggunakan `ProjectSerializer` untuk melakukan serialisasi dan validasi data proyek. `Serializer` ini mencakup semua field proyek, namun menjadikan `id`, `created_at`, dan `updated_at` sebagai *read-only*. Fungsi `validate_assigned_admins()` digunakan untuk memastikan bahwa daftar admin proyek berisi pengguna yang valid dan tidak terjadi duplikasi.

Pada bagian *view*, modul ini memanfaatkan `ProjectViewSet` yang mengimplementasikan operasi *CRUD* terhadap entitas proyek (`list`, `retrieve`, `create`, `update`, dan `destroy`). Selain operasi dasar, `ProjectViewSet` juga menyediakan *endpoint* khusus seperti `assign_admin` dan `unassign_admin` untuk mengatur admin proyek, `documents` untuk mengambil daftar dokumen di dalam proyek, serta `assigned_users` untuk melihat pengguna yang terlibat. Seluruh *endpoint* dilindungi oleh *permission class* `IsProjectAdminOrKepalaRiset`, sehingga hanya pengguna yang memiliki *role* `admin project` atau `kepala riset` saja yang dapat memodifikasi data proyek.

4.5.3 Modul *Document*

Gambar 4.11 menggambarkan implementasi modul *document* yang berfungsi untuk mengelola dokumen anotasi, penugasan dokumen kepada pengguna, serta memecah dokumen menjadi kalimat-kalimat yang akan dianotasi. Pada bagian model, beberapa kelas utama yang terlibat adalah `Document`, `DocumentAssignment`, dan `Sentence`. Kelas



Gambar 4.10: Class Diagram Modul Project

Document menyimpan atribut seperti `hash_string`, `title`, `institution`, dan `text` yang berisi *raw document*, serta informasi waktu pembuatan dan pembaruan. Setiap Document terhubung ke satu Project dan dapat ditugaskan kepada banyak User melalui relasi `ManyToMany` yang difasilitasi oleh kelas `DocumentAssignment`. Kelas `DocumentAssignment` berfungsi untuk menyimpan detail penugasan seperti pengguna yang ditugaskan (`assigned_user`), pemberi tugas (`assigned_by`), status aktif dan submit, catatan penugasan, waktu submit maupun *reopen*, serta deadline. Sementara itu, kelas `Sentence` merepresentasikan unit terkecil anotasi berupa kalimat dalam suatu dokumen dengan atribut `text`, `corrected_text`, *flag* `has_error`, serta relasi `ForeignKey` ke Document. Metode seperti `split_text_to_sentences()`, `assign_to_user()`, dan `get_assignment_history()` pada Document mengenkapsulasi logika bisnis untuk memecahan teks, pembuatan penugasan baru, dan pelacakan histori penugasan dokumen.



Pada bagian *serializer*, `DocumentSerializer` menangani serialisasi umum objek `Document`, termasuk validasi ketika dokumen baru dibuat atau diperbarui. `AdminDocumentListSerializer` dan `AdminDocumentDetailSerializer` digunakan untuk tampilan daftar dan detail dokumen pada sisi admin, dengan tambahan `SerializerMethodField` untuk menampilkan status dokumen, ringkasan penugasan, dan jumlah penugasan aktif. `MyAssignedDocumentSerializer` digunakan untuk menampilkan dokumen yang sedang ditugaskan kepada pengguna yang sedang login, termasuk informasi progres jumlah kalimat yang sudah dianotasi. Detail setiap penugasan

direpresentasikan oleh `DocumentAssignmentSerializer` yang menghubungkan antara `Document` dan `User`, sedangkan `SentenceSerializer` digunakan untuk melakukan serialize data setiap kalimat yang menjadi objek anotasi.

Bagian *view* pada modul *document* diimplementasikan menggunakan *class-based view* Django REST Framework. `DocumentRecordView` dan `DocumentDetailView` menyediakan operasi pembacaan terhadap daftar dokumen serta detail sebuah dokumen. Alur penugasan diatur melalui `AssignDocumentView` dan `UnassignDocumentView` yang memanfaatkan `DocumentAssignmentSerializer` untuk membuat, memperbarui, atau mencabut penugasan dokumen kepada pengguna. Riwayat penugasan dapat diakses melalui `DocumentAssignmentHistoryView`, sedangkan `ReopenDocumentSubmissionView` digunakan untuk membuka kembali penugasan yang sudah disubmit dengan alasan tertentu. Selain itu, terdapat `AssignedUsersListView` untuk menampilkan daftar pengguna yang terlibat pada suatu dokumen, `SentenceCountPreviewView` untuk melihat jumlah kalimat, serta `DocumentExportView` yang mengeksport hasil anotasi ke format yang dibutuhkan. Seluruh *view* ini dikaitkan dengan *permission class* yang sesuai (misalnya `IsAdmin`) sehingga hanya peran tertentu yang dapat melakukan operasi administratif pada dokumen.

4.5.4 Modul *Annotation*

Gambar 4.12 menggambarkan *class diagram* untuk modul *annotation* yang menangani proses peninjauan kualitas anotasi pada sistem ANOTA. Pada bagian model, kelas utama yang terlibat adalah `Annotation` yang merepresentasikan hasil peninjauan terhadap suatu anotasi. Setiap entri `Annotation` terhubung dengan `User` dengan *role annotator*, `Review` yang meninjau anotasi itu sendiri, `Sentence` dan `Document` tempat kesalahan ditemukan, serta `ErrorType` yang menjadi kategori kesalahan. Sistem turut mencatat secara rinci bagian mana yang dikoreksi (melalui `start_index` dan `end_index`), jenis kesalahan, serta status apakah anotasi telah disimpan apa belum. Relasi ini tetap berada dalam konteks `Project` dan `DocumentAssignment` sehingga hanya dokumen yang memang ditugaskan kepada *annotator* tertentu yang dapat dilihat oleh yang bersangkutan.

`Annotation` terhubung dengan `User` dengan *role annotator*, `Review` yang meninjau anotasi itu sendiri, `Sentence` dan `Document` tempat kesalahan ditemukan, serta `ErrorType` yang menjadi kategori kesalahan. Sistem turut mencatat secara rinci bagian mana yang dikoreksi (melalui `start_index` dan `end_index`), jenis kesalahan, serta status apakah



4.5.5 Modul Review

Bagian *serializer* direpresentasikan oleh `AnnotationSerializer` yang bertugas memvalidasi dan mengubah objek `Annotation` menjadi format JSON untuk dikirim melalui API. `Serializer` ini memanfaatkan `AnnotationValidationMixin` yang berisi logika validasi antara modul anotasi dan review, misalnya pemeriksaan konsistensi indeks kalimat, kecocokan `ErrorType` dengan proyek, atau pengecekan apakah kalimat dan dokumen yang direview memang berada dalam proyek yang sama. Dengan pendekatan *mixin*, aturan validasi kritis hanya didefinisikan satu kali namun dapat digunakan kembali oleh beberapa `serializer` atau *view* yang membutuhkan.

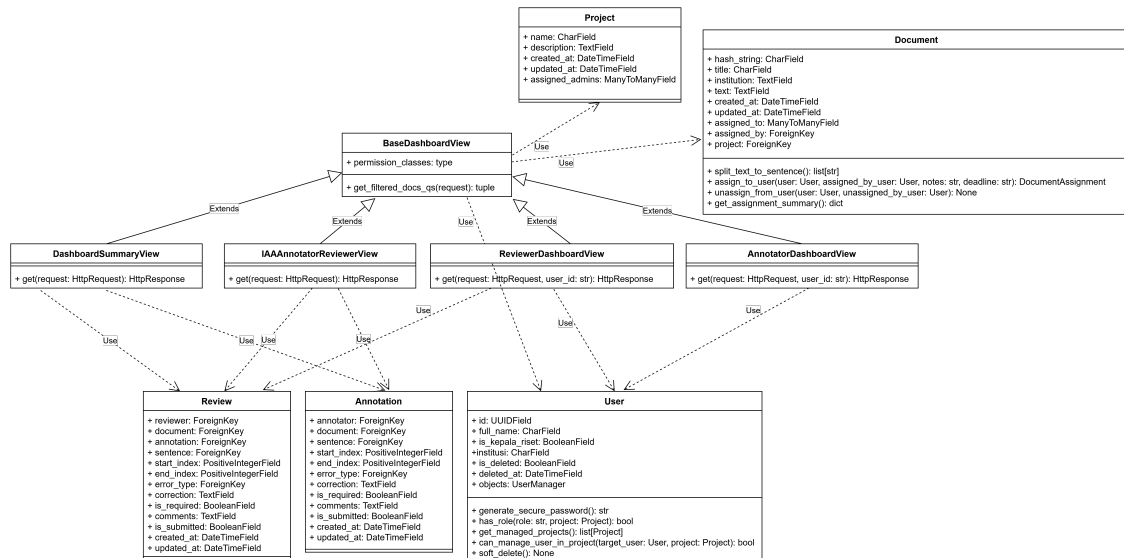
Pada bagian *view*, modul ini menyediakan beberapa *class-based view* yang memanfaatkan *AnnotationSerializer* dan kelas *permission* khusus. *AnnotationListCreateView* digunakan untuk menampilkan daftar annotation dan membuat annotation baru, sedangkan *AnnotationRetrieveUpdateDestroyView* menangani operasi membaca detail, memperbarui, dan menghapus review tertentu. Kedua *view* ini menggunakan *IsReviewer* untuk memastikan hanya pengguna dengan peran *annotator* yang dapat mengakses *endpoint*, serta *IsOwnerOfAnnotation* agar hanya pemilik anotasi atau pihak berwenang tertentu yang boleh memodifikasi data. Sementara itu, *AnnotatorSubmitDocumentView* menangani proses ketika reviewer menyelesaikan peninjauan sebuah dokumen dan menandai status penugasan pada *DocumentAssignment* sebagai *submitted*. Jika diperlukan pembukaan kembali proses anotasi, *AnnotatorReopenDocumentView* dan *AdminReopenAnnotatorDocumentView* digunakan untuk mengubah status penugasan (baik oleh *annotator* maupun admin), sekaligus menyimpan alasan *reopen*.

Gambar 4.13 menggambarkan *class diagram* untuk modul *review* yang menangani proses peninjauan kualitas anotasi pada sistem Anota. Pada bagian model, kelas utama yang terlibat adalah *Review* yang merepresentasikan hasil peninjauan terhadap suatu anotasi. Setiap entri *Review* terhubung dengan *User* dengan *role reviewer*, *Annotation* yang ditinjau, *Sentence* dan *Document* tempat kesalahan ditemukan, serta *ErrorType* yang menjadi kategori kesalahan. Sistem turut mencatat secara rinci bagian mana yang dikoreksi (melalui *start_index* dan *end_index*), jenis kesalahan, usulan perbaikan, komentar, serta status pengiriman hasil *review*. Relasi ini tetap berada dalam konteks *Project* dan *DocumentAssignment* sehingga hanya dokumen yang memang ditugaskan kepada *reviewer* tertentu yang dapat diulas.

Bagian *serializer* direpresentasikan oleh *ReviewSerializer* yang bertugas memvalidasi dan mengubah objek *Review* menjadi format JSON untuk dikirim melalui API. *Serializer* ini memanfaatkan *AnnotationValidationMixin* yang berisi logika validasi antara modul anotasi dan *review*, misalnya pemeriksaan konsistensi indeks kalimat, kecocokan *ErrorType* dengan proyek, atau pengecekan apakah kalimat dan dokumen yang direview memang berada dalam proyek yang sama. Dengan pendekatan *mixin*, aturan validasi kritis hanya didefinisikan satu kali namun dapat digunakan kembali oleh beberapa *serializer* atau *view* yang membutuhkan.



Pada bagian *view*, modul ini menyediakan beberapa *class-based view* yang memanfaatkan `ReviewSerializer` dan kelas *permission* khusus. `ReviewListCreateView` digunakan untuk menampilkan daftar *review* dan membuat *review* baru, sedangkan `ReviewRetrieveUpdateDestroyView` menangani operasi membaca detail, memperbarui, dan menghapus *review* tertentu. Kedua *view* ini menggunakan `IsReviewer` untuk memastikan hanya pengguna dengan peran *reviewer* yang dapat mengakses *endpoint*, serta `IsOwnerOfReview` agar hanya pemilik *review* atau pihak berwenang tertentu yang boleh memodifikasi data. Sementara itu, `ReviewerSubmitDocumentView` menangani proses ketika *reviewer* menyelesaikan peninjauan sebuah dokumen dan menandai status penugasan pada `DocumentAssignment` sebagai *submitted*. Jika diperlukan pembukaan kembali proses *review*, `ReviewerReopenDocumentView` dan `AdminReopenReviewerDocumentView` digunakan untuk mengubah status penugasan (baik oleh *reviewer* maupun *admin*), sekaligus menyimpan alasan *reopen*.



Gambar 4.14: Class Diagram Modul Dashboard

4.5.6 Modul Dashboard

Gambar 4.14 menunjukkan *class diagram* untuk modul *dashboard* yang berfungsi menyajikan ringkasan progres anotasi dan kualitas data pada sistem Anota. Pada bagian ini terdapat entitas utama seperti *Project* dan *Document*, yang menjadi sumber data agregasi untuk berbagai tampilan ringkasan. Selain itu terdapat juga entitas *Review*, *Annotation*, dan *User* yang menyimpan informasi detail aktivitas anotator maupun *reviewer*. Seluruh entitas ini dimanfaatkan oleh kelaskelas pada *view* untuk menghitung metrik dan menampilkan informasi yang relevan di *dashboard*.

Terdapat *BaseDashboardView* yang merepresentasikan *class-based view* dasar pada Django REST Framework. Kelas ini menggunakan (*use*) data dari *Project* dan *Document*, serta menyimpan logika umum seperti pengambilan konteks proyek, *filtering* dokumen, dan pola perhitungan metrik dasar. Empat kelas turunan, yaitu *DashboardSummaryView*, *IAAAnnotatorReviewerView*, *ReviewerDashboardView*, dan *AnnotatorDashboardView*, masing-masing *extends* *BaseDashboardView* sehingga dapat mewarisi mekanisme pengambilan data yang sama namun menambahkan perhitungan dan representasi metrik yang lebih spesifik.

DashboardSummaryView menggunakan data dari *Review*, *Annotation*, dan *User* untuk menampilkan ringkasan umum progres proyek, seperti jumlah dokumen, jumlah kalimat yang sudah dianotasi, serta tingkat penyelesaian anotasi. *IAAAnnotatorReviewerView*

memanfaatkan entitas Annotation, Review, dan User untuk menghitung metrik *Inter-Annotator Agreement* antara anotator dan *reviewer*. ReviewerDashboardView berfokus pada kinerja dan beban kerja. *reviewer* dengan mengolah data dari Review dan User, sedangkan AnnotatorDashboardView menyajikan progres anotasi per pengguna berdasarkan data Annotation dan User.

4.6 Rancangan *Application Programming Interface* (API)

Tahap perencanaan selanjutnya adalah dengan membuat rancangan *Application Programming Interface* yang berfokus pada merancang seluruh *endpoint* yang dibutuhkan dalam pengembangan. Perancangan ini melibatkan penentuan spesifikasi dari masing-masing *endpoint* seperti *header*, *body*, dan *response status*. Pada proses ini dihasilkan sebanyak 62 *endpoint* yang mendukung modul-modul pada sistem ANOTA. Berikut merupakan contoh dari perancangan salah satu *endpoint* ANOTA, yaitu login.

```

1      # URL:
2      POST /api/v1/login/
3
4      # Headers:
5      Content-Type: application/json
6      Accept: application/json
7
8      # Body:
9      {
10         "username": "string",
11         "password": "string"
12     }
13
14     # Example Body:
15     {
16         "username": "john_doe",
17         "password": "mySecurePassword123"
18     }
19
20     # Status 200 OK - Login berhasil:
21     {
22         "access": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9...",
23         "refresh": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
24     }
25
26     # Status 400 Bad Request:
27     {
28         "username": ["This field is required."],
29         "password": ["This field is required."]
30     }
31

```

```
32     # Status 401 Unauthorized:
33     {
34         "detail": "No active account found with the given credentials."
35     }
36
```

Kode 4.1: Login Endpoint Complete Request

Seluruh rancangan *endpoint* yang dikembangkan kemudian akan digunakan pada proses implementasi. List terkait *endpoint* yang tersedia dapat dilihat pada Lampiran 3.

BAB 5

IMPLEMENTASI

Pada bab ini, tim penulis akan memaparkan aspek-aspek teknis mengenai implementasi dari proyek ini yang mencakup pemilihan teknologi, penyusunan arsitektur sistem, implementasi *core business logic*, dan konfigurasi *deployment*.

5.1 Pemilihan Teknologi

Pada bagian ini, tim penulis akan menjelaskan teknologi apa saja yang digunakan dalam pengembangan sistem *backend* dari aplikasi ANOTA. Berikut adalah teknologi-teknologi yang digunakan untuk mengembangkan sistem *backend* dari aplikasi ANOTA:

- Python

Python digunakan sebagai bahasa pemrograman untuk pengembangan *backend* aplikasi ini. Bahasa ini mendukung berbagai paradigma seperti *procedural*, *object-oriented*, dan *functional programming*. Python populer karena memiliki ekosistem *library* yang luas dan komunitas pengembang yang aktif, sehingga memudahkan proses pengembangan aplikasi¹.

- Django

Django merupakan *web framework* berbasis Python yang digunakan untuk membangun sistem *backend* pada penelitian ini. *Framework* ini dipilih karena memiliki fitur yang lengkap, dokumentasi yang baik, serta dukungan komunitas yang aktif dengan pembaruan rutin yang meningkatkan keamanan dan stabilitas sistem². Selain itu, Django dilengkapi dengan *Object Relational Mapping* (ORM) yang mempermudah pengelolaan data pada basis data. Untuk mempermudah pembuatan API, Django diperluas dengan Django REST Framework (DRF)³. DRF menyediakan komponen tambahan seperti *Serializer*, *ViewSet*, dan *Router* yang memudahkan pengembangan layanan *RESTful* API secara efisien dan terstruktur.

- PostgreSQL

PostgreSQL merupakan *database management system* (DBMS) yang digunakan sebagai basis data untuk menyimpan informasi pada aplikasi ANOTA. PostgreSQL termasuk

¹<https://www.python.org/about/apps/>

²<https://www.djangoproject.com/start/overview/>

³<https://www.django-rest-framework.org/>

dalam kategori *relational database* yang mendukung integritas dan konsistensi data melalui hubungan antar tabel. Untuk meningkatkan performa, koneksi dengan PostgreSQL akan ditengahi oleh PgBouncer⁴ yang melakukan manajemen koneksi basis data yang meminimalkan konsumsi sumber daya pada sistem. Dalam pengembangan *backend* pada penelitian ini, PostgreSQL dipilih sebagai sistem basis data utama untuk menggantikan basis data *default* Django, yaitu SQLite3⁵. Pemilihan PostgreSQL didasarkan pada kemampuannya dalam menangani volume data yang lebih besar, mendukung transaksi kompleks, serta kompatibel penuh dengan *Object-Relational Mapping* (ORM) milik Django⁶.

- Git

Git⁷ digunakan sebagai sistem *version control* untuk mengelola *source code backend*. Penggunaan Git memudahkan kolaborasi antar anggota tim dalam proses pengembangan.

- Docker

Docker digunakan sebagai alat *containerization* untuk memastikan kode *backend* dapat dijalankan secara konsisten di berbagai lingkungan pengembangan dan produksi secara terisolasi⁸. Dengan Docker, seluruh dependensi dan konfigurasi sistem dikemas ke dalam *container*, sehingga mempermudah proses *deployment*, meningkatkan keamanan pada sistem, dan mengurangi potensi perbedaan konfigurasi antar lingkungan.

Selain itu, penggunaan Docker Compose memungkinkan pengelolaan beberapa layanan seperti *backend* Django, basis data PostgreSQL, dan layanan pendukung lainnya dalam satu konfigurasi terintegrasi⁹. Pendekatan ini meningkatkan efisiensi pengujian serta mempermudah replikasi lingkungan sistem oleh anggota tim pengembang.

- Caddy

Caddy digunakan sebagai *web server* sekaligus *reverse proxy* untuk mengelola lalu lintas jaringan antara klien dan *server backend*. Caddy dipilih karena memiliki konfigurasi yang sederhana serta mendukung otomatisasi sertifikat HTTPS melalui integrasi bawaan dengan Lets Encrypt, sehingga meningkatkan keamanan komunikasi data¹⁰.

⁴<https://www.pgбouncer.org/>

⁵<https://sqlite.org/>

⁶<https://www.postgresql.org/about/>

⁷<https://git-scm.com/about>

⁸<https://www.docker.com/>

⁹<https://docs.docker.com/compose/>

¹⁰<https://caddyserver.com/>

Selain itu, Caddy digunakan untuk meneruskan permintaan (*request forwarding*) ke layanan *backend* Django yang berjalan di dalam *container* Docker. Dengan fitur manajemen koneksi yang efisien dan dukungan *load balancing*, Caddy membantu menjaga kinerja sistem tetap stabil serta mempermudah proses *deployment* dalam lingkungan produksi.

- Prometheus

Prometheus digunakan sebagai alat *monitoring* dan *metrics collection* yang berfungsi untuk mengumpulkan serta menyimpan data performa dari sistem *backend* ¹¹. Prometheus secara berkala mengambil (*scrape*) metrik dari berbagai layanan, seperti penggunaan sumber daya *server*, waktu respons API, dan beban basis data, untuk kemudian dianalisis dan divisualisasikan melalui Grafana.

- Grafana

Grafana digunakan sebagai alat *monitoring* dan visualisasi data untuk memantau kinerja sistem *backend* selama proses pengujian dan implementasi. Grafana mempermudah analisis metrik seperti penggunaan CPU, memori, waktu respons API, serta performa basis data ¹².

- Pytest

Pytest digunakan sebagai *testing framework* untuk melakukan pengujian otomatis pada sistem *backend*. Alat ini mendukung berbagai jenis pengujian seperti *unit test* dan *regression test* dengan sintaks yang sederhana serta dukungan *fixture* untuk penyiapan data uji ¹³.

- VSCode

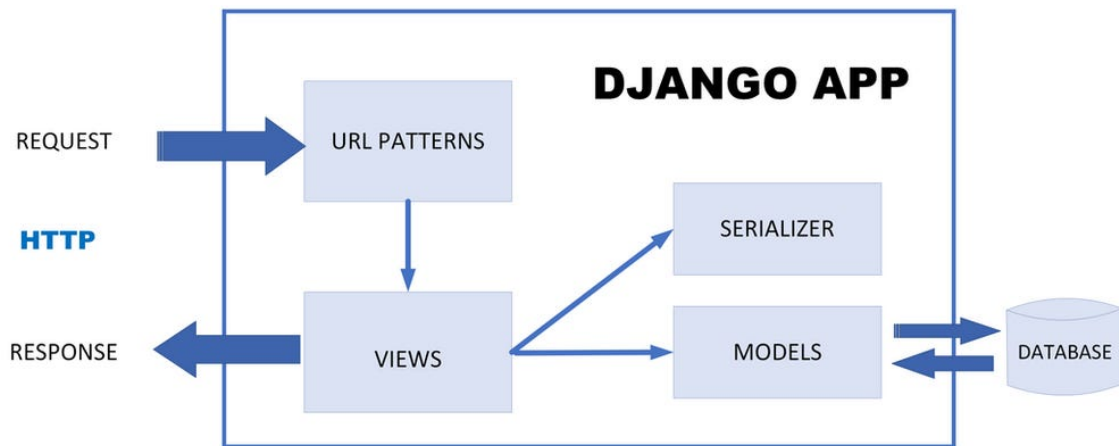
Visual Studio Code digunakan sebagai text editor utama dalam penulisan kode *backend*. Aplikasi ini dipilih karena memiliki banyak *extension* yang mendukung proses pengembangan, seperti integrasi dengan *version control*, *linting*, serta *debugging*. Selain itu, Visual Studio Code mendukung berbagai bahasa pemrograman, sehingga memudahkan pengembang dalam mengelola proyek secara efisien ¹⁴.

¹¹<https://prometheus.io/>

¹²<https://grafana.com/>

¹³<https://docs.pytest.org/en/stable/>

¹⁴<https://code.visualstudio.com/>



Gambar 5.1: Arsitektur dari Django Rest Framework

Sumber: python.plainenglish.io/part-3-exploring-serializers-and-views-in-django-rest-framework-class-based-view-f96ceafd10f9

5.2 Arsitektur Sistem

Struktur arsitektur sistem *backend* dibangun menggunakan arsitektur *Model-View-Template* (MVT) dari Django yang telah ditambahkan dengan ekstensi Django REST Framework (DRF). Struktur ini dirancang secara modular dengan setiap komponen memiliki tanggung jawabnya masing-masing yang terpisah secara jelas dan dikembangkan secara terpisah. Susunan arsitektur sistem DRF dapat dilihat pada Gambar 5.1. Berikut penjelasan lebih lanjut mengenai setiap komponen yang ada pada arsitektur:

1. Model

Komponen *Model* berfungsi untuk merepresentasikan struktur dan berperan sebagai perantara interaksi dengan basis data melalui *Object-Relational Mapping* (ORM) dari Django. Setiap entitas pada basis data seperti *User*, *Document*, *Project*, dan seterusnya, direpresentasikan menjadi kelas-kelas yang didefinisikan di dalam `models.py`. *Model* bertanggung jawab untuk menentukan atribut dan tipe data setiap entitas, menjalankan logika validasi dan pemrosesan data, serta mengatur relasi antar entitas menggunakan `ForeignKey`, `OneToOneField`, `ManyToManyField`.

2. View

Komponen *View* bertanggung jawab untuk menangani panggilan HTTP yang dikirimkan dari *browser*. *View* didefinisikan di dalam `views.py` dan berfungsi untuk mengelola alur logika, validasi input, pemrosesan data, mengecek izin akses berdasarkan *role* dari pengguna (*Role-Based Access Control*), serta manipulasi basis data melalui

pemanggilan fungsi-fungsi model.

3. *Serializer*

Serializer digunakan untuk mengonversi data dari objek *Model* menjadi format JSON yang siap dikirim melalui API ke *browser*. *Serializer* diimplementasikan di dalam `serializers.py` dan berperan untuk menentukan struktur respons dari API, melakukan validasi data sebelum disimpan sebagai *Model*, memfasilitasi konversi dua arah antara data model dan representasi JSON.

4. *Router* dan *URL Mapping*

Router dan *URL Mapping* didefinisikan di dalam `urls.py` dan berfungsi untuk memetakan pemanggilan API terhadap suatu *endpoint* ke suatu fungsi *View* yang sesuai.

5.3 Implementasi Logika Bisnis Inti Sistem

Bagian ini membahas implementasi komponen-komponen sistem khusus yang memenuhi kebutuhan alur kerja dan *business process* dari aplikasi. Di luar fungsionalitas CRUD dasar, terdapat bagian-bagian yang memerlukan logika khusus. Seperti perhitungan skor IAA, alur otorisasi berbasis RBAC, pencegahan duplikasi dokumen, dan integrasi *library* NLTK. Berikut adalah rincian implementasi dari kode-kode tersebut:

5.3.1 Perhitungan Skor *Inter-Annotator Agreement* (IAA)

Perhitungan *Inter-Annotator Agreement* (IAA) digunakan untuk menilai tingkat kesesuaian hasil anotasi yang diberikan oleh dua anotator terhadap kalimat yang sama. Pengukuran ini berfungsi untuk memastikan agar kualitas anotasi yang dihasilkan itu konsisten, IAA dihitung dengan membandingkan himpunan koreksi yang diberikan oleh dua anotator menggunakan persamaan berikut:

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

Di mana:

- p_o adalah probabilitas kesepakatan yang diamati (*relative observed agreement*) di antara anotator.

- p_e adalah probabilitas kesepakatan yang diharapkan terjadi secara kebetulan (*hypothetical probability of chance agreement*).

Berikut adalah implementasi pada sistem seperti yang dapat dilihat pada Kode 5.1. Sistem ANOTA menghitung nilai IAA secara otomatis setelah dua anotator menyelesaikan anotasi terhadap kalimat yang sama.

```

1 def cohen_kappa(a_labels, b_labels):
2     N = len(a_labels)
3     if N == 0:
4         return 0.0
5     if len(a_labels) != len(b_labels):
6         raise ValueError("Labels arrays must have the same length")
7     valid_labels = {0, 1}
8     if not set(a_labels).issubset(valid_labels) or not
        set(b_labels).issubset(valid_labels):
9         raise ValueError("Labels must be binary (0 or 1)")
10    n11 = sum(1 for i in range(N) if a_labels[i] == 1 and b_labels[i] == 1)
11    n00 = sum(1 for i in range(N) if a_labels[i] == 0 and b_labels[i] == 0)
12    n10 = sum(1 for i in range(N) if a_labels[i] == 1 and b_labels[i] == 0)
13    n01 = sum(1 for i in range(N) if a_labels[i] == 0 and b_labels[i] == 1)
14    Po = (n11 + n00) / N
15    pA1 = (n11 + n10) / N
16    pA0 = 1 - pA1
17    pB1 = (n11 + n01) / N
18    pB0 = 1 - pB1
19    Pe = pA1 * pB1 + pA0 * pB0
20    denom = 1 - Pe
21    if abs(denom) < 1e-10:
22        return 0.0
23    kappa = (Po - Pe) / denom
24    if math.isnan(kappa) or math.isinf(kappa):
25        return 0.0
26    return max(-1.0, min(1.0, kappa))

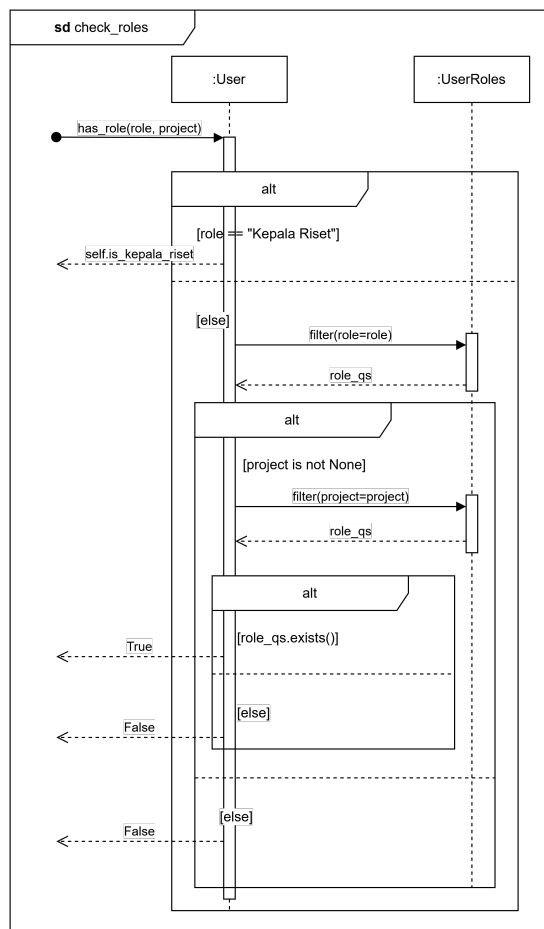
```

Kode 5.1: Implementasi Perhitungan Cohen's Kappa

Sistem mengekstraksi seluruh *span* koreksi, mencocokkannya berdasarkan posisi *token* dan label kesalahan, lalu menghitung skor *agreement*, detail lengkap mengenai IAA selanjutnya dapat dilihat pada Bab 2.1.4.

5.3.2 Role-Based Access Control (RBAC)

Pengelolaan hak akses merupakan bagian penting dalam sistem karena proses anotasi melibatkan beberapa peran dengan tanggung jawab yang berbeda. Penerapan *Role-*



Gambar 5.2: Mekanisme Pengecekan Role

Based Access Control (RBAC) digunakan untuk mengatur akses terhadap data dan operasi berdasarkan peran pengguna. Setiap pengguna memiliki satu atau lebih peran, dan peran-peran yang dimiliki pengguna pada suatu proyek dapat berbeda dengan proyek lainnya (Misal pengguna dapat memiliki *role Admin* dan *Reviewer* di proyek A, sedangkan di proyek B pengguna yang sama memiliki *role Annotator* saja). Sistem kemudian mencocokkan identitas pengguna dengan izin yang diperlukan untuk suatu aksi sebelum mengizinkan permintaan API diproses. Proses pengecekan *role* pada suatu pengguna dapat dilihat pada diagram yang ada pada Gambar 5.2.

Kemudian, berbagai *permission class* digunakan untuk menerapkan aturan akses secara lebih terperinci, dan contoh implementasi yang digunakan pada penelitian ini adalah `IsProjectAdminOrKepalaRiset`. Kelas tersebut memanfaatkan fungsi `has_role()` untuk memvalidasi peran pengguna pada konteks proyek tertentu, sehingga hanya pengguna dengan otoritas yang tepat yang dapat melakukan operasi sensitif. Pada beberapa kasus,

sistem juga menyediakan versi *decorator* yang memungkinkan suatu metode atau fungsi tertentu diberikan aturan izin yang berbeda dari *permission* bawaan pada kelas induknya. Pendekatan ini memberikan fleksibilitas tambahan ketika sebuah aksi memerlukan pembatasan akses yang lebih ketat atau berbeda dari alur umum.

5.3.3 Tokenisasi Kalimat secara Otomatis

Proses anotasi pada sistem ini diawali dengan tahap *pre-processing* dokumen, di mana teks diuraikan dan dipenggal menjadi sekumpulan kalimat-kalimat. Sistem menggunakan *Punkt Sentence Tokenizer* dari *library* NLTK¹⁵. Model Punkt bekerja menggunakan pendekatan statistik yang dilatih untuk mempelajari pola batas kalimat, sehingga mampu mengenali akhir kalimat secara lebih akurat, termasuk pada kasus kompleks seperti singkatan, angka bertitik, atau tanda baca yang ambigu.

```

1 def split_text_to_sentences(self): # pragma: no cover
2     try:
3         return nltk.sent_tokenize(self.text)
4     except Exception:
5         text = self.text or ''
6         parts = re.split(r'(?<=[.!?])\s+', text)
7         return [p for p in (s.strip() for s in parts) if p]

```

Kode 5.2: Metode Tokenisasi Kalimat dengan NLTK dan Fallback RegEx

Dalam implementasi sistem, fungsi `sent_tokenize()` digunakan sebagai metode utama untuk memecah dokumen ke dalam daftar kalimat. Setiap kalimat hasil pemrosesan kemudian disimpan sebagai entitas tersendiri di dalam basis data. Proses *split* juga dilengkapi dengan mekanisme *fallback*, jika pemanggilan NLTK gagal, misalnya karena sistem tidak dapat memuat model, maka sistem secara otomatis beralih ke strategi pemisahan berbasis *regular expression*. Teknik ini memecah teks berdasarkan pola umum akhir kalimat (`[.!?]`), sehingga proses tokenisasi tetap dapat berlangsung meskipun tanpa dukungan model statistik. Implementasi mekanisme dapat dilihat di Kode 5.2.

5.3.4 Document Deduplication dengan Hashing

Pada sistem anotasi, kemungkinan adanya dokumen yang diunggah lebih dari satu kali menjadi tantangan yang perlu diantisipasi untuk mencegah adanya duplikasi dokumen. Untuk mengatasi hal ini, sistem menerapkan mekanisme *document deduplication*. Pen-

¹⁵<https://www.nltk.org/api/nltk.tokenize>

dekatan deduplikasi dilakukan dengan menghasilkan nilai *hash* untuk setiap dokumen berdasarkan isi teks yang diunggah. *Hash* tersebut dibuat menggunakan fungsi *hash* kriptografis SHA-256. Jika nilai *hash* sudah ada, maka sistem dapat menolak unggahan, mengembalikan referensi ke dokumen lama, atau menandai bahwa dokumen tersebut telah digunakan dalam proyek lain tanpa perlu menyimpan duplikasi fisiknya. Aksi yang dilakukan ketika *hash* ditemukan ditentukan oleh *input* dari *client* ketika mengirimkan *request* HTTP. Berikut adalah detail implementasi fitur ini pada *backend* aplikasi ANOTA:

```

1 import hashlib
2
3 class Document(models.Model):
4     text = models.TextField()
5     content_hash = models.CharField(max_length=64, unique=True)
6
7     @staticmethod
8     def compute_hash(text: str) -> str:
9         return hashlib.sha256(text.encode('utf-8')).hexdigest()
10
11     @classmethod
12     def create_if_not_exists(cls, text: str):
13         h = cls.compute_hash(text)
14         existing = cls.objects.filter(content_hash=h).first()
15         if existing:
16             return existing, False # dokumen sudah ada
17         return cls.objects.create(text=text, content_hash=h), True

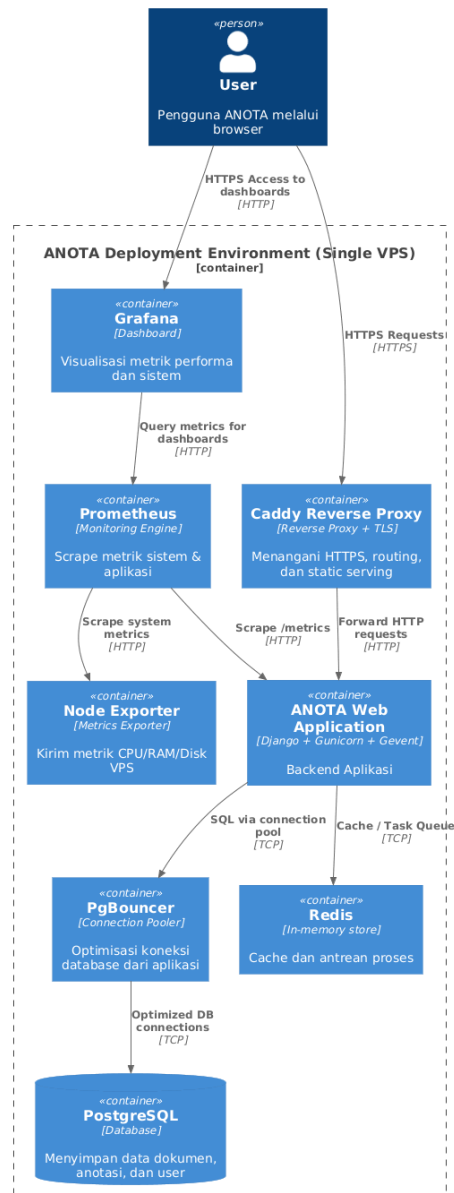
```

Kode 5.3: Contoh Implementasi Deduplication via Hashing

Implementasi deduplikasi diintegrasikan langsung pada proses pembuatan entitas *Document*. Kode 5.3 menunjukkan contoh pembuatan hash dan pemeriksaan keberadaan dokumen sebelum proses penyimpanan dilakukan.

5.4 Konfigurasi *Deployment*

Deployment aplikasi ANOTA dilakukan pada sebuah *Virtual Private Server* (VPS) menggunakan *container* Docker. Seluruh komponen, mulai dari aplikasi Django, basis data PostgreSQL, *connection pooler* PgBouncer, Redis, hingga komponen *monitoring* seperti Prometheus dan Grafana didefinisikan sebagai layanan terpisah di dalam sebuah berkas *docker-compose.yml*. Pendekatan ini memudahkan replikasi konfigurasi dan memberikan isolasi yang lebih baik antar layanan. Semua *request* HTTP yang masuk ke dalam VM kemudian diarahkan ke kontainer menggunakan *reverse proxy* Caddy. Konfigurasi *deployment* dapat dilihat pada C4 *container diagram* pada Gambar 5.3.



Gambar 5.3: Container Layer dari Diagram C4 Arsitektur Deployment Backend

Aplikasi utama dijalankan melalui gunicorn dengan *worker class* *gevent* untuk mendukung *asynchronous I/O*, serta memanfaatkan 4-5 *workers* sesuai kapasitas VPS. Pemilihan jumlah *worker* didasarkan pada rekomendasi dari dokumentasi resmi Gunicorn¹⁶ yang menyarankan untuk menggunakan *worker* sebanyak $2 \times \text{jumlah_core_cpu} + 1$. Sebelum *server* dijalankan, Django akan melakukan *database migration* secara otomatis sehingga memastikan bahwa skema basis data selalu sesuai dengan versi aplikasi yang *dideploy*.

Konfigurasi *Dockerfile* untuk layanan aplikasi ditunjukkan pada Kode 5.4. Seluruh

¹⁶<https://docs.gunicorn.org/en/latest/design.html>

dependensi diinstal pada lingkungan Python 3.10, termasuk model tokenisasi NLTK *punkt* yang diperlukan untuk proses NLP. Berkas statis dikumpulkan pada tahap *build* agar dapat dilayani secara efisien pada saat *runtime*.

```

1 # syntax=docker/dockerfile:1
2 FROM python:3.10-slim
3
4 ENV PYTHONDONTWRITEBYTECODE=1
5 ENV PYTHONUNBUFFERED=1
6
7 RUN apt-get update && apt-get install -y build-essential python3-dev
8
9 WORKDIR /app
10
11 COPY requirements.txt .
12 RUN pip install --no-cache-dir -r requirements.txt
13
14 # reinstall nltk and download punkt tokenizer
15 RUN pip uninstall -y nltk && pip install -U nltk
16 RUN python -m nltk.downloader punkt
17
18 COPY . .
19
20 # collect static files
21 RUN python manage.py collectstatic --noinput
22
23 CMD ["gunicorn", "--bind", "0.0.0.0:8000",
24     "--workers", "5",
25     "--worker-class", "gevent",
26     "--worker-connections", "1000",
27     "anota.wsgi:application"]

```

Kode 5.4: Dockerfile untuk Layanan Aplikasi Anota

Untuk menjalankan seluruh layanan, `docker-compose.yml` digunakan sebagai definisi utama orkestrasi. Setiap layanan dapat dijalankan secara konsisten melalui perintah `docker compose up -d`, sementara Caddy bertindak sebagai gerbang pengaturan lalu lintas dan sertifikasi HTTPS. Berikut adalah penjelasan lebih rinci mengenai setiap bagian dari konfigurasi file *docker-compose.yml*

5.4.1 Layanan Inti Aplikasi

```

1 services:
2   anota-server:
3     build: .
4     image: 008009005/anota-server
5     command: >

```

```

6      sh -c "python manage.py migrate --noinput &&
7          unicorn --workers 4 --worker-class gevent
8          --worker-connections 1000
9          --bind 0.0.0.0:8000 anota.wsgi:application"
10     ports:
11     - "8000:8000"
12     env_file:
13     - .env
14     depends_on:
15     - db
16     - pgbouncer
17     - redis
18     networks:
19     - localprom
20     environment:
21     DB_HOST: pgbouncer
22     DB_PORT: ${DB_PORT}
23     REDIS_URL: redis://redis:${REDIS_PORT}/${REDIS_DB}

```

Kode 5.5: Layanan Inti Aplikasi

Layanan anota-server menjalankan *backend* Django menggunakan Unicorn dengan *worker* gevent untuk mendukung banyak koneksi serentak. Perintah `python manage.py migrate` dijalankan sebelum *server* dinyalakan untuk memastikan skema basis data selalu dalam keadaan terbaru.

5.4.2 Layanan Penyimpanan Data dan Konektivitas

```

1  db:
2    image: postgres:16-alpine
3    restart: always
4    env_file:
5    - .env
6    environment:
7      POSTGRES_USER: ${POSTGRES_USER}
8      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
9      POSTGRES_DB: ${POSTGRES_DB}
10   ports:
11   - "${POSTGRES_PORT}:5432"
12   volumes:
13   - db_data:/var/lib/postgresql/data
14
15   pgbouncer:
16     image: edoburu/pgbouncer
17     restart: always
18     env_file:
19     - .env
20     environment:

```

```

21     DB_USER: ${POSTGRES_USER}
22     DB_PASSWORD: ${POSTGRES_PASSWORD}
23     DB_HOST: db
24     DB_NAME: ${POSTGRES_DB}
25     POOL_MODE: transaction
26     MAX_CLIENT_CONN: ${PGBOUNCER_MAX_CONN}
27     DEFAULT_POOL_SIZE: ${PGBOUNCER_POOL_SIZE}
28     ADMIN_USERS: ${POSTGRES_USER}
29     AUTH_TYPE: plain
30   ports:
31     - "${PGBOUNCER_PORT}:5432"
32   depends_on:
33     - db
34
35   redis:
36     image: redis:7-alpine
37     restart: always
38     env_file:
39       - .env
40   ports:
41     - "${REDIS_PORT}:6379"

```

Kode 5.6: Layanan Database, Connection Pooling, dan Redis

Bagian ini terdiri dari tiga layanan kunci:

1. db sebagai PostgreSQL dengan volume *persistent* agar data tidak hilang saat kontainer dihentikan;
2. pgbouncer sebagai *connection pooler* yang mencegah aplikasi membuka terlalu banyak koneksi langsung ke PostgreSQL;
3. redis untuk *cache* serta penyimpanan *in-memory* yang digunakan untuk keperluan performa.

5.4.3 Layanan Monitoring

```

1   prometheus:
2     image: prom/prometheus
3     command:
4       - --config.file=/etc/prometheus/prometheus.yml
5     volumes:
6       - ./prometheus.yml:/etc/prometheus/prometheus.yml
7     networks:
8       - localprom
9     ports:
10      - "9090:9090"
11
12   node-exporter:

```

```

13     image: prom/node-exporter
14     networks:
15         - localprom
16     ports:
17         - "9100:9100"
18
19     grafana:
20         image: grafana/grafana:6.7.2
21         container_name: grafana
22         env_file:
23             - .env
24         environment:
25             GF_SERVER_ROOT_URL: ${GRAFANA_ROOT_URL}
26         ports:
27             - "3000:3000"

```

Kode 5.7: Layanan Monitoring: Prometheus, Node Exporter, Grafana

Layanan monitoring ini terdiri dari:

1. prometheus yang mengumpulkan metrik dari aplikasi dan sistem;
2. node-exporter yang menyediakan metrik *host-level* seperti CPU dan memori;
3. grafana sebagai antarmuka visual analitik.

Seluruh layanan berada dalam network `localprom` untuk menjaga isolasi logis dari aplikasi inti.

5.4.4 Volume dan Network

```

1 volumes:
2     db_data:
3     grafana_data:
4
5 networks:
6     localprom:
7         driver: bridge

```

Kode 5.8: Volume dan Network

Bagian akhir mendefinisikan penyimpanan *persistent* untuk PostgreSQL dan Grafana, serta jaringan `bridge` khusus yang memisahkan lingkungan *monitoring* dari lalu lintas umum aplikasi.

BAB 6

EVALUASI

Bab ini mencakup rancangan dan hasil dari proses evaluasi yang terdiri dari *unit test*, *load test*, dan *user acceptance test*. Hal ini untuk memastikan bahwa *backend* yang telah dibuat sesuai dengan kebutuhan fungsional dan non-fungsional yang telah dipaparkan sebelumnya.

6.1 *Unit Test*

Bagian ini menjelaskan perancangan, pelaksanaan, serta hasil dari *unit test* yang digunakan untuk memverifikasi bahwa setiap komponen dalam sistem berfungsi sesuai dengan kebutuhan fungsional. Pengujian dilakukan pada level fungsi dan modul untuk memastikan perilaku sistem tetap konsisten, bebas dari regresi, dan memiliki keandalan yang baik selama proses pengembangan.

6.1.1 Cakupan Pengujian

Berikut adalah rincian cakupan *unit test* yang diimplementasikan pada proyek ini:

1. *User Management*:

- **Model dan Serializer:** Pengujian mencakup pembuatan *user*, mekanisme *soft delete*, serta validasi unik pada *username* dan *email*. Pengujian juga dilakukan pada logika pembuatan *password* otomatis saat registrasi.
- **Views dan Permissions:** *Unit test* memverifikasi manajemen *user* dalam *project* (tambah, hapus, *list*), manajemen *role* (*Annotator*, *Reviewer*, *Admin*), serta fitur *password reset* berbasis OTP. Pengujian *permissions* seperti *IsProjectAdminOrKepalaRiset* dilakukan untuk memastikan batasan akses yang tepat.

2. *Project Management*:

- **Project Operations:** Pengujian meliputi operasi CRUD *project* (khusus Kepala Riset) dan logika penugasan admin *project*.
- **Access Control:** Memastikan Kepala Riset dapat melihat semua *project*, sedangkan Admin hanya dapat mengakses *project* yang ditugaskan kepadanya. Validasi juga dilakukan pada fitur *assigned users* untuk memantau partisipan dalam *project*.

3. *Document Management*:

- **Document Processing:** Pengujian pada model `Document` memverifikasi logika pemecahan kalimat (*sentence splitting*) otomatis dan mekanisme penugasan dokumen ke *user*.
- **Workflow dan Status:** *Unit test* memastikan status dokumen (belum dianotasi, sedang dianotasi, siap review, dan lainnya) diperbarui secara dinamis sesuai progres. Pengujian juga mencakup fitur ekspor dokumen (format pararel dan M2) serta validasi input saat pembuatan dokumen.

4. **Annotation dan Review:**

- **Error Types:** Pengujian manajemen tipe *error*, membedakan antara tipe *default* sistem dan tipe spesifik *project*, serta mencegah duplikasi kode *error*.
- **Annotation Lifecycle:** Memverifikasi validasi pembuatan anotasi (kesesuaian dokumen dan kalimat) dan mekanisme *submit* oleh *Annotator*.
- **Review Workflow:** Pengujian mencakup pembuatan *review* (baik koreksi maupun isu baru), validasi prasyarat (anotasi harus di-*submit* sebelum di-*review*), serta mekanisme *reopen* dokumen jika revisi diperlukan.

5. **Dashboard dan Analytics:**

- **Statistics:** Pengujian memastikan akurasi perhitungan statistik pada *dashboard* untuk setiap *role* (jumlah anotasi, *review*, dan status dokumen).
- **Inter-Annotator Agreement (IAA):** *Unit test* memverifikasi perhitungan nilai kesepakatan antar anotator (Cohen's Kappa) antara *Annotator* dan *Reviewer* untuk mengukur kualitas anotasi.

Selanjutnya ketika membuat *unit test*, beberapa dependensi eksternal perlu disimulasikan menggunakan teknik *mocking* dan *stubbing* agar pengujian dapat berjalan secara terisolasi, deterministik, serta tidak menimbulkan efek samping seperti pengiriman *email* sungguhan atau pemanggilan layanan yang tidak diperlukan. Pendekatan ini memungkinkan setiap skenario pengujian untuk fokus pada logika bisnis inti tanpa ketergantungan pada *runtime environment*, layanan pihak ketiga, atau kondisi basis data yang kompleks. Dalam konteks pengembangan aplikasi ini, beberapa komponen yang disimulasikan adalah sebagai berikut:

1. **Layanan Email**

Fungsi pengiriman *email* seperti `send_user_credentials_email`, `send_otp_email`, dan `send_password_reset_success_email` di-*mock* untuk mencegah pengiriman

email asli selama proses pengujian serta memverifikasi bahwa fungsi tersebut dipanggil dengan argumen yang benar.

2. **Package NLTK**

Fungsi `nltk.sent_tokenize` di-*mock* dalam pengujian `Document` untuk mengisolasi logika pemecahan kalimat. Hal ini memastikan hasil pengujian bersifat deterministik dan tidak bergantung pada instalasi data NLTK eksternal.

3. **Perhitungan Statistik**

Fungsi `cohen_kappa` pada modul *dashboard* di-*mock* untuk memverifikasi respons API *Inter-Annotator Agreement* (IAA) tanpa harus mengandalkan perhitungan statistik yang kompleks atau menunggu variasi data nyata.

4. **Authentication dan Permission**

Objek `request.user` dan metode `has_role` di-*mock* dalam pengujian *unit* untuk *decorators* dan *permissions*. Teknik ini memungkinkan simulasi berbagai status pengguna seperti *unauthenticated*, *admin*, atau *kepala riset*, tanpa harus membuat entitas pengguna yang berbeda di basis data.

5. **Interaksi Basis Data Spesifik**

Metode seperti `Project.objects.get` di-*mock* dalam pengujian *permissions* untuk mensimulasikan skenario batas seperti objek yang tidak ditemukan (`DoesNotExist`), sehingga tidak perlu memodifikasi kondisi basis data sebenarnya.

6.1.2 **Lingkungan Pengujian**

Pengujian dijalankan menggunakan *framework* `unittest` dari Python yang di-*extend* oleh `django.test.TestCase` dan `testttrest_framework.test.APITestCase` untuk pengujian API.

Konfigurasi basis data menggunakan **SQLite** (`django.db.backends.sqlite3`), hal ini dipilih untuk mempercepat eksekusi pengujian dan memastikan bahwa setiap sesi tes dimulai dengan basis data yang bersih, tanpa mempengaruhi data di PostgreSQL yang digunakan pada *production*.

Teknik *mocking* diimplementasikan menggunakan `unittest.mock`, khususnya `patch`, untuk menggantikan dependensi eksternal dan fungsi yang memakan waktu atau memiliki efek samping (seperti pengiriman *email* dan tokenisasi teks).

6.1.3 Metode Pengujian

Pengujian dilakukan dengan mencakup berbagai skenario:

1. **Positive Case:** Memverifikasi bahwa sistem berperilaku benar saat menerima input yang valid. Contohnya adalah `test_create_user` yang memastikan pengguna baru berhasil dibuat dengan atribut yang benar, dan `test_assign_document_success` yang memvalidasi penugasan dokumen kepada anotator.
2. **Negative Case:** Memastikan sistem menangani input tidak valid atau tindakan yang tidak diizinkan dengan benar. Contohnya adalah `test_validation_mixin_doc_mismatch` yang memverifikasi penolakan anotasi jika kalimat tidak sesuai dengan dokumen, dan `test_validation_mixin_no_assignment` yang mencegah pengguna tanpa penugasan untuk membuat anotasi.
3. **Edge Case:** Menguji batasan sistem dan kondisi ekstrem. Contohnya adalah `test_dashboard_invalid_project_id` yang menangani format ID proyek yang tidak valid pada *dashboard*, dan `test_serializer_exceptions` yang mensimulasikan kegagalan tak terduga di dalam *serializer* menggunakan *mock object*.
4. **Regression Test:** Pengujian yang ditambahkan untuk memverifikasi perbaikan *bug* dan mencegah masalah yang sama muncul kembali. Contohnya adalah `test_destroy_review_already_submitted` yang memastikan *review* yang sudah di-*submit* tidak dapat dihapus, dan `test_user_assigned_documents_invalid_user` yang memverifikasi respons 404 yang tepat saat mengakses dokumen untuk pengguna yang tidak ada.

6.1.4 Hasil Pengujian

Seluruh rangkaian pengujian telah berhasil berjalan tanpa ada masalah. Pengujian berhasil memvalidasi fungsionalitas inti pada modul `user`, `project`, `document`, `annotation`, dan `dashboard`. Berdasarkan laporan dari alat `coverage.py`, cakupan kode (*code coverage*) yang diperoleh adalah sebesar 100%. Angka ini menunjukkan bahwa setiap baris kode dalam aplikasi, termasuk logika percabangan (*branch*), penanganan *exception*, dan *views*, telah dieksekusi setidaknya satu kali selama proses pengujian.

Tabel 6.1: Spesifikasi VM untuk *Load Test*

Komponen	Spesifikasi
Vendor	Tencent Cloud
Region	Singapore
Arsitektur	64-bit
CPU	2 vCPU (Intel Xeon Platinum 8255C @ 2.50GHz)
RAM	2 GiB
Penyimpanan	40 GiB (Virtio Block, EXT4)
Kartu Jaringan	Virtio Network Device
BIOS	SeaBIOS (1.9.1)

6.1.5 Kesimpulan *Unit Test*

Secara keseluruhan, proses *unit test* telah berhasil memvalidasi seluruh modul aplikasi ANOTA. Kualitas kode dinilai sangat baik dengan cakupan pengujian mencapai 100% dan seluruh skenario uji, termasuk kasus positif, negatif, dan tepi, berjalan dengan sukses. Dengan tidak adanya kegagalan tes dan perbaikan pada *bug* kritis yang ditemukan, sistem dinyatakan stabil dan siap untuk melanjutkan ke tahap pengujian selanjutnya, yaitu *load test* dan *user acceptance test* (UAT).

6.2 *Load Test*

Load test dilakukan untuk mengukur performa dan *latency* dari server ketika digunakan oleh sejumlah pengguna. Uji coba ini dilakukan untuk memastikan bahwa *server* dapat memiliki performa yang baik meskipun ketika digunakan oleh banyak pengguna secara bersamaan serta mengukur apakah kapasitas maksimal *server* sesuai dengan kebutuhan aplikasi.

6.2.1 Lingkungan Uji Coba

Eksperimen dilakukan pada sebuah *environment staging* yang di-deploy pada layanan *cloud* Tencent dengan spesifikasi *virtual machine* (VM) sebagaimana ditunjukkan pada Tabel 6.1.

Backend aplikasi dijalankan menggunakan *web server* Gunicorn dengan konfigurasi sebagai berikut:

Tabel 6.2: Skenario Normal Load

Tahap	Durasi	VU
Ramp-up 1	30 s	20
Sustained 1	5 m	20
Ramp-up 2	30 s	50
Sustained 2	5 m	50
Ramp-down	30 s	0

Tabel 6.3: Skenario Peak Load

Tahap	Durasi	VU
Ramp-up 1	1 m	50
Sustain 1	6 m	50
Ramp-up 2	2 m	150
Sustain 2	8 m	150
Ramp-down	1 m	0

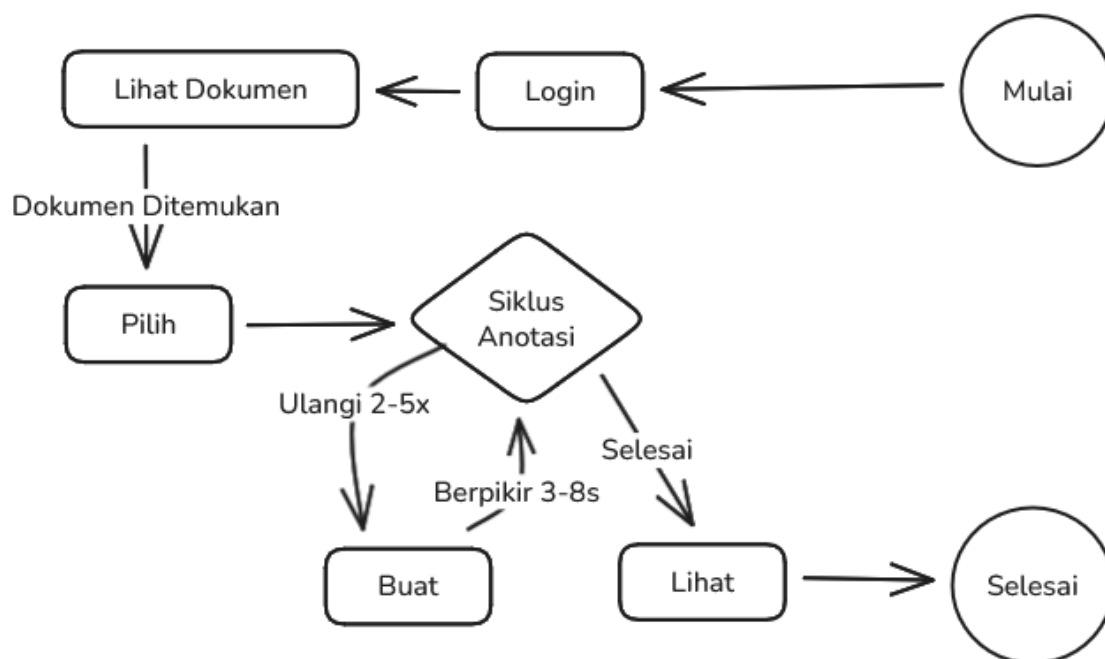
```
gunicorn --bind :8000 --workers 5 anota.wsgi:application \
--worker-class gevent --worker-connections 1000
```

Konfigurasi ini memanfaatkan *worker* sebanyak 5 proses dengan *worker class* *gevent* untuk mendukung *asynchronous I/O*, serta hingga 1000 koneksi per *worker*.

6.2.2 Skenario Uji Coba

Untuk memastikan bahwa *server* dapat mempertahankan performa yang baik dalam berbagai kondisi, *load test* dilakukan dalam dua skenario. Dalam merancang skenario *load test*, penelitian ini merujuk pada acuan pedagogis dari Németh and Sótér (2021) yang membahas penyusunan skenario berdasarkan perilaku pengguna. Penelitian ini akan melakukan *load test* menggunakan skenario penggunaan normal dan skenario penggunaan tinggi (*peak traffic*). Rincian konfigurasi masing-masing skenario dapat dilihat pada Tabel 6.2 dan Tabel 6.3.

Load test dilakukan menggunakan Locust dengan tiga skenario pengguna yang berbeda *Annotator*, *Reviewer*, dan *Admin*. Masing-masing skenario dirancang untuk merepresentasikan pola interaksi nyata dari pengguna asli terhadap sistem, seperti proses anotasi, peninjauan, hingga penugasan dokumen. Setiap tipe pengguna memiliki bobot kemunculan tertentu untuk meniru proporsi penggunaan di dunia nyata. Distribusi bobot ini menghasilkan estimasi proporsi sekitar 66,7% skenario *Annotator*, 22,2% skenario *Reviewer*, dan 11,1% *Admin* selama pengujian berlangsung. Berikut rincian dari masing-masing skenario:



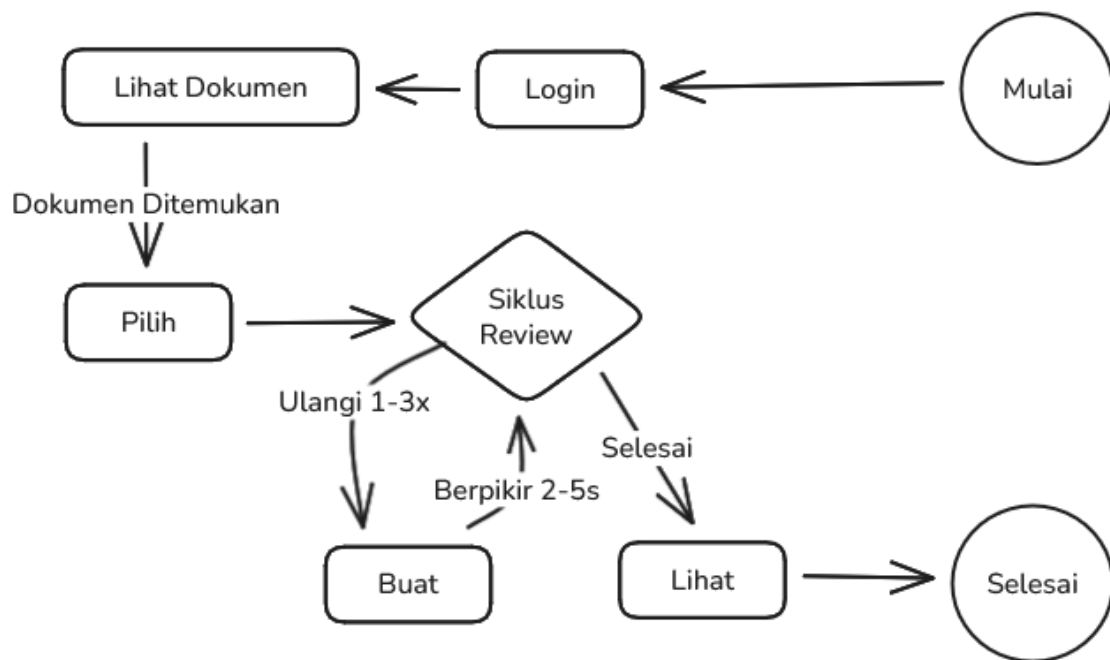
Gambar 6.1: Skenario Uji Coba untuk *Annotator*

6.2.2.1 Skenario *Annotator*

Skenario *Annotator* seperti yang dapat dilihat pada Gambar 6.1 memodelkan alur kerja pengguna yang melakukan proses anotasi dokumen pada sistem. Pada awal pengujian, setiap pengguna virtual hanya melakukan proses *login* satu kali melalui *endpoint* `POST /api/v1/auth/login/`. Token JWT yang dihasilkan kemudian disimpan di *memory* dan digunakan kembali untuk seluruh permintaan berikutnya, sehingga autentikasi tidak perlu dilakukan ulang. Pendekatan ini mencerminkan perilaku pengguna sebenarnya, di mana token tetap berlaku selama sesi penggunaan, dan sekaligus mencegah beban autentikasi mendominasi hasil pengujian.

Setelah autentikasi selesai, pengguna memulai alur utama dengan mengambil daftar dokumen yang ditugaskan menggunakan `GET /api/v1/users/assigned-documents/`. Jika tersedia dokumen yang dapat dikerjakan, pengguna memilih satu dokumen secara acak dan memuat kontennya melalui `GET /api/v1/documents/my-assigned/{assignment_id}/`.

Tahap inti skenario ini adalah proses anotasi yang berlangsung selama 2-5 iterasi pada setiap siklus. Pada setiap iterasi, pengguna memilih satu kalimat secara acak dari dokumen yang telah dimuat, memilih tipe kesalahan (yang telah diambil pada awal skenario), lalu



Gambar 6.2: Skenario Uji Coba untuk *Reviewer*

membuat anotasi baru melalui POST `/api/v1/annotations/`. Di antara setiap iterasi anotasi, pengguna menunggu selama 3-8 detik sebagai waktu berpikir untuk meniru proses membaca, memahami konteks, serta membuat keputusan sebelum mengirim anotasi.

Setelah seluruh iterasi anotasi selesai, pengguna menutup siklus aktivitas dengan melihat daftar anotasi miliknya melalui GET `/api/v1/users/annotations/`. Secara keseluruhan, skenario *Annotator* ini merepresentasikan beban dominan dalam penggunaan sistem sehari-hari, karena sebagian besar aktivitas pengguna berkaitan dengan pemuatan dokumen dan pengiriman anotasi baru.

6.2.2.2 Skenario *Reviewer*

Skenario *Reviewer* seperti yang dapat dilihat pada Gambar 6.2 memodelkan perilaku pengguna yang bertugas meninjau anotasi yang telah dibuat oleh anotator lain. Sama seperti pada skenario sebelumnya, setiap pengguna virtual hanya melakukan proses *login* satu kali pada awal pengujian melalui *endpoint* POST `/api/v1/auth/login/`. Token JWT yang dihasilkan kemudian disimpan dan digunakan kembali untuk seluruh permintaan selanjutnya, sehingga proses autentikasi tidak membebani hasil pengujian dan tetap mencerminkan pola penggunaan nyata pada sistem.

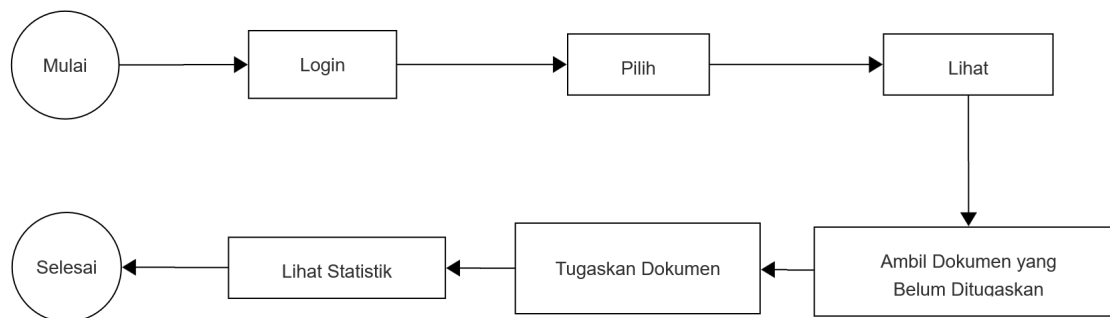
Setelah autentikasi, pengguna memulai alur kerja dengan mengambil daftar dokumen yang ditugaskan kepadanya menggunakan `GET /api/v1/users/assigned-documents/`. Jika terdapat dokumen yang memerlukan peninjauan, pengguna memilih satu dokumen secara acak dan memilih salah satu dokumen secara acak dan mulai memasuki proses *review*. Untuk menjaga realisme skenario, pengguna mensimulasikan waktu membaca antara 2–4 detik sebelum melanjutkan ke tahap berikutnya.

Setelah dokumen dipilih, pengguna mengambil antrian anotasi yang siap ditinjau melalui `GET /api/v1/documents/my-assigned/:id/review-queue/`. Antrian ini memuat daftar kalimat yang telah dianotasi dan menunggu evaluasi dari *reviewer*. Apabila terdapat anotasi yang memenuhi kriteria untuk ditinjau, pengguna akan meninjau antara satu hingga tiga anotasi secara acak dalam satu sesi. Setiap aksi peninjauan dilakukan dengan mengirimkan permintaan `POST /api/v1/annotations/reviews/` yang berisi data lengkap mengenai anotasi tersebut, seperti ID dokumen, ID kalimat, rentang teks kesalahan, tipe kesalahan, usulan koreksi, dan komentar singkat. Di antara setiap proses peninjauan, diberikan jeda tambahan selama dua hingga lima detik untuk mensimulasikan waktu berpikir atau membaca ulang sebelum *reviewer* mengirimkan hasil penilaiannya.

Selain melakukan peninjauan, pengguna virtual juga sesekali memeriksa statistik progres peninjauan mereka melalui *endpoint* `GET /api/v1/annotations/reviews/`. Aksi ini muncul dengan frekuensi yang lebih rendah untuk merepresentasikan perilaku asli *reviewer* yang tidak melakukan pengecekan statistik secara terus-menerus. Secara keseluruhan, skenario ini membentuk pola beban yang realistis dan merepresentasikan aktivitas seorang *reviewer* dalam lingkungan kerja ANOTA, sehingga dapat digunakan untuk menguji performa sistem secara akurat pada modul peninjauan.

6.2.2.3 Skenario *Admin* dan Kepala Riset

Skenario *Admin* dan Kepala Riset seperti yang dapat dilihat pada Gambar 6.3 digunakan untuk memodelkan perilaku pengguna dengan peran administratif yang bertanggung jawab mengelola proyek, pengguna, dan dokumen di dalam sistem. Seperti pada skenario lainnya, setiap pengguna virtual hanya melakukan proses *login* sekali di awal melalui *endpoint* `POST /api/v1/auth/login/`, dan token JWT yang diperoleh akan disimpan dan digunakan kembali selama seluruh pengujian. Dengan demikian, beban autentikasi tetap minimal dan sesuai dengan pola penggunaan nyata pada aplikasi.



Gambar 6.3: Skenario Uji Coba untuk *Admin* dan Kepala Riset

Setelah berhasil masuk, *admin* dan kepala riset memulai aktivitas dengan memuat daftar proyek yang dapat diakses melalui GET `/api/v1/users/projects/`. Setelah salah satu proyek dipilih, *admin* dan kepala riset mengambil daftar pengguna anotator terkait proyek tersebut menggunakan GET `/api/v1/users/projects/{project_id}/users/`. Selanjutnya, untuk mendapatkan dokumen yang belum ditugaskan kepada siapa pun, *admin* dan kepala riset mengakses GET `/api/v1/projects/{project_id}/documents/`. Jika tersedia dokumen yang belum memiliki penugasan, *admin* dan kepala riset memilih satu dokumen secara acak dan melakukan proses penugasan kepada anotator tertentu menggunakan POST `/api/v1/assignments/`.

Tidak ada iterasi berulang seperti pada skenario *Annotator* maupun *Reviewer*, karena aktivitas *admin* cenderung berupa tindakan administratif tunggal atau *batch* kecil. Namun, untuk meniru pola penggunaan yang realistis, setiap langkah *admin* disertai jeda pendek berupa *think time* 1–3 detik yang mencerminkan proses pengambilan keputusan pengguna di dunia nyata.

Pada tahap akhir, kepala riset memantau perkembangan proyek dengan memanggil GET `/api/v1/projects/{project_id}/statistics/`, yang menampilkan ringkasan status anotasi dan progres keseluruhan. Skenario ini terutama menguji beban yang berkaitan dengan pemuatan data proyek dan proses penugasan dokumen, dua komponen krusial dalam pengelolaan operasional sistem.

6.2.3 Hasil Uji Coba

Bagian ini memaparkan hasil pengujian performa yang telah dilakukan berdasarkan konfigurasi dan skenario yang dijelaskan pada bagian sebelumnya. Pengujian dilakukan untuk menilai kemampuan sistem dalam menangani *normal load* maupun *peak load*, serta untuk

Tabel 6.4: *Request Statistics* untuk Skenario *Normal Load*

Type	Name	# Req	# Fails	Med (ms)	Avg (ms)	Min (ms)	Max (ms)	Avg Size (B)
GET	/api/v1/annotations/	107	0	470	548.08	232	1342	85211.73
POST	/api/v1/annotations/	1781	0	46	57.94	27	983	498.32
GET	/api/v1/annotations/error-types/	33	0	30	42.04	15	318	597
GET	/api/v1/annotations/reviews/	93	0	79	95.57	21	408	8265.12
POST	/api/v1/annotations/reviews/	171	0	47	53.54	34	287	631.02
GET	/api/v1/documents/my-assigned/:id/	988	0	80	98.91	38	1002	79367.89
GET	/api/v1/documents/my-assigned/:id/review-queue/	466	0	39	49.96	25	809	529.7
POST	/api/v1/login/	50	0	690	695.87	541	1026	582
GET	/api/v1/users/me/assigned-documents/	991	0	270	294.57	161	1306	242098.09
GET	/api/v1/users/me/projects/	473	0	25	33.19	15	434	640
Aggregated		5153	0	59	125.11	15	1342	64004.42

mengevaluasi stabilitas dan efisiensi *backend* ANOTA pada lingkungan produksi.

6.2.3.1 Ringkasan Hasil Pengujian

Secara keseluruhan, hasil pengujian menunjukkan bahwa sistem ANOTA berada dalam kondisi stabil dan mampu menangani sebagian besar skenario uji dengan baik. Pada skenario *Normal Load* (50 VU), sistem menunjukkan performa optimal dengan *error rate* 0%, waktu respons median agregat sebesar 59 ms, dan nilai p95 sebesar 390 ms. Seluruh endpoint memberikan respons dalam rentang yang sangat baik, dengan waktu respons median berkisar antara 25-470 ms dan nilai p95 tetap berada di bawah 1.000 ms. Hasil evaluasi lebih lengkap dapat dilihat pada Tabel 6.4 dan Tabel 6.5.

Tabel 6.5: *Response Statistics* (in milliseconds) untuk Skenario *Normal Load*

Method	Name	50 th ile	60 th ile	70 th ile	80 th ile	90 th ile	95 th ile	99 th ile	100 th ile
GET	/api/v1/annotations/	470	530	620	740	950	1000	1200	1300
POST	/api/v1/annotations/	46	50	54	63	79	100	280	980
GET	/api/v1/annotations/error-types/	30	32	36	44	57	75	320	320
GET	/api/v1/annotations/reviews/	79	86	98	120	160	190	410	410
POST	/api/v1/annotations/reviews/	47	50	54	61	78	84	130	290
GET	/api/v1/documents/my-assigned/:id/	81	89	99	120	160	200	390	1000
GET	/api/v1/documents/my-assigned/:id/review-queue/	39	42	46	51	66	94	260	810
POST	/api/v1/login/	690	700	720	790	900	990	1000	1000
GET	/api/v1/users/me/assigned-documents/	270	290	320	350	400	470	700	1300
GET	/api/v1/users/me/projects/	25	26	29	32	42	64	240	430
Aggregated		59	75	110	220	310	390	730	1300

Pada skenario *Peak Load* (150 VU), sistem tetap beroperasi secara fungsional meskipun mengalami degradasi performa yang signifikan. Terjadi peningkatan latensi yang sub-

Tabel 6.6: *Request Statistics* untuk Skenario *Peak Load*

Type	Name	# Req	# Fails	Med (ms)	Avg (ms)	Min (ms)	Max (ms)	Avg Size (B)
GET	/api/v1/annotations/	192	0	1300	2777.9	214	51547	88344.43
POST	/api/v1/annotations/	2991	28	81	1236.74	26	50987	511.11
GET	/api/v1/annotations/error-types/	100	0	2000	1673.74	20	4361	597
GET	/api/v1/annotations/reviews/	147	0	110	862.09	19	5751	7656.48
POST	/api/v1/annotations/reviews/	238	0	72	1057.35	31	7703	633
GET	/api/v1/documents/my-assigned/:id/	1570	0	150	1317.34	34	50195	78957.36
GET	/api/v1/documents/my-assigned/:id/review-queue/	668	0	54	1377.32	24	50233	531.49
POST	/api/v1/login/	150	0	11000	16228.4	525	52240	582
GET	/api/v1/users/me/assigned-documents/	1585	0	410	1893.8	158	51337	237221.93
GET	/api/v1/users/me/projects/	812	0	42	1309.9	14	50678	666
Aggregated		8453	28	280	1687.69	14	52240	61607.67

Tabel 6.7: *Response Statistics* (in milliseconds) untuk Skenario *Peak Load*

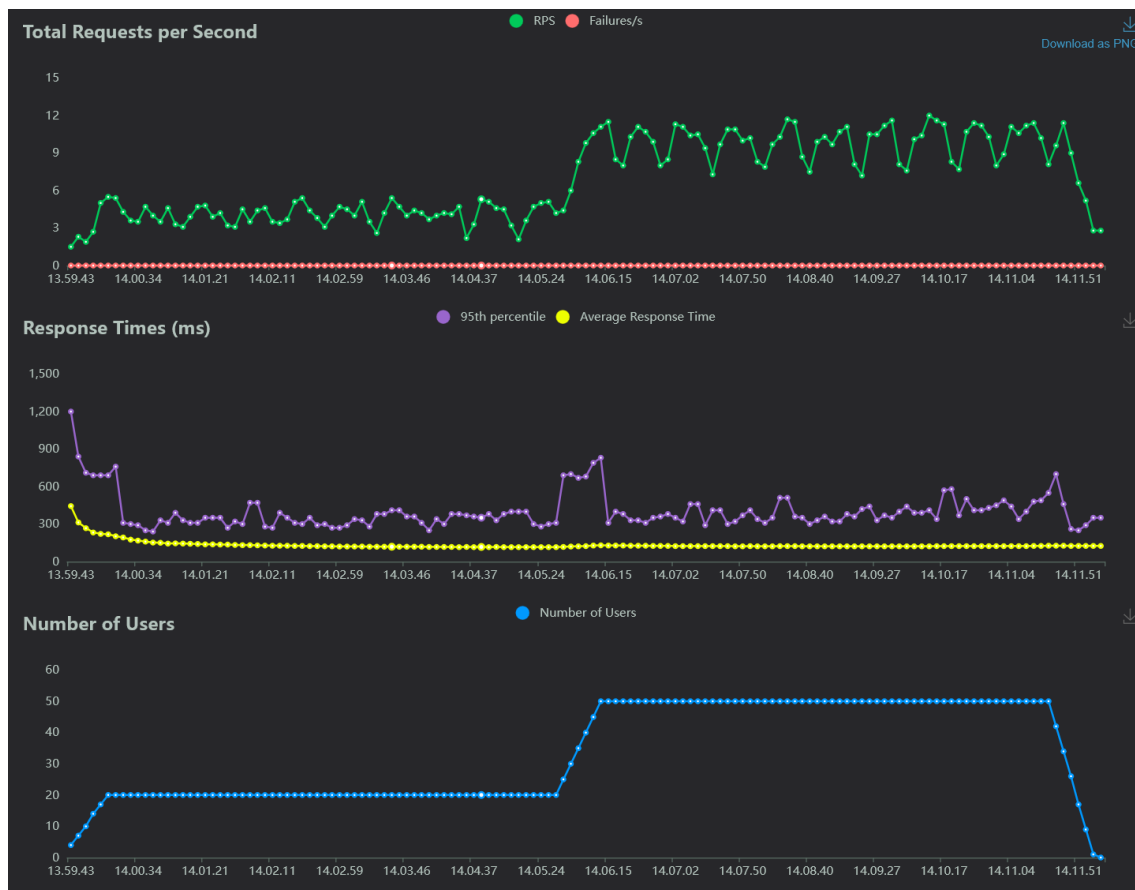
Method	Name	50%ile	60%ile	70%ile	80%ile	90%ile	95%ile	99%ile	100%ile
GET	/api/v1/annotations/	1500	2800	3400	4400	5300	6800	51000	52000
POST	/api/v1/annotations/	81	510	1300	2200	3100	4000	8900	51000
GET	/api/v1/annotations/error-types/	2000	2200	2500	2800	3300	3700	4400	4400
GET	/api/v1/annotations/reviews/	110	230	1100	1800	2800	3600	5400	5800
POST	/api/v1/annotations/reviews/	74	500	1400	2300	3300	4000	6600	7700
GET	/api/v1/documents/my-assigned/:id/	150	490	1400	2300	3200	4400	8200	50000
GET	/api/v1/documents/my-assigned/:id/review-queue/	54	150	1000	1900	2900	4300	46000	50000
POST	/api/v1/login/	11000	19000	25000	34000	42000	46000	51000	52000
GET	/api/v1/users/me/assigned-documents/	410	840	1800	2700	3900	5200	38000	51000
GET	/api/v1/users/me/projects/	43	410	1300	2000	3200	4300	8500	51000
Aggregated		280	660	1500	2400	3500	4900	32000	52000

stansial dengan median agregat mencapai 280 ms dan p95 agregat naik menjadi 4.900 ms. Tercatat 28 kegagalan (*error rate* 0,33%) berupa HTTP 502 pada endpoint /api/v1/annotations/ selama fase *ramp-up* menuju beban tertinggi, sehingga *server* memiliki ingkat keberhasilan 99,67% hal ini menunjukkan batasan dari kapasitas *server* dan degradasi pada performa sudah mulai terlihat. Hasil evaluasi lebih lengkap dapat dilihat pada Tabel 6.6 dan Tabel 6.7.

6.2.3.2 Hasil Pengujian pada Skenario Normal Load

Pada skenario *Normal Load*, sistem diuji dalam kondisi penggunaan harian yang representatif dengan 20 *virtual users* yang kemudian di-*ramp* ke 50 *virtual users*. Ringkasan hasil dari uji coba pada skenario ini dapat dilihat pada Gambar 6.4.

Hasil pengujian pada skenario ini menunjukkan stabilitas performa yang sangat baik. Berdasarkan metrik yang diperoleh, rata-rata waktu respons (*Average Response Time*)



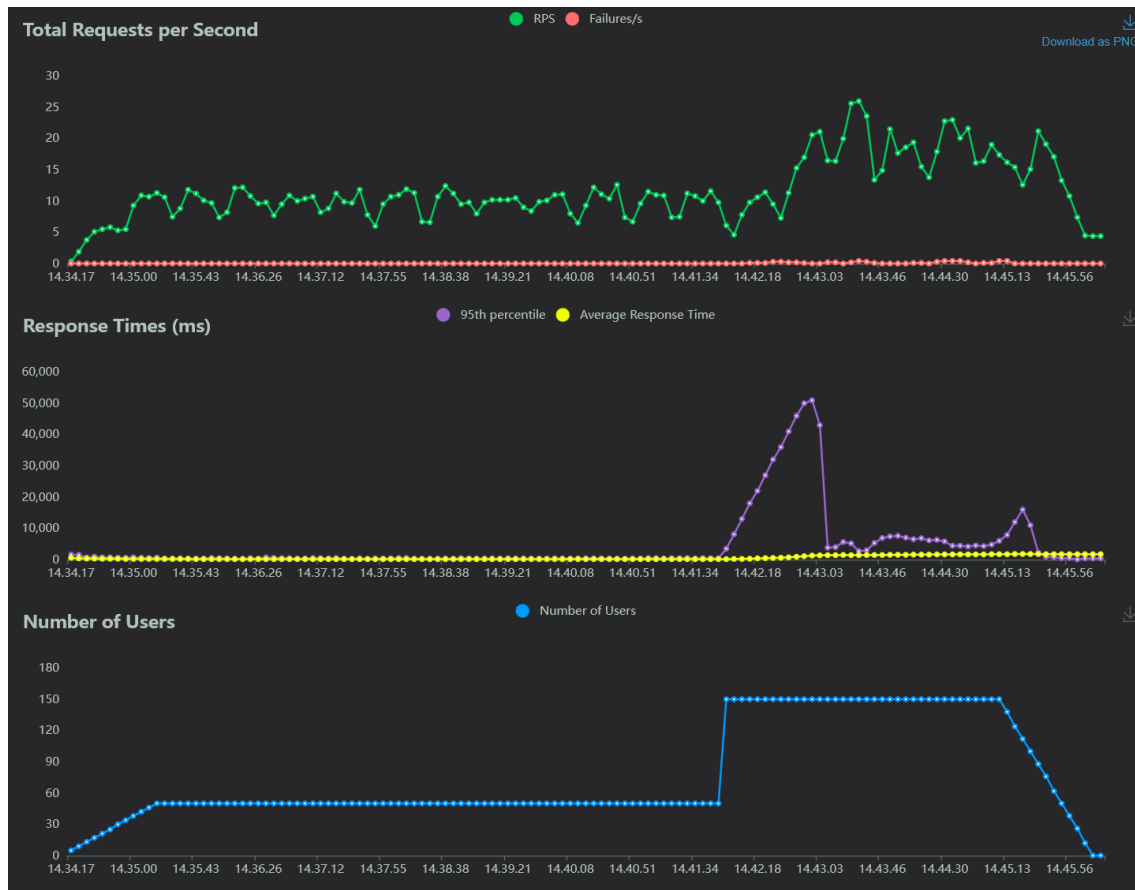
Gambar 6.4: Hasil *Load Test* pada Skenario *Normal Load*

sepanjang proses *test* berkisar di sekitar 125–250ms, yang mengindikasikan bahwa sistem sangat responsif terhadap sebagian besar permintaan pengguna dengan sedikit lonjakan (*spike*) disetiap fase *ramp-up* yang kemudian kembali stabil turun lagi. Nilai *percentile* ke-95 (p95) berada di antara 300–700ms dengan lonjakan sebesar 1.200ms dan 830ms pada saat fase *ramp-up*. Sebanyak 5.153 permintaan yang dikirimkan selama durasi pengujian berhasil diproses dengan status sukses, menghasilkan *error rate* sebesar 0%.

6.2.3.3 Hasil Pengujian pada Skenario Peak Load

Skenario *Peak Load* digunakan untuk melihat bagaimana sistem merespons ketika jumlah pengguna meningkat hingga 150 *virtual users*. Skenario ini bertujuan untuk mengidentifikasi potensi *bottleneck* yang tidak muncul pada kondisi normal. Ringkasan dari hasil uji coba pada skenario ini dapat dilihat pada Gambar 6.5

Peningkatan beban hingga 150 *virtual users* memberikan tekanan signifikan terhadap antrian pemrosesan di sisi server, sistem mulai mengalami degradasi dengan 28 *request*



Gambar 6.5: Hasil *Load Test* pada Skenario *Peak Load*

yang mengalami HTTP *error* 502. Waktu response pada persentil ke-95 melonjak hingga 51.000 ms (51 detik). Fenomena ini mengindikasikan terjadinya *request queuing*, di mana permintaan masuk harus mengantre di lapisan *web server* (Gunicorn) sebelum dapat diproses oleh *worker*. Namun secara keseluruhan waktu respons rata-rata (*average response time*) masih berada di dalam *range* yang cukup baik dalam rentang 800–1.700ms pada 150 VU.

6.2.3.4 Perbandingan Kinerja Antar Skenario

Bagian ini membandingkan hasil dari kedua skenario untuk melihat perubahan performa ketika beban meningkat. Tabel 6.8 memperlihatkan perbandingan langsung metrik utama antara kondisi beban normal dan beban puncak.

Tabel 6.8: Perbandingan Metrik Performa antara *Normal Load* dan *Peak Load*

[t] heightMetrik	Normal Load (50 VU)	Peak Load (150 VU)	Delta
Total Request	5.153	8.453	+64,1%
RPS (Rata-rata)	~10 req/s	~17 req/s	+70%
Agr. Median Latency	59 ms	280 ms	+375%
Agr. p95 Latency	390 ms	4.900 ms	+1.156%
Agr. p99 Latency	730 ms	32.000 ms	+4.284%
Error Rate	0%	0,33%	+0,33%

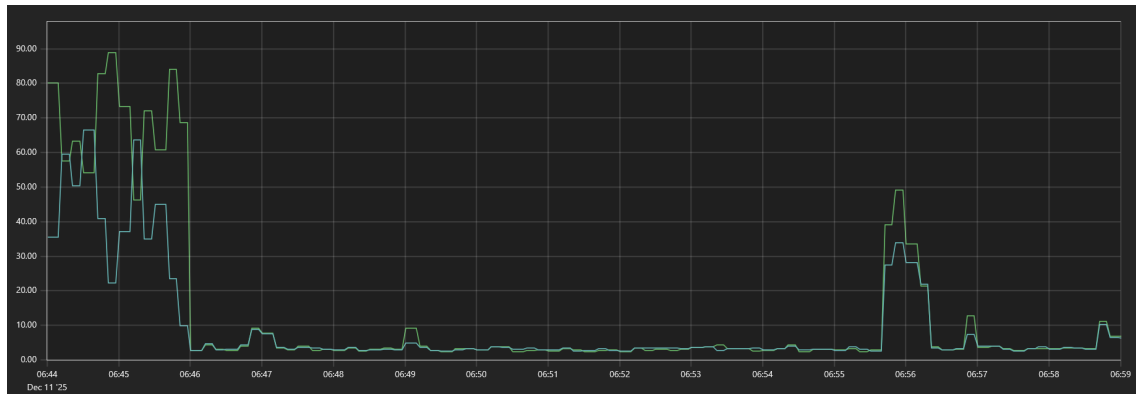
Analisis perbandingan menunjukkan bahwa meskipun *throughput* (RPS) sistem mampu berskala mengikuti peningkatan jumlah pengguna (naik 70%), kualitas layanan dari sisi responsivitas menurun drastis. Peningkatan p95 dari 390 ms ke 4.900 ms dan p99 dari 730 ms ke 32.000 ms menunjukkan degradasi signifikan pada *tail distribution* latensi. Catatan penting adalah munculnya error rate sebesar 0,33% (28 *failures*) pada *Peak Load*, yang disebabkan oleh HTTP 502 pada *endpoint* `/api/v1/annotations/`. Meskipun demikian, tingkat keberhasilan 99,67% masih dalam batas toleransi untuk aplikasi internal kampus yang tidak bersifat *real-time critical*. Peningkatan p99 yang ekstrim pada *Peak Load* menandakan bahwa batas kenyamanan pengguna mulai terlampaui ketika konkurensi melebihi 100 pengguna aktif bersamaan.

6.2.3.5 Analisis Konsumsi Sumber Daya Server

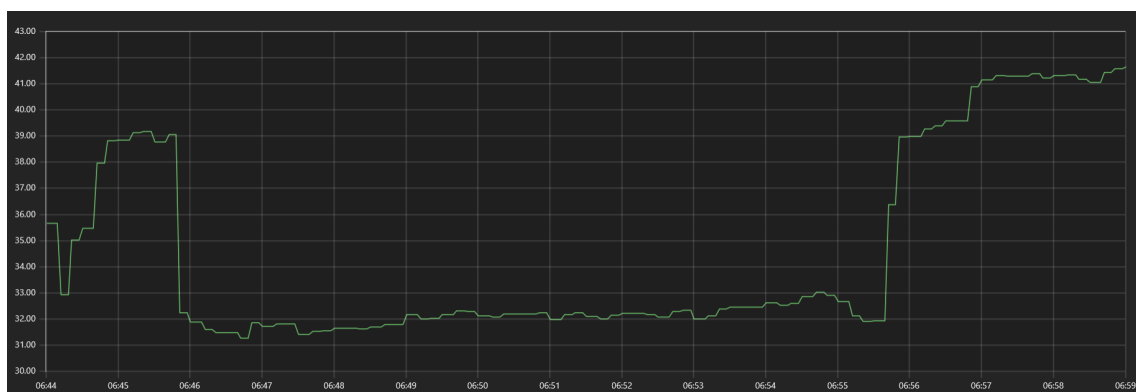
Pengujian performa juga diamati melalui penggunaan sumber daya server yang mencakup penggunaan CPU dan *memory*.

Berdasarkan grafik pada Gambar 6.6 dan Gambar 6.7 terdapat lonjakan yang sangat besar pada tingkat utilisasi CPU dan juga RAM pada tahap *ramp-up*, hal ini menunjukkan proses *login* sangatlah berkontribusi terhadap *stress* yang dialami oleh sistem. Hal ini wajar, karena Django menggunakan algoritma *hash* PBKDF2 yang membutuhkan komputasi yang cukup berat dan memakan *resource* yang cukup signifikan pada proses *login*. Hal ini dilakukan agar *brute force attack* sulit untuk dilakukan (Visconti et al., 2019).

Grafik pada Gambar 6.8 dan Gambar 6.9 turut menunjukkan bahwa permasalahan se-



Gambar 6.6: Persentase Penggunaan CPU pada Skenario *Normal Load*



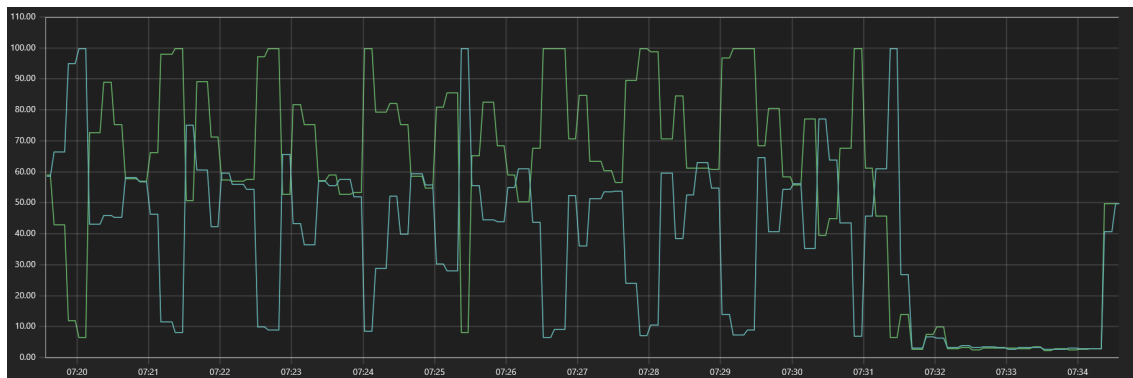
Gambar 6.7: Persentasi Penggunaan RAM pada Skenario *Normal Load*

belumnya semakin parah ketika jumlah pengguna ditingkatkan. Dapat dilihat terjadi banyak sekali *spike* penggunaan 100% CPU dan juga 90% penggunaan RAM. Hal ini menjadi penyebab terjadinya *failuire* pada saat *load test*.

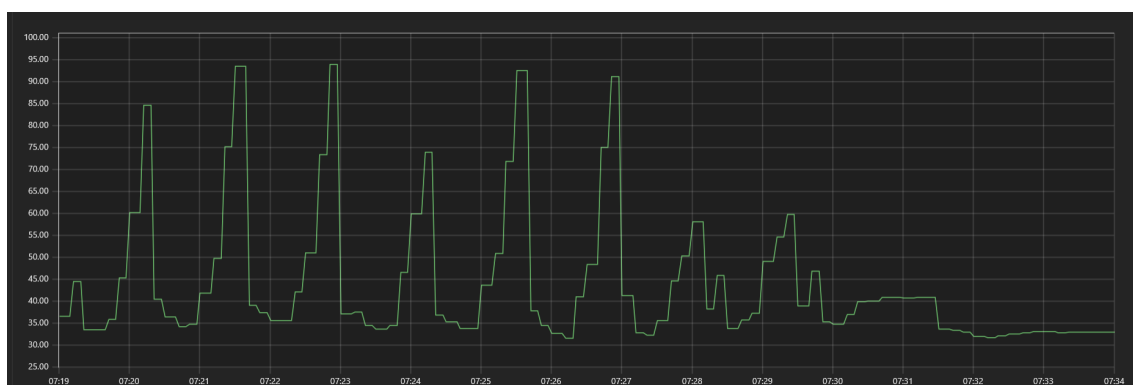
6.2.3.6 Analisis Bottleneck dan Temuan Teknis

Selama pengujian berlangsung, beberapa temuan teknis terkait *bottleneck* sistem berhasil diidentifikasi:

1. **Endpoint Login sebagai Bottleneck Utama:** *Endpoint* POST `/api/v1/login/` menjadi titik paling lambat dengan agregat median 11.000ms dan agregat p95 mencapai 46.000ms pada *Peak Load*, dibandingkan dengan agregat median 690ms dan agregat p95 990ms pada *Normal Load*. Lonjakan penggunaan CPU hingga 100% yang terjadi bersamaan dengan fase *ramp-up* mengkonfirmasi bahwa ini adalah *CPU-bound bottleneck* akibat algoritma PBKDF2 *hashing* yang intensif secara komputasi.
2. **Indikasi Request Queuing:** Terjadi kesenjangan besar antara median *response time*



Gambar 6.8: Persentase Penggunaan CPU pada Skenario *Peak Load*



Gambar 6.9: Persentase Penggunaan RAM pada Skenario *Peak Load*

agregat (280ms) dengan agregat p99 (32.000ms) pada *Peak Load*. Selain itu munculnya 28 HTTP 502 *failures* pada *endpoint* POST `/api/v1/annotations/` saat CPU mencapai 100% mengindikasikan terjadinya *request queuing* di lapisan Unicorn, di mana permintaan harus mengantre karena *worker* sedang sibuk memproses *request* lain.

3. **Degradasi pada *Endpoint* dengan *Payload* Besar:** *Endpoint* GET `/api/v1/users/me/assigned-documents/` (ukuran rata-rata 237–242 KB) mengalami peningkatan agregat median dari 270ms menjadi 410ms, dan agregat p95 dari 470ms menjadi 5.200ms antara *Normal Load* dan *Peak Load*. Pola serupa terlihat pada GET `/api/v1/annotations/` (85–88 KB) dengan agregat median naik dari 470ms ke 1.300ms dan agregat p95 dari 1.000ms ke 6.800 ms, mengindikasikan kombinasi antara beban *query database* dan *overhead serialization* sebagai faktor pembatas.

6.2.3.7 Kesimpulan Evaluasi Performa

Berdasarkan hasil pengujian, dapat disimpulkan bahwa:

- **Pemenuhan Kebutuhan:** Pada *Normal Load* (50 VU), sistem ANOTA menunjukkan performa stabil dengan *error rate* 0%, median *response time* 59 ms, dan p95 390 ms. Dengan demikian, sistem telah sepenuhnya memenuhi kebutuhan operasional harian.
- **Batas Kapasitas:** Dengan spesifikasi 2 vCPU dan 2 GiB RAM, sistem optimal hingga 50 VU. Pada *Peak Load* (150 VU), sistem tetap bekerja (99,67% berhasil) namun mengalami degradasi besar (median 280ms, p95 4.900ms) dan mencapai saturasi CPU (100%) serta RAM (90%).
- **Bottleneck Kritis:** POST /api/v1/login/ menjadi sumber masalah utama dengan p95 46.000ms akibat komputasi PBKDF2. Lonjakan p99 (32.000ms) dan munculnya 28 error 502 menunjukkan antrean *request* dan keterbatasan *worker* Unicorn.
- **Rekomendasi Utama:**
 1. Optimalkan proses autentikasi (mis. *token refresh*, atau migrasi ke Argon2).
 2. Tambah jumlah atau ubah tipe *worker* Unicorn ke model asinkron.
 3. Terapkan *caching* (Redis/Memcached) untuk *endpoint* statis dan kueri berulang yang belum di *cache*.
 4. Optimalkan kueri database melalui indeks, *pagination*, dan optimisasi *query* pada ORM Django.
 5. Pertimbangkan *vertical scaling* (setidaknya lebih dari atau setara dengan 4 vCPU, 4 GiB RAM) jika aplikasi sudah digunakan oleh lebih dari 100 pengguna secara bersamaan.
- **Kelayakan Produksi:** Untuk kebutuhan internal dengan kurang dari 50 pengguna bersamaan, sistem sudah layak diproduksi. Pemantauan berkala metrik performa tetap diperlukan untuk mengantisipasi peningkatan beban.

6.3 User Acceptance Test (UAT)

Pada tahap ini dilakukan *User Acceptance Test* (UAT) untuk memverifikasi bahwa layanan *backend* telah memenuhi kebutuhan bisnis dan alur kerja pengguna sebagaimana yang dirancang pada dokumen kebutuhan. Karena ruang lingkup penelitian ini hanya mencakup pengembangan sisi *backend*, proses UAT dilakukan tidak melalui antarmuka *frontend*, melainkan menggunakan serangkaian skenario yang dieksekusi melalui Postman sebagai

pengganti interaksi pengguna.

6.3.1 Tujuan UAT

Tujuan dari *User Acceptance Testing* (UAT) ini adalah untuk memverifikasi bahwa sistem *backend* ANOTA telah memenuhi kebutuhan fungsional dan non-fungsional yang didefinisikan. Secara spesifik, UAT bertujuan untuk:

- Memastikan setiap *endpoint* API bekerja sesuai dengan spesifikasi desain dan mengembalikan respons yang diharapkan.
- Memvalidasi bahwa alur kerja utama (*happy path*) untuk setiap peran pengguna (*Annotator*, *Reviewer*, *Admin*, *Kepala Riset*) dapat dijalankan tanpa hambatan.
- Memverifikasi penanganan kasus negatif (misalnya, *login* dengan kredensial salah, akses tanpa izin) untuk memastikan keamanan dan *robustness* sistem.
- Memastikan integritas data terjaga selama operasi CRUD (*Create*, *Read*, *Update*, *Delete*).
- Memastikan seluruh aturan bisnis (*business rules*), seperti batasan akses berdasarkan peran dan status dokumen, diterapkan dengan benar oleh *backend*.

6.3.2 Metodologi UAT

Pengujian dilakukan menggunakan pendekatan *Black Box Testing* melalui antarmuka API. Alat utama yang digunakan adalah *Postman*, yang memungkinkan simulasi permintaan HTTP ke server *backend*. Metodologi pengujian mencakup langkah-langkah berikut:

1. **Persiapan Lingkungan:** Menggunakan *Postman Environment* untuk mengelola variabel global seperti `base_url` dan `access_token`, sehingga pengujian dapat dijalankan secara dinamis tanpa mengubah skrip secara manual.
2. **Skenario Berbasis Peran:** Pengujian dikelompokkan ke dalam koleksi (*Collection*) berdasarkan peran pengguna untuk meniru alur kerja nyata.
3. **Otomatisasi Validasi:** Menggunakan fitur *tests* pada Postman (berbasis JavaScript) untuk secara otomatis memvalidasi status kode respons, struktur JSON, dan nilai data yang dikembalikan.

6.3.3 Skenario Pengujian

Skenario pengujian dirancang untuk mencakup seluruh siklus hidup data dalam aplikasi, mulai dari inisialisasi proyek hingga penyelesaian anotasi dan *review*. Pada bagian ini akan dijelaskan beberapa contoh alur kerja yang kami uji. Detail lengkap dari skenario yang kami uji dapat dilihat pada bagian Lampiran 4.

6.3.3.1 Skenario *Annotator*

Skenario ini memvalidasi alur kerja seorang *Annotator* dalam menyelesaikan tugas anotasi dokumen. Langkah-langkah pengujian meliputi:

1. **Autentikasi:** *Login* menggunakan kredensial *Annotator* valid untuk mendapatkan token akses (AUTH-001).
2. **Melihat Tugas:** Mengakses *endpoint* `/api/v1/users/assigned-documents/` untuk melihat daftar dokumen yang perlu dianotasi (USER-002).
3. **Detail Dokumen:** Mengambil detail dokumen spesifik termasuk kalimat-kalimat di dalamnya (DOC-006).
4. **Manajemen Anotasi:**
 - Membuat anotasi baru untuk kalimat tertentu (ANNO-007).
 - Memperbarui anotasi yang sudah ada (ANNO-009).
 - Menghapus anotasi yang salah (ANNO-010).
 - Memverifikasi daftar anotasi yang telah dibuat (ANNO-006, ANNO-022).
5. **Finalisasi:** Mengirimkan (*submit*) hasil anotasi untuk dokumen tersebut, yang akan mengubah status dokumen menjadi siap untuk di-*review* (ANNO-011).

6.3.3.2 Skenario *Reviewer*

Skenario ini memvalidasi alur kerja seorang *Reviewer* dalam memeriksa kualitas anotasi. Langkah-langkah pengujian meliputi:

1. **Autentikasi:** *Login* sebagai *Reviewer* (AUTH-001).
2. **Melihat Antrian Review:** Mengakses daftar dokumen yang telah disubmit oleh *Annotator* dan ditugaskan kepadanya (USER-002).
3. **Pemeriksaan Anotasi:** Mengambil daftar anotasi pada dokumen yang perlu di-*review* (DOC-007).

4. **Pemberian Umpan Balik:**

- Membuat *review* untuk anotasi tertentu, memberikan status (Benar/Salah) dan komentar (ANNO-014).
- Mengedit *review* jika diperlukan (ANNO-016).

5. **Penyelesaian Review:** Mengirimkan (*submit*) hasil *review*, yang menandai dokumen sebagai selesai atau perlu revisi (ANNO-018).

6.3.3.3 Skenario Admin

Skenario ini memvalidasi fungsi manajerial yang dilakukan oleh *Admin*. Langkah-langkah pengujian meliputi:

1. **Manajemen Proyek:**

- Membuat proyek baru (PROJ-002).
- Memperbarui informasi proyek (PROJ-004).
- Melihat detail proyek (PROJ-003).

2. **Manajemen Pengguna:**

- Menambahkan pengguna baru ke dalam proyek (USER-004).
- Mengatur peran pengguna (USER-008).
- Me-reset *password* pengguna jika diperlukan (USER-007).

3. **Manajemen Dokumen:**

- Mengunggah dokumen baru (DOC-002).
- Menugaskan dokumen kepada *Annotator* dan *Reviewer* (DOC-008).
- Memantau progres pengerjaan melalui *dashboard* (DASH-001).

4. **Ekspor Data:** Mengunduh hasil anotasi dalam format paralel (TSV) atau M2 untuk evaluasi model (DOC-009, DOC-010).

6.3.3.4 Skenario Kepala Riset

Skenario ini memvalidasi fungsi manajerial yang dilakukan oleh Kepala Riset. Langkah-langkah pengujian meliputi:

1. **Manajemen Proyek:**

- Membuat proyek baru (PROJ-002).
- Memperbarui informasi proyek (PROJ-004).
- Melihat detail proyek (PROJ-003).

- Menambahkan *admin* pada proyek (PROJ-006)
- Menghapus *admin* dari proyek (PROJ-007)

2. Manajemen Pengguna:

- Menambahkan pengguna baru ke dalam proyek (USER-004).
- Mengatur peran pengguna (USER-008).
- *Reset password* pengguna jika diperlukan (USER-007).

3. Ekspor Data: Mengunduh hasil anotasi dalam format paralel (TSV) atau M2 untuk evaluasi model (DOC-009, DOC-010).

6.3.4 Instrumen dan Kriteria Evaluasi

Evaluasi dilakukan secara otomatis menggunakan skrip pengujian (*Test Scripts*) yang disematkan pada setiap permintaan di Postman. Kriteria keberhasilan meliputi:

- **Status Kode HTTP:** Setiap respons harus mengembalikan kode status yang sesuai standar RESTful:
 - 200 OK untuk permintaan berhasil yang mengembalikan data.
 - 201 Created untuk keberhasilan pembuatan sumber daya baru.
 - 204 No Content untuk keberhasilan penghapusan.
 - 400 Bad Request untuk validasi input yang gagal (skenario negatif).
 - 401 Unauthorized untuk akses tanpa token valid.
 - 403 Forbidden untuk akses tanpa izin peran yang sesuai.
- **Struktur Respons JSON:** Memastikan *payload* respons memiliki struktur data yang benar, tipe data yang tepat (misalnya, ID berupa *integer*, tanggal berupa *string* ISO 8601), dan *field* wajib tidak bernilai *null*.
- **Konsistensi Data:** Memverifikasi bahwa data yang disimpan (misalnya, anotasi baru) dapat diambil kembali dengan nilai yang sama persis.
- **Logika Bisnis:** Memastikan aturan sistem terpenuhi, seperti *Annotator* tidak dapat mengedit anotasi setelah dokumen di-*submit*, atau *Admin* hanya dapat mengelola proyek yang ditugaskan kepadanya.

6.3.5 Hasil UAT

Berdasarkan eksekusi skenario pengujian menggunakan Postman, berikut adalah ringkasan hasil UAT seperti yang dapat dilihat pada Tabel 6.9. Seluruh skenario pengujian

Kategori Skenario	Total Test Case	Passed	Failed	Pass Rate
Autentikasi	17	17	0	100%
Manajemen User	42	42	0	100%
Manajemen Proyek	14	14	0	100%
Manajemen Dokumen	19	19	0	100%
Anotasi & Review	25	25	0	100%
Dashboard	18	18	0	100%
Total	135	135	0	100%

Tabel 6.9: Ringkasan Hasil UAT *Backend*

jian berhasil dijalankan tanpa adanya kegagalan fungsional (*functional failure*). Validasi input dan penanganan *error* juga telah berfungsi sesuai harapan, di mana sistem menolak permintaan yang tidak valid dengan pesan *error* yang informatif.

6.3.6 Pembahasan

Hasil pengujian menunjukkan bahwa *backend* ANOTA memiliki tingkat stabilitas dan keandalan yang tinggi. Beberapa poin penting dari hasil evaluasi adalah:

- **Keamanan Akses:** Mekanisme kontrol akses berbasis peran (*Role-Based Access Control* - RBAC) berfungsi efektif. Pengujian negatif menunjukkan bahwa pengguna tidak dapat mengakses sumber daya di luar wewenang mereka (misalnya, *Annotator* tidak dapat menghapus proyek).
- **Integritas Data:** Relasi antar entitas (Proyek, Dokumen, Pengguna, Anotasi) terjaga dengan baik. Penghapusan data induk (seperti Proyek) ditangani dengan benar melalui mekanisme *cascade* atau proteksi sesuai konfigurasi model.
- **Kinerja Alur Kerja:** Alur kerja bertahap dari penugasan, anotasi, hingga *review* berjalan mulus. Transisi status dokumen terpicu secara otomatis sesuai tindakan pengguna.

6.3.7 Kesimpulan UAT

Berdasarkan hasil pengujian yang komprehensif, dapat disimpulkan bahwa sistem *backend* telah memenuhi seluruh kriteria penerimaan (*acceptance criteria*) yang ditetapkan. Sistem telah siap untuk diintegrasikan dengan aplikasi *frontend* dan digunakan dalam lingkungan produksi. Seluruh fungsionalitas inti untuk manajemen proyek anotasi teks, mulai dari administrasi hingga operasional anotasi dan *review*, telah terverifikasi berjalan dengan

baik dan bebas dari cacat fungsional kritis.

BAB 7

PENUTUP

Pada bab ini tim penulis memaparkan kesimpulan dari hasil perancangan, implementasi, dan pengujian sistem *backend* ANOTA, serta memberikan saran yang diharapkan dapat menjadi acuan bagi pengembangan penelitian selanjutnya.

7.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan evaluasi yang telah dilakukan, dapat disimpulkan bahwa seluruh tujuan penelitian telah berhasil dicapai, dengan rincian sebagai berikut:

- **Merancang sebuah sistem yang mampu mendukung proses anotasi *dataset* GEC bahasa Indonesia**

Sistem *backend* ANOTA telah berhasil dirancang untuk mendukung proses anotasi *dataset* GEC bahasa Indonesia melalui tahapan analisis dan perancangan yang sistematis. Pada tahap analisis, dilakukan identifikasi kebutuhan fungsional dan non-fungsional yang mencakup alur kerja *annotator*, *reviewer*, dan kepala riset. Tahap perancangan menghasilkan artefak utama berupa *Entity Relationship Diagram* (ERD), skema relasional basis data, *class diagram*, serta spesifikasi *RESTful API* yang mendefinisikan struktur *request*, *response*, dan aturan otorisasi berbasis RBAC. Seluruh rancangan tersebut menjadi dasar dalam membangun sistem yang mampu mendukung pengelolaan proyek, manajemen dokumen, proses anotasi dan *review*, serta pemantauan kualitas data melalui metrik *Inter-Annotator Agreement*.

- **Mengimplementasikan rancangan penelitian menjadi sebuah *platform* anotasi berbasis web yang dapat membantu *annotator*, *reviewer*, dan peneliti dalam melakukan proses anotasi**

Rancangan sistem telah berhasil diimplementasikan menjadi *platform* anotasi berbasis web yang dapat digunakan oleh *annotator*, *reviewer*, dan peneliti. Sistem mengimplementasikan modul autentikasi dan otorisasi berbasis *Role-Based Access Control* (RBAC), manajemen akun, manajemen proyek, pengelolaan dokumen, modul anotasi dan *review*, serta *dashboard* statistik untuk memantau progres anotasi. Proses pengembangan dilakukan menggunakan metodologi Agile dengan pendekatan Kanban untuk

mendukung pengembangan yang iteratif dan adaptif.

- **Melakukan evaluasi terhadap sistem yang dikembangkan untuk memastikan fungsionalitas, kegunaan, serta kesesuaiannya dalam mendukung proses pembangunan *dataset* GEC bahasa Indonesia**

Hasil evaluasi menunjukkan bahwa sistem telah memenuhi aspek fungsionalitas, performa, dan kesesuaian dalam mendukung pembangunan *dataset* GEC bahasa Indonesia. *Unit Testing* mencapai *code coverage* sebesar 100% pada seluruh logika bisnis utama. Hasil *Load Testing* menunjukkan sistem stabil pada beban normal 50 pengguna bersamaan dengan *error rate* 0% dan tetap berfungsi pada beban puncak 150 pengguna dengan tingkat keberhasilan 99,67%, meskipun teridentifikasi adanya *bottleneck* pada proses *login* akibat komputasi *hashing*. Selain itu, *User Acceptance Test* yang dilakukan secara khusus pada sisi *backend* melalui emulasi klien menggunakan *Postman collection* membuktikan bahwa seluruh alur kerja bisnis utama berjalan sesuai spesifikasi dan tanpa kegagalan fungsional.

Dengan demikian, sistem ANOTA yang dikembangkan telah memenuhi seluruh tujuan penelitian dan siap diintegrasikan dengan *frontend* sebagai *platform* pendukung pembangunan *dataset* GEC bahasa Indonesia.

7.2 Saran

Berdasarkan hasil evaluasi kinerja dan potensi pengembangan sistem untuk mendukung riset GEC lebih lanjut, berikut adalah saran yang dapat dilakukan pada penelitian selanjutnya:

1. Optimisasi Kinerja *Backend* Berdasarkan Temuan *Load Test*

Hasil *load testing* menunjukkan adanya peningkatan latensi yang signifikan pada skenario *peak load*, terutama akibat antrean pada *worker* dan beban komputasi pada proses autentikasi. Penelitian selanjutnya disarankan untuk melakukan optimisasi arsitektur, seperti penerapan *caching* (Redis) untuk data statis, penggunaan *asynchronous task queue* (Celery) untuk proses berat, serta penambahan jumlah *worker* guna meningkatkan *throughput* sistem saat menangani konkurensi tinggi.

2. Integrasi Kecerdasan Buatan untuk *AI-Assisted Annotation*

Sistem saat ini sepenuhnya mengandalkan anotasi manual. Untuk meningkatkan efisiensi, penelitian selanjutnya dapat mengintegrasikan model GEC berbasis *Deep*

Learning (seperti Transformer) ke dalam *backend* untuk memberikan saran perbaikan kesalahan secara otomatis. Dalam alur kerja ini, peran *annotator* dan *reviewer* akan bergeser menjadi verifikator yang memvalidasi atau memperbaiki saran dari AI, sehingga mempercepat produksi *dataset*.

3. Eksplorasi dan Standardisasi Taksonomi Jenis Kesalahan (*Error Types*)

Penelitian ini telah menyediakan fitur manajemen tipe kesalahan yang fleksibel. Penelitian selanjutnya disarankan untuk melakukan eksplorasi linguistik yang lebih mendalam guna merumuskan taksonomi jenis kesalahan yang baku dan komprehensif khusus untuk Bahasa Indonesia (misalnya membedakan kesalahan morfologi, sintaksis, dan ortografi secara lebih granular). Hal ini penting agar *dataset* yang dihasilkan memiliki standar yang lebih tinggi dan konsisten.

DAFTAR REFERENSI

- Ahmad, M. O., Markkula, J., and Oivo, M. (2013). Kanban in software development: A systematic literature review. In *39th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 9–16. IEEE.
- Aladwani, A. M. and Palvia, P. C. (2002). Developing and validating an instrument for measuring user-perceived web quality. *Information & Management*, 39(6):467–476.
- Artstein, R. and Poesio, M. (2008). Inter-coder agreement for computational linguistics. *Computational Linguistics*, 34(4):555–596.
- Baskaran, S. (2020). Monitoring and observability for cloud site reliability: Tools and techniques. *TIJER – International Research Journal*, 7(8):14–24.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., and Thomas, D. (2001). Manifesto for agile software development. <https://agilemanifesto.org/>. Diakses 2025.
- Beller, M., Gousios, G., Panichella, A., and Zaidman, A. (2015). How (much) do developers test? In *37th International Conference on Software Engineering, Vol. 2 (ICSE Companion)*, pages 559–562. IEEE.
- Bryant, C., Felice, M., Andersen, Ø. E., and Briscoe, T. (2019). The BEA-2019 shared task on grammatical error correction. In Yannakoudakis, H., Kochmar, E., Leacock, C., Madnani, N., Pilán, I., and Zesch, T., editors, *Proceedings of the Fourteenth Workshop on Innovative Use of NLP for Building Educational Applications*, pages 52–75, Florence, Italy. Association for Computational Linguistics.
- Bryant, C., Felice, M., and Briscoe, T. (2017). Automatic annotation and evaluation of error types for grammatical error correction. In Barzilay, R. and Kan, M.-Y., editors, *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 793–805, Vancouver, Canada. Association for Computational Linguistics.
- Bryant, C., Yuan, Z., Qorib, M. R., Cao, H., Ng, H. T., and Briscoe, T. (2023). Grammatical error correction: A survey of the state of the art. *Computational Linguistics*, 49(3):643–701.
- Cohen, J. (1960). A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46.

- Dahlmeier, D. and Ng, H. T. (2012). Better evaluation for grammatical error correction. In Fosler-Lussier, E., Riloff, E., and Bangalore, S., editors, *Proceedings of the 2012 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pages 568–572, Montréal, Canada. Association for Computational Linguistics.
- Dahlmeier, D., Ng, H. T., and Wu, S. M. (2013). Building a large annotated corpus of learner English: The NUS corpus of learner English. In Tetreault, J., Burstein, J., and Leacock, C., editors, *Proceedings of the Eighth Workshop on Innovative Use of NLP for Building Educational Applications*, pages 22–31, Atlanta, Georgia. Association for Computational Linguistics.
- Davis, F. D. and Venkatesh, V. (2004). Toward preprototype user acceptance testing of new information systems: Implications for software project management. *IEEE Transactions on Engineering Management*, 51(1):31–46.
- Dingsøyr, T., Dybå, T., and Moe, N. B. (2012). A decade of agile methodologies: Towards explaining agile software development. *Journal of Systems and Software*, 85(6):1213–1221.
- Elberzhager, F., Münch, J., and Tichy, M. (2012). A systematic literature review on performance testing. *Information and Software Technology*, 54(1):1–15.
- Ikonen, M., Kettunen, P., and Fagerholm, F. (2011). On the impact of kanban on software project work: An empirical case study investigation. In *16th IEEE International Conference on Engineering of Complex Computer Systems (ICECCS)*, pages 305–314. IEEE.
- Jain, R. (1991). *The Art of Computer Systems Performance Analysis*. John Wiley & Sons.
- Ladas, C. (2009). *Scrumban: Essays on Kanban Systems for Lean Software Development*. Modus Cooperandi Press, Seattle, WA.
- Legramante, G., Bernardino, M., Rodrigues, E., and Basso, F. (2020). Systematic literature review on web performance testing. In *Anais da IV Escola Regional de Engenharia de Software (ERES 2020)*, pages 285–295. Sociedade Brasileira de Computação.
- Lin, N., Zeng, M., Huang, W., Jiang, S., Xiao, L., and Yang, A. (2024). A simple yet effective corpus construction framework for indonesian grammatical error correction.
- Lu, Y., Chen, L., Zhang, Y., Shen, M., Wang, H., Wang, X., van Rechem, C., Fu, T., and Wei, W. (2025). Machine learning for synthetic data generation: A review.
- Marier, S. M., Kong, X., Zhu, L., Chen, X., Badruzzaman, A., and Ramatryana, I. N. A.

- (2025). Castle: Context-aware semantic transformer with knowledge graph enhancement for low-resource grammar correction. *Expert Systems with Applications*, 299:130233.
- Meier, J. D., Farre, C., Bansode, P., Barber, S., and Rea, D. (2007). *Performance Testing Guidance for Web Applications*. Microsoft Press.
- Musyafa, A., Gao, Y., Solyman, A., Khan, S., Cai, W., and Khan, M. F. (2024). Dynamic decoding and dual synthetic data for automatic correction of grammar in low-resource scenario. *PeerJ. Computer science*, 10:e2122.
- Myers, G. J., Sandler, C., and Badgett, T. (2011). *The Art of Software Testing*. John Wiley & Sons, 3 edition.
- Németh, G. Á. and Sótér, P. (2021). Teaching performance testing. *Teaching Mathematics and Computer Science*, 19(1):1733.
- Ng, H. T., Wu, S. M., Briscoe, T., Hadiwinoto, C., Susanto, R. H., and Bryant, C. (2014). The CoNLL-2014 shared task on grammatical error correction. In Ng, H. T., Wu, S. M., Briscoe, T., Hadiwinoto, C., Susanto, R. H., and Bryant, C., editors, *Proceedings of the Eighteenth Conference on Computational Natural Language Learning: Shared Task*, pages 1–14, Baltimore, Maryland. Association for Computational Linguistics.
- Ng, H. T., Wu, S. M., Wu, Y., Hadiwinoto, C., and Tetreault, J. (2013). The CoNLL-2013 shared task on grammatical error correction. In Ng, H. T., Tetreault, J., Wu, S. M., Wu, Y., and Hadiwinoto, C., editors, *Proceedings of the Seventeenth Conference on Computational Natural Language Learning: Shared Task*, pages 1–12, Sofia, Bulgaria. Association for Computational Linguistics.
- Qorib, M. R. and Ng, H. T. (2022). Grammatical error correction: Are we there yet? In *Proceedings of the 29th International Conference on Computational Linguistics*, pages 2794–2800, Gyeongju, Republic of Korea. International Committee on Computational Linguistics.
- Runeson, P. (2006). A survey of unit testing practices. *IEEE Software*, 23(4):22–29.
- Sigelman, B. H., Barroso, L. A., Burrows, M., Stephenson, P., Plakal, M., Beaver, D., Jaspan, S., and Shanbhag, C. (2010). Dapper, a large-scale distributed systems tracing infrastructure. Technical Report dapper-2010-1, Google Inc.
- Stahlberg, F. and Kumar, S. (2021). Synthetic data generation for grammatical error correction with tagged corruption models. In Burstein, J., Horbach, A., Kochmar,

- E., Laarmann-Quante, R., Leacock, C., Madnani, N., Pilán, I., Yannakoudakis, H., and Zesch, T., editors, *Proceedings of the 16th Workshop on Innovative Use of NLP for Building Educational Applications*, pages 37–47, Online. Association for Computational Linguistics.
- Tadjudin, R. (2023). Gramatika: Dataset sintetik untuk grammatical error correction bahasa indonesia = gramatika: A synthetic dataset for indonesian grammatical error correction. Undergraduate thesis, Fakultas Ilmu Komputer, Universitas Indonesia.
- Venkatesh, V., Morris, M. G., Davis, G. B., and Davis, F. D. (2003). User acceptance of information technology: Toward a unified view. *MIS Quarterly*, 27(3):425–478.
- Visconti, A., Mosnek, O., Bro, M., and Maty, V. (2019). Examining pbkdf2 security margincase study of luks. *Journal of Information Security and Applications*, 46:296–306.
- West, M. and Harrison, P. J. (1997). *Bayesian Forecasting and Dynamic Models*. Springer, New York, 2 edition.
- Wilkinson, P. (2019). Metrics, monitoring and observability in modern cloud environments. *Journal of Cloud Computing*, 8(1):1–16.
- Zhang, Y. and Mesbah, A. (2015). Automated web application testing: A systematic review. *IEEE Transactions on Software Engineering*, 41(3):231–264.

LAMPIRAN

Lampiran 1: Pembagian Tugas Pengembangan Aplikasi

Pengembangan Aplikasi merupakan langkah awal dan bagian krusial dalam tugas akhir ANOTA. Dalam pengembangannya kami membagi pengerjaan berdasarkan modul-modul inti sesuai dengan tabel 2.

Juan Dharmananda Khusuma	Lim Bodhi Wijaya
• Modul Anotasi • Modul Review • Unit Testing • Load Testing • Deployment	• Modul Autentikasi • Modul Manajemen User • Modul Manajemen Dokumen • Modul Manajemen Proyek • Modul Dashboard

Tabel 2: Pembagian Tugas Pengembangan Aplikasi

Lampiran 2: Pembagian Tugas Menulis Laporan TA

Setelah melakukan pengembangan aplikasi, penulis menuangkan seluruh pekerjaan yang dilakukan kedalam laporan tugas akhir ini. Pada proses penulisan, tim penulis membagi tugas berdasarkan subbab-subbab yang ada. Namun, perlu dicatat bahwa dalam praktik penulisan laporan, kedua penulis tetap saling membantu dan memberikan kontribusi pada setiap tulisan yang dibubuhkan, walaupun bagian tersebut belum tentu tanggung jawab spesifik seorang penulis. Pembagian tugas menulis dapat terlihat pada tabel di bawah ini.

Juan Dharmananda Khusuma	Lim Bodhi Wijaya
• Subbab 1.1 • Subbab 1.2 • Subbab 1.3 • Subbab 1.4 • Subbab 1.6 • Subbab 1.7 • Subbab 3.3 (Sesuai Pembagian) • Subbab 6.1 • Subbab 6.2 • Subbab 6.3 • Subbab 7.1	• Subbab 1.5 • Subbab 2.1 • Subbab 2.2 • Subbab 2.3 • Subbab 3.1 • Subbab 3.2 • Subbab 3.3 (Sesuai Pembagian) • Subbab 4.1 • Subbab 4.2 • Subbab 4.3 • Subbab 4.4 • Subbab 4.5 • Subbab 6.3 • Subbab 7.2

Tabel 4: Pembagian Tugas Menulis Laporan TA

Lampiran 3: List *endpoint* ANOTA

Bagian ini menunjukkan *list endpoint* yang tersedia pada *backend* ANOTA saat ini, *list* ini dapat digunakan sebagai acuan awal bagi pengembangan aplikasi dan penelitian pada masa yang mendatang.

No	Method	URL	Body	User
1	POST	/api/v1/ login/	username (string), password (string)	All User
2	POST	/api/v1/ token/ refresh/	refresh (string)	All User
3	GET	/api/v1/ users/me/	-	All User
4	GET	/api/v1/ users/me/ assigned -documents/	-	All User
5	GET	/api/v1/ users/me/ projects/	-	All User
6	GET	/api/v1/ users/ available -roles/	-	All User
7	POST	/api/v1/ users/ password -reset/ request/	email (string)	All User
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
8	POST	/api/v1/ users/ password -reset/ verify/	email (string), otp_code (string), new_password (string)	All User
9	DELETE	/api/v1/ users/{user _id}/delete/	-	Kepala Riset
10	GET	/api/v1/ users/{user _id}/projects/ {project_id}/ roles/	-	Admin, Kepala Riset
11	GET	/api/v1/ users/ projects/ {project_id}/ users/	-	Admin, Kepala Riset
12	POST	/api/v1/ users/ projects/ {project_id}/ users/create/	username (string), email (string), full_name (string), institusi (string), send_email (boolean), login_url (string, optional)	Admin, Kepala Riset
13	GET	/api/v1/ users/ projects/ {project_id}/ users/ {user_id}/	-	Admin, Kepala Riset
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
14	PUT/PATCH	/api/v1/ users/ projects/ {project_id}/ users/{user _id}/update/	username (string), email (string), full_name (string)	Admin, Kepala Riset
15	DELETE	/api/v1/ users/ projects/ {project_id}/ users/{user _id}/remove/	-	Admin, Kepala Riset
16	POST	/api/v1/ users/ projects/ {project_id}/ users/{user _id}/reset -password/	send_email (boolean), login_url (string, optional)	Admin, Kepala Riset
17	GET	/api/v1/ users/ projects/ {project_id}/ users/{user _id}/assigned -documents/	-	Admin, Kepala Riset
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
18	GET	/api/v1/ users/ projects/ {project_id}/ available -users/	-	Admin, Kepala Riset
19	POST	/api/v1/ users/ projects/ {project_id}/ role -management/	user_id (uuid), role (string), action (string)	Admin, Kepala Riset
20	GET	/api/v1/ documents/	-	Admin
21	POST	/api/v1/ documents/	title (string), text (string), project (integer), institution (string, optional), allow_duplicate (boolean)	Admin
22	GET	/api/v1/ documents/ {id}/	-	Admin
23	DELETE	/api/v1/ documents/ {id}/	-	Admin
24	GET	/api/v1/ documents/ my-assigned/ {id}/	-	All User
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
25	GET	/api/v1/ documents/ my-assigned/ {id}/ review-queue/	-	Reviewer
26	GET	/api/v1/ documents/ {id}/export/ {fmt}/	-	Admin
27	POST	/api/v1/ documents/ preview/	text (string)	Admin
28	POST	/api/v1/ documents/ assignment/	document_id (integer), user_id (uuid), deadline (date), notes (string, optional)	Admin
29	GET	/api/v1/ projects/	-	Admin, Kepala Riset
30	POST	/api/v1/ projects/	name (string), description (string), assigned_admins (array[uuid])	Kepala Riset
31	GET	/api/v1/ projects/ {id}/	-	Admin, Kepala Riset
32	PUT/PATCH	/api/v1/ projects/ {id}/	name (string), description (string)	Admin, Kepala Riset
33	DELETE	/api/v1/ projects/ {id}/	-	Admin, Kepala Riset
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
34	POST	/api/v1/ projects/ {id}/ assign-admin/	user_id (uuid)	Kepala Riset
35	POST	/api/v1/ projects/ {id}/unassign _admin/	user_id (uuid)	Kepala Riset
36	GET	/api/v1/ projects/ {id}/ documents/	-	Admin, Kepala Riset
37	GET	/api/v1/ projects/ {id}/assigned -users/	-	Admin, Kepala Riset
38	GET	/api/v1/ annotations/ error-types/	-	All User
39	POST	/api/v1/ annotations/ error-types/	name (string), description (string), project (integer)	Admin
40	GET	/api/v1/ annotations/ error-types/ {id}/	-	All User
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
41	PUT/PATCH	/api/v1/ annotations/ error-types/ {id}/	name (string), description (string)	Admin
42	DELETE	/api/v1/ annotations/ error-types/ {id}/	-	Admin
43	GET	/api/v1/ annotations/	-	Annotator
44	POST	/api/v1/ annotations/	document (integer), sentence (integer), error_type (integer), start_idx (integer), end_idx (integer), correction (string), notes (string, optional)	Annotator
45	GET	/api/v1/ annotations/ {id}/	-	Annotator
46	PUT/PATCH	/api/v1/ annotations/ {id}/	document (integer), sentence (integer), error_type (integer), start_idx (integer), end_idx (integer), correction (string), notes (string, optional)	Annotator
47	DELETE	/api/v1/ annotations/ {id}/	-	Annotator
48	POST	/api/v1/ annotations/ submit/	document (integer)	Annotator
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
49	POST	/api/v1/ annotations/ reopen/	document (integer), reason (string, optional)	Annotator
50	POST	/api/v1/ annotations/ admin/ projects/ {project_id}/ reopen -annotator/	document_id (integer), annotator_id (uuid), reason (string)	Admin, Kepala Riset
51	GET	/api/v1/ annotations/ reviews/	-	Reviewer
52	POST	/api/v1/ annotations/ reviews/	document (integer), sentence (integer), annotation (integer), status (string), feedback (string, optional)	Reviewer
53	GET	/api/v1/ annotations/ reviews/{id}/	-	Reviewer
54	PUT/PATCH	/api/v1/ annotations/ reviews/{id}/	document (integer), sentence (integer), annotation (integer), status (string), feedback (string, optional)	Reviewer
55	DELETE	/api/v1/ annotations/ reviews/{id}/	-	Reviewer
56	POST	/api/v1/ annotations/ reviews/ submit/	document (integer)	Reviewer
Bersambung ke halaman berikutnya				

Lanjutan dari halaman sebelumnya

No	Method	URL	Body	User
57	POST	/api/v1/ annotations/ reviews/ reopen/	document (integer), reason (string, optional)	Reviewer
58	POST	/api/v1/ annotations/ admin/ projects/ {project_id}/ reopen -review/	document_id (integer), reviewer_id (uuid), reason (string)	Admin, Kepala Riset
59	GET	/api/v1/ dashboard/	-	Admin, Kepala Riset
60	GET	/api/v1/ dashboard/ annotators/ {user_id}/	-	Admin, Kepala Riset
61	GET	/api/v1/ dashboard/ reviewers/ {user_id}/	-	Admin, Kepala Riset
62	GET	/api/v1/ dashboard/ iaa/	-	Admin, Kepala Riset

Tabel 5: List endpoint ANOTA

Lampiran 4: List User Acceptance Test Backlog

Skenario Pengujian (Normal Flow)**Modul Autentikasi****AUTH-001 – Login**

Deskripsi: Verifikasi pengguna dapat login menggunakan username

Langkah-langkah:

1. Buka halaman login
2. Masukkan username yang valid
3. Masukkan password yang valid
4. Klik tombol login

Ekspektasi:

Pengguna berhasil login dan mendapatkan token akses

AUTH-002 – Refresh token

Deskripsi: Verifikasi pengguna dapat memperbarui token akses

Langkah-langkah:

1. Login dengan kredensial valid
2. Gunakan refresh token untuk mendapatkan token akses baru
3. Kirim request ke endpoint refresh token

Ekspektasi:

Sistem mengembalikan token akses baru yang valid

AUTH-003 – Reset password - request OTP

Deskripsi: Verifikasi pengguna dapat meminta OTP untuk reset password

Langkah-langkah:

1. Buka halaman reset password
2. Masukkan email yang terdaftar

3. Klik tombol kirim OTP

Ekspektasi:

Sistem mengirim OTP ke email dan menampilkan pesan sukses

AUTH-004 – Reset password - verifikasi OTP

Deskripsi: Verifikasi pengguna dapat reset password dengan OTP

Langkah-langkah:

1. Masukkan email yang terdaftar
2. Masukkan OTP yang valid (6 digit)
3. Masukkan password baru
4. Konfirmasi password baru

Ekspektasi:

Password berhasil diubah dan pengguna dapat login dengan password baru

AUTH-005 – Reset password

Deskripsi: Verifikasi admin/kepala riset dapat reset password dari pengguna yang dikelolanya

Langkah-langkah:

1. Login ke sistem sebagai Admin/Kepala Riset
2. Akses endpoint `users/projects/{project_id}/users/{user_id}/reset-password/`

Ekspektasi:

Password berhasil di generate secara otomatis dan dikirimkan ke email pengguna

Modul Manajemen User

USER-001 – Lihat informasi pengguna saat ini

Deskripsi: Verifikasi pengguna dapat melihat informasi profilnya

Langkah-langkah:

1. Login ke sistem

2. Akses endpoint `/api/v1/users/me/`

Ekspektasi:

Sistem menampilkan informasi lengkap pengguna (id, username, email, full_name, roles, dll)

USER-002 – Lihat dokumen yang ditugaskan

Deskripsi: Verifikasi pengguna dapat melihat dokumen yang ditugaskan kepadanya

Langkah-langkah:

1. Login sebagai annotator/reviewer
2. Akses endpoint `/api/v1/users/me/assigned-documents/`

Ekspektasi:

Sistem menampilkan daftar dokumen yang ditugaskan ke pengguna

USER-003 – Lihat proyek pengguna

Deskripsi: Verifikasi pengguna dapat melihat proyek di mana ia memiliki peran

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint `/api/v1/users/me/projects/`

Ekspektasi:

- Sistem menampilkan daftar proyek di mana pengguna memiliki peran - Kepala Riset dapat melihat semua project yang tersedia

USER-004 – Buat pengguna baru dalam proyek

Deskripsi: Verifikasi admin dapat membuat pengguna baru dalam proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/users/projects/{project_id}/users/create/`
3. Isi form dengan data pengguna baru

Ekspektasi:

- Pengguna baru berhasil dibuat dengan password otomatis - Pengguna baru terhubung secara otomatis dengan role tersebut pada projects - Admin hanya dapat membuat pengguna baru pada project yang di kelola

USER-005 – Update informasi pengguna

Deskripsi: Verifikasi admin dapat memperbarui informasi pengguna

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/users/projects/{project_id}/users/{user_id}/update/`
3. Ubah data pengguna

Ekspektasi:

- Admin hanya dapat update informasi pengguna yang ada pada project yang dikelola - Kepala Riset dapat update informasi seluruh pengguna

USER-006 – Hapus pengguna dari proyek

Deskripsi: Verifikasi admin dapat menghapus pengguna dari proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/users/projects/{project_id}/users/{user_id}/remove/`

Ekspektasi:

Pengguna berhasil dihapus dari proyek (semua peran dihapus)

USER-007 – Reset password pengguna - Admin Reset

Deskripsi: Verifikasi admin dapat mereset password pengguna

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/users/projects/{project_id}/users/{user_id}/reset-password/`

3. Kirim request reset

Ekspektasi:

Admin: - Admin hanya dapat mereset password pengguna yang ada pada project yang dikelolanya - Password berhasil di generate secara otomatis dan dikirimkan ke email pengguna

Kepala Riset: - Kepala Riset dapat mereset semua password pengguna - Password berhasil di generate secara otomatis dan dikirimkan ke email pengguna

USER-008 – Kelola peran pengguna dalam proyek

Deskripsi: Verifikasi admin/kepala riset dapat menambah/menghapus peran pengguna

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/users/projects/{project_id}/role-management/`
3. Tambah/hapus peran pengguna

Ekspektasi:

- Admin hanya dapat menambahkan pengguna dengan role Annotator dan Reviewer -
Kepala Riset hanya dapat menambahkan pengguna sebagai admin pada project tersebut

USER-009 – Lihat daftar peran tersedia

Deskripsi: Verifikasi pengguna dapat melihat daftar peran yang tersedia

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint `/api/v1/users/available-roles/`

Ekspektasi:

Sistem menampilkan daftar peran yang tersedia (Admin, Annotator, Reviewer)

USER-010 – Soft delete pengguna

Deskripsi: Verifikasi kepala riset dapat melakukan soft delete pengguna

Langkah-langkah:

1. Login sebagai kepala riset
2. Akses endpoint `/api/v1/users/{user_id}/delete/`

Ekspektasi:

Pengguna ditandai sebagai deleted dan dinonaktifkan

USER-011 – Lihat daftar pengguna dalam proyek

Deskripsi: Verifikasi admin dapat melihat daftar pengguna dalam proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{project_id}/users/`

Ekspektasi:

Sistem menampilkan daftar pengguna yang terkait dengan proyek

USER-012 – Lihat detail pengguna dalam proyek

Deskripsi: Verifikasi admin dapat melihat detail pengguna dalam proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{project_id}/users/{user_id}/`

Ekspektasi:

Sistem menampilkan detail lengkap pengguna

USER-013 – Lihat pengguna tersedia untuk proyek

Deskripsi: Verifikasi admin dapat melihat pengguna yang tersedia untuk proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{project_id}/available-users/`

Ekspektasi:

Sistem menampilkan daftar pengguna aktif dengan indikator membership

USER-014 – Lihat dokumen yang ditugaskan ke pengguna

Deskripsi: Verifikasi admin/Kepala Riset dapat melihat dokumen yang ditugaskan ke pengguna

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{project_id}/users/{user_id}/assigned-documents/`

Ekspektasi:

Sistem menampilkan daftar dokumen yang ditugaskan ke pengguna pada project tersebut

USER-015 – Lihat peran pengguna dalam proyek

Deskripsi: Verifikasi pengguna dapat melihat perannya dalam proyek

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint `/api/v1/users/{user_id}/projects/{project_id}/roles/`

Ekspektasi:

Sistem menampilkan daftar peran pengguna dalam proyek

Modul Manajemen Proyek

PROJ-001 – Lihat daftar proyek

Deskripsi: Verifikasi admin dapat melihat daftar proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/`

Ekspektasi:

- Admin: Sistem menampilkan daftar proyek yang dikelolanya - Kepala Riset: Sistem

menampilkan seluruh daftar proyek

PROJ-002 – Buat proyek baru

Deskripsi: Verifikasi kepala riset dapat membuat proyek baru

Langkah-langkah:

1. Login sebagai kepala riset
2. Akses endpoint `/api/v1/projects/`
3. Isi form dengan data proyek

Ekspektasi:

Proyek baru berhasil dibuat

PROJ-003 – Lihat detail proyek

Deskripsi: Verifikasi Admin/Kepala Riset dapat melihat detail proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{id}/`

Ekspektasi:

Sistem menampilkan detail lengkap proyek

PROJ-004 – Update informasi proyek

Deskripsi: Verifikasi admin dapat memperbarui informasi proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{id}/`
3. Ubah data proyek

Ekspektasi:

Informasi proyek berhasil diperbarui

PROJ-005 – Hapus proyek

Deskripsi: Verifikasi admin dapat menghapus proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{id}/`
3. Kirim request DELETE

Ekspektasi:

Proyek berhasil dihapus

PROJ-006 – Assign admin ke proyek

Deskripsi: Verifikasi kepala riset dapat menambah admin ke proyek

Langkah-langkah:

1. Login sebagai kepala riset
2. Akses endpoint `/api/v1/projects/{id}/assign_admin/`
3. Pilih user untuk diassign

Ekspektasi:

Admin berhasil ditambahkan ke proyek

PROJ-007 – Hapus admin dari proyek

Deskripsi: Verifikasi kepala riset dapat menghapus admin dari proyek

Langkah-langkah:

1. Login sebagai kepala riset
2. Akses endpoint `/api/v1/projects/{id}/unassign_admin/`
3. Pilih user untuk diunassign

Ekspektasi:

Admin berhasil dihapus dari proyek

PROJ-008 – Lihat dokumen dalam proyek

Deskripsi: Verifikasi admin/kepala riset dapat melihat dokumen dalam proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{id}/documents/`

Ekspektasi:

Sistem menampilkan daftar dokumen dalam proyek

PROJ-009 – Lihat pengguna yang ditugaskan dalam proyek

Deskripsi: Verifikasi admin/kepala riset dapat melihat pengguna yang ditugaskan dalam proyek

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/projects/{id}/assigned-users/`

Ekspektasi:

Sistem menampilkan daftar pengguna yang ditugaskan dalam proyek

Modul Manajemen Dokumen

DOC-001 – Lihat daftar dokumen

Deskripsi: Verifikasi admin/kepala riset dapat melihat daftar dokumen

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/documents/`

Ekspektasi:

Sistem menampilkan daftar dokumen dengan pagination

DOC-002 – Buat dokumen baru

Deskripsi: Verifikasi admin dapat membuat dokumen baru

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/`
3. Isi form dengan judul dan teks dokumen

Ekspektasi:

Dokumen baru berhasil dibuat dengan kalimat otomatis terpisah

DOC-003 – Lihat detail dokumen

Deskripsi: Verifikasi admin dapat melihat detail dokumen

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/{id}/`

Ekspektasi:

Sistem menampilkan detail dokumen dengan semua kalimat

DOC-004 – Update dokumen

Deskripsi: Verifikasi admin dapat memperbarui dokumen

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/{id}/`
3. Ubah data dokumen

Ekspektasi:

Dokumen berhasil diperbarui

DOC-005 – Hapus dokumen

Deskripsi: Verifikasi admin dapat menghapus dokumen

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/{id}/`

3. Kirim request DELETE

Ekspektasi:

Dokumen berhasil dihapus (cascade ke kalimat dan anotasi)

DOC-006 – Lihat detail dokumen yang ditugaskan

Deskripsi: Verifikasi annotator/reviewer dapat melihat detail dokumen yang ditugaskan

Langkah-langkah:

1. Login sebagai annotator/reviewer
2. Akses endpoint `/api/v1/documents/my-assigned/{id}/`

Ekspektasi:

Sistem menampilkan detail dokumen dengan status spesifik pengguna

DOC-007 – Lihat antrian review untuk dokumen

Deskripsi: Verifikasi reviewer dapat melihat anotasi siap review

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/documents/my-assigned/{id}/review-queue/`

Ekspektasi:

Sistem menampilkan daftar anotasi siap review

DOC-008 – Tugaskan dokumen ke pengguna

Deskripsi: Verifikasi admin dapat menugaskan dokumen ke pengguna

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/assignment/`
3. Pilih dokumen dan user

Ekspektasi:

Dokumen berhasil ditugaskan ke user

DOC-009 – Export dokumen format parallel

Deskripsi: Verifikasi admin dapat export dokumen format TSV

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/{id}/export/parallel/`

Ekspektasi:

Sistem mengunduh file TSV dengan format original corrected

DOC-010 – Export dokumen format M2

Deskripsi: Verifikasi admin dapat export dokumen format M2

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/{id}/export/m2/`

Ekspektasi:

Sistem mengunduh file M2 untuk koreksi error tata bahasa

DOC-011 – Preview jumlah kalimat

Deskripsi: Verifikasi admin dapat preview jumlah kalimat sebelum membuat dokumen

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/preview/`
3. Masukkan teks

Ekspektasi:

Sistem menampilkan jumlah kalimat dan daftar kalimat

DOC-012 – Hapus assignment dokumen

Deskripsi: Verifikasi admin dapat menghapus assignment dokumen dari user

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/documents/assignment/` dengan method DELETE
3. Pilih dokumen dan user

Ekspektasi:

Assignment dokumen berhasil dihapus

Modul Anotasi & Review

ANNO-001 – Lihat daftar tipe error

Deskripsi: Verifikasi pengguna dapat melihat daftar tipe error

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint `/api/v1/annotations/error-types/`

Ekspektasi:

Sistem menampilkan daftar tipe error

ANNO-002 – Buat tipe error baru

Deskripsi: Verifikasi admin dapat membuat tipe error baru

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/annotations/error-types/`
3. Isi form dengan data tipe error

Ekspektasi:

Tipe error baru berhasil dibuat

ANNO-004 – Update tipe error

Deskripsi: Verifikasi admin dapat memperbarui tipe error

Langkah-langkah:

1. Login sebagai admin

2. Akses endpoint `/api/v1/annotations/error-types/{id}/`
3. Ubah data tipe error

Ekspektasi:

Tipe error berhasil diperbarui

ANNO-005 – Hapus tipe error

Deskripsi: Verifikasi admin dapat menghapus tipe error

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint `/api/v1/annotations/error-types/{id}/`
3. Kirim request DELETE

Ekspektasi:

Tipe error berhasil dihapus

ANNO-006 – Lihat daftar anotasi

Deskripsi: Verifikasi annotator dapat melihat daftar anotasinya

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/`

Ekspektasi:

Sistem menampilkan daftar anotasi oleh pengguna saat ini

ANNO-007 – Buat anotasi baru

Deskripsi: Verifikasi annotator dapat membuat anotasi baru

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/`
3. Isi form dengan data anotasi

Ekspektasi:

Anotasi baru berhasil dibuat

ANNO-008 – Lihat detail anotasi

Deskripsi: Verifikasi annotator dapat melihat detail anotasi

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/{id}/`

Ekspektasi:

Sistem menampilkan detail anotasi

ANNO-009 – Update anotasi

Deskripsi: Verifikasi annotator dapat memperbarui anotasi

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/{id}/`
3. Ubah data anotasi

Ekspektasi:

Anotasi berhasil diperbarui

ANNO-010 – Hapus anotasi

Deskripsi: Verifikasi annotator dapat menghapus anotasi

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/{id}/`
3. Kirim request DELETE

Ekspektasi:

Anotasi berhasil dihapus

ANNO-011 – Submit anotasi untuk dokumen

Deskripsi: Verifikasi annotator dapat submit semua anotasi untuk dokumen

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/submit/`
3. Pilih dokumen untuk disubmit

Ekspektasi:

Semua anotasi untuk dokumen berhasil disubmit

ANNO-012 – Buka kembali dokumen untuk anotasi

Deskripsi: Verifikasi annotator dapat membuka kembali dokumen yang disubmit

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/reopen/`
3. Pilih dokumen untuk dibuka kembali

Ekspektasi:

Dokumen berhasil dibuka kembali untuk anotasi

ANNO-013 – Lihat daftar review

Deskripsi: Verifikasi reviewer dapat melihat daftar reviewnya

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/`

Ekspektasi:

Sistem menampilkan daftar review oleh pengguna saat ini

ANNO-014 – Buat review baru

Deskripsi: Verifikasi reviewer dapat membuat review baru

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/`
3. Isi form dengan data review

Ekspektasi:

Review baru berhasil dibuat

ANNO-015 – Lihat detail review

Deskripsi: Verifikasi reviewer dapat melihat detail review

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/{id}/`

Ekspektasi:

Sistem menampilkan detail review

ANNO-016 – Update review

Deskripsi: Verifikasi reviewer dapat memperbarui review

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/{id}/`
3. Ubah data review

Ekspektasi:

Review berhasil diperbarui

ANNO-017 – Hapus review

Deskripsi: Verifikasi reviewer dapat menghapus review

Langkah-langkah:

1. Login sebagai reviewer

2. Akses endpoint `/api/v1/annotations/reviews/{id}/`
3. Kirim request DELETE

Ekspektasi:

Review berhasil dihapus

ANNO-018 – Submit review untuk dokumen

Deskripsi: Verifikasi reviewer dapat submit semua review untuk dokumen

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/submit/`
3. Pilih dokumen untuk disubmit

Ekspektasi:

Semua review untuk dokumen berhasil disubmit

ANNO-019 – Buka kembali dokumen untuk review

Deskripsi: Verifikasi reviewer dapat membuka kembali dokumen review yang disubmit

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/reopen/`
3. Pilih dokumen untuk dibuka kembali

Ekspektasi:

Dokumen berhasil dibuka kembali untuk review

ANNO-020 – Admin buka kembali dokumen annotator

Deskripsi: Verifikasi admin dapat membuka kembali dokumen annotator yang disubmit

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/annotations/admin/projects/{project_id}/reopen-annotator/`

3. Pilih dokumen dan annotator

Ekspektasi:

Dokumen annotator berhasil dibuka kembali

ANNO-021 – Admin buka kembali dokumen reviewer

Deskripsi: Verifikasi admin dapat membuka kembali dokumen reviewer yang disubmit

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/annotations/admin/projects/{project_id}/reopen-review/`
3. Pilih dokumen dan reviewer

Ekspektasi:

Dokumen reviewer berhasil dibuka kembali

ANNO-022 – Lihat daftar anotasi milik saya

Deskripsi: Verifikasi annotator dapat melihat daftar anotasi miliknya

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/`
3. Periksa response

Ekspektasi:

Daftar anotasi milik annotator ditampilkan

ANNO-023 – Lihat detail anotasi milik saya

Deskripsi: Verifikasi annotator dapat melihat detail anotasi miliknya

Langkah-langkah:

1. Login sebagai annotator
2. Akses endpoint `/api/v1/annotations/{id}/`
3. Periksa response

Ekspektasi:

Detail anotasi lengkap ditampilkan

ANNO-024 – Lihat daftar review milik saya

Deskripsi: Verifikasi reviewer dapat melihat daftar review miliknya

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/`
3. Periksa response

Ekspektasi:

Daftar review milik reviewer ditampilkan

ANNO-025 – Lihat detail review milik saya

Deskripsi: Verifikasi reviewer dapat melihat detail review miliknya

Langkah-langkah:

1. Login sebagai reviewer
2. Akses endpoint `/api/v1/annotations/reviews/{id}/`
3. Periksa response

Ekspektasi:

Detail review lengkap ditampilkan

Modul Dashboard**DASH-001 – Lihat ringkasan dashboard**

Deskripsi: Verifikasi admin dapat melihat ringkasan dashboard

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/dashboard/`

Ekspektasi:

Sistem menampilkan statistik keseluruhan (total dokumen, anotasi, review, dll)

DASH-002 – Lihat dashboard annotator

Deskripsi: Verifikasi admin dapat melihat dashboard performa annotator

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/dashboard/annotators/{user_id}/`

Ekspektasi:

Sistem menampilkan statistik performa annotator

DASH-003 – Lihat dashboard reviewer

Deskripsi: Verifikasi admin dapat melihat dashboard performa reviewer

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/dashboard/reviewers/{user_id}/`

Ekspektasi:

Sistem menampilkan statistik performa reviewer

DASH-004 – Hitung Inter-Annotator Agreement (IAA)

Deskripsi: Verifikasi admin dapat menghitung IAA antara annotator dan reviewer

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint `/api/v1/dashboard/iaa/`
3. Pilih annotator dan reviewer

Ekspektasi:

Sistem menampilkan skor IAA dan detail perbandingan

Skenario Pengujian (Negative Flow)

Negative Case - Modul Autentikasi

AUTH-NEG-001 – Login dengan kredensial salah

Deskripsi: Verifikasi sistem menangani skenario Login dengan kredensial salah

Langkah-langkah:

1. Buka halaman login
2. Masukkan username valid
3. Masukkan password salah
4. Klik tombol login

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Kredensial tidak valid"}
```

AUTH-NEG-002 – Login dengan username tidak terdaftar

Deskripsi: Verifikasi sistem menangani skenario Login dengan username tidak terdaftar

Langkah-langkah:

1. Buka halaman login
2. Masukkan username tidak terdaftar
3. Masukkan password sembarang
4. Klik tombol login

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Kredensial tidak valid"}
```

AUTH-NEG-003 – Login dengan field kosong

Deskripsi: Verifikasi sistem menangani skenario Login dengan field kosong

Langkah-langkah:

1. Buka halaman login
2. Tidak mengisi username dan password
3. Klik tombol login

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Username/email dan password harus diisi" }
```

AUTH-NEG-004 – Login dengan akun tidak aktif

Deskripsi: Verifikasi sistem menangani skenario Login dengan akun tidak aktif

Langkah-langkah:

1. Buka halaman login
2. Masukkan kredensial akun soft delete
3. Klik tombol login

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Akun tidak aktif" }
```

AUTH-NEG-005 – Refresh token dengan token invalid

Deskripsi: Verifikasi sistem menangani skenario Refresh token dengan token invalid

Langkah-langkah:

1. Kirim request ke endpoint refresh token
2. Gunakan refresh token invalid/expired

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Token is invalid or expired",  
  "code": "token_not_valid" }
```

AUTH-NEG-006 – Refresh token dengan token kosong

Deskripsi: Verifikasi sistem menangani skenario Refresh token dengan token kosong

Langkah-langkah:

1. Kirim request ke endpoint refresh token tanpa refresh token

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "refresh": [  
  "This field is required."  
]}
```

AUTH-NEG-007 – Reset password - request OTP email tidak terdaftar

Deskripsi: Verifikasi sistem menangani skenario Reset password - request OTP email tidak terdaftar

Langkah-langkah:

1. Buka halaman reset password
2. Masukkan email tidak terdaftar
3. Klik kirim OTP

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "email": [  
  "Email tidak terdaftar dalam sistem."  
]}
```

AUTH-NEG-008 – Reset password - request OTP email akun tidak aktif

Deskripsi: Verifikasi sistem menangani skenario Reset password - request OTP email akun tidak aktif

Langkah-langkah:

1. Buka halaman reset password

2. Masukkan email akun soft delete
3. Klik kirim OTP

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "email": [
  "Akun tidak aktif."
]}
```

AUTH-NEG-009 – Reset password - request OTP melebihi rate limit

Deskripsi: Verifikasi sistem menangani skenario Reset password - request OTP melebihi rate limit

Langkah-langkah:

1. Request OTP lebih dari 5 kali dalam 1 jam
2. Lakukan request OTP keenam

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "non_field_errors": [
  "Terlalu banyak permintaan reset password. Coba lagi dalam 1
jam."
]}
```

AUTH-NEG-010 – Reset password - verifikasi OTP kode salah

Deskripsi: Verifikasi sistem menangani skenario Reset password - verifikasi OTP kode salah

Langkah-langkah:

1. Masukkan email valid
2. Masukkan OTP salah
3. Masukkan password baru

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "non_field_errors": [
  "Kode OTP tidak valid."
]}
```

AUTH-NEG-011 – Reset password - verifikasi OTP expired

Deskripsi: Verifikasi sistem menangani skenario Reset password - verifikasi OTP expired

Langkah-langkah:

1. Masukkan email valid
2. Masukkan OTP melewati 15 menit
3. Masukkan password baru

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "non_field_errors": [
  "OTP sudah kedaluwarsa. Silakan request OTP baru."
]}
```

AUTH-NEG-012 – Reset password - verifikasi OTP format tidak valid

Deskripsi: Verifikasi sistem menangani skenario Reset password - verifikasi OTP format tidak valid

Langkah-langkah:

1. Masukkan email valid
2. Masukkan OTP non-angka
3. Masukkan password baru

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "otp_code": [
  "Kode OTP harus berupa 6 digit angka."
]}
```

AUTH-NEG-013 – Reset password - password baru terlalu pendek

Deskripsi: Verifikasi sistem menangani skenario Reset password - password baru terlalu pendek

Langkah-langkah:

1. Masukkan email valid
2. Masukkan OTP valid
3. Masukkan password baru ;6 karakter

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "new_password": [
  "Ensure this field has at least 6 characters."
]}
```

Negative Case - Modul Manajemen User

USER-NEG-001 – Lihat informasi pengguna tanpa autentikasi

Deskripsi: Verifikasi sistem menangani skenario Lihat informasi pengguna tanpa autentikasi

Langkah-langkah:

1. Akses endpoint /api/v1/users/me/ tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided."}
```

USER-NEG-002 – Lihat dokumen ditugaskan tanpa autentikasi

Deskripsi: Verifikasi sistem menangani skenario Lihat dokumen ditugaskan tanpa autentikasi

Langkah-langkah:

1. Akses endpoint `/api/v1/users/me/assigned-documents/` tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided." }
```

USER-NEG-003 – Lihat proyek pengguna tanpa autentikasi

Deskripsi: Verifikasi sistem menangani skenario Lihat proyek pengguna tanpa autentikasi

Langkah-langkah:

1. Akses endpoint `/api/v1/users/me/projects/` tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided." }
```

USER-NEG-004 – Buat pengguna baru dengan username sudah ada

Deskripsi: Verifikasi sistem menangani skenario Buat pengguna baru dengan username sudah ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint create user
3. Isi form dengan username terdaftar

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "username": [
  "Username sudah digunakan."
]}
```

USER-NEG-005 – Buat pengguna baru dengan email sudah ada

Deskripsi: Verifikasi sistem menangani skenario Buat pengguna baru dengan email sudah ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint create user
3. Isi form dengan email terdaftar

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "email": [
  "Email sudah terdaftar."
]}
```

USER-NEG-006 – Buat pengguna baru tanpa field wajib

Deskripsi: Verifikasi sistem menangani skenario Buat pengguna baru tanpa field wajib

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint create user
3. Kirim request tanpa username, email, full_name

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "username": [
  "This field is required."
]}
```

```

],
"email": [
  "This field is required."
],
"full_name": [
  "This field is required."
]}

```

USER-NEG-007 – Buat pengguna baru sebagai non-admin

Deskripsi: Verifikasi sistem menangani skenario Buat pengguna baru sebagai non-admin

Langkah-langkah:

1. Login annotator/reviewer
2. Akses endpoint create user

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action."}
```

USER-NEG-008 – Update informasi pengguna tidak ada

Deskripsi: Verifikasi sistem menangani skenario Update informasi pengguna tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint update user dengan UUID tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Not found."}
```

USER-NEG-009 – Update username ke username sudah ada

Deskripsi: Verifikasi sistem menangani skenario Update username ke username sudah ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint update user
3. Ubah username ke username user lain

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "username": [
  "Username sudah digunakan."
]}
```

USER-NEG-010 – Update email ke email sudah ada

Deskripsi: Verifikasi sistem menangani skenario Update email ke email sudah ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint update user
3. Ubah email ke email user lain

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "email": [
  "Email sudah terdaftar."
]}
```

USER-NEG-011 – Hapus pengguna dari proyek tanpa akses

Deskripsi: Verifikasi sistem menangani skenario Hapus pengguna dari proyek tanpa akses

Langkah-langkah:

1. Login annotator
2. Akses endpoint remove user dari proyek

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action." }
```

USER-NEG-012 – Hapus pengguna tidak ada dari proyek

Deskripsi: Verifikasi sistem menangani skenario Hapus pengguna tidak ada dari proyek

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint remove user dengan UUID user tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "User tidak ditemukan" }
```

USER-NEG-013 – Hapus pengguna dari proyek tidak ada

Deskripsi: Verifikasi sistem menangani skenario Hapus pengguna dari proyek tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint remove user dengan project_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "Project tidak ditemukan" }
```

USER-NEG-014 – Reset password pengguna tanpa akses project

Deskripsi: Verifikasi sistem menangani skenario Reset password pengguna tanpa akses

project

Langkah-langkah:

1. Login admin project A
2. Coba reset password user di project B

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "error": "Anda tidak memiliki akses untuk mengelola user ini di  
project ini" }
```

USER-NEG-015 – Reset password pengguna dengan user_id kosong

Deskripsi: Verifikasi sistem menangani skenario Reset password pengguna dengan user_id kosong

Langkah-langkah:

1. Login admin/kepala riset
2. Kirim request reset password tanpa user_id

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "error": "user_id harus diisi" }
```

USER-NEG-016 – Kelola peran dengan role tidak valid

Deskripsi: Verifikasi sistem menangani skenario Kelola peran dengan role tidak valid

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint role management
3. Kirim role tidak valid

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "role": [
  "Role harus salah satu dari: Admin, Annotator, Reviewer"
]}
```

USER-NEG-017 – Kelola peran dengan action tidak valid

Deskripsi: Verifikasi sistem menangani skenario Kelola peran dengan action tidak valid

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint role management
3. Kirim action tidak valid

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "action": [
  "Action harus salah satu dari: add, remove"
]}
```

USER-NEG-018 – Kelola peran untuk user tidak ada

Deskripsi: Verifikasi sistem menangani skenario Kelola peran untuk user tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint role management
3. Kirim user_id tidak ada

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "user_id": [
  "Pengguna tidak ditemukan."
]}
```

USER-NEG-019 – Lihat daftar peran tersedia tanpa autentikasi

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar peran tersedia tanpa autentikasi

Langkah-langkah:

1. Akses endpoint /api/v1/users/available-roles/ tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided." }
```

USER-NEG-020 – Soft delete pengguna sebagai non-kepala riset

Deskripsi: Verifikasi sistem menangani skenario Soft delete pengguna sebagai non-kepala riset

Langkah-langkah:

1. Login admin (bukan kepala riset)
2. Akses endpoint global delete user

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action." }
```

USER-NEG-021 – Lihat daftar pengguna dalam proyek tanpa akses

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar pengguna dalam proyek tanpa akses

Langkah-langkah:

1. Login annotator
2. Akses endpoint list users dalam proyek

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action."}
```

USER-NEG-022 – Lihat daftar pengguna dalam proyek tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar pengguna dalam proyek tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint list users dengan project_id tidak ada

Ekspektasi:

Response: 200 OK;

Body:

```
{ "count": 0,
  "results": []}
```

USER-NEG-023 – Lihat detail pengguna tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat detail pengguna tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint user detail dengan UUID tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Not found."}
```

USER-NEG-024 – Lihat pengguna tersedia untuk proyek tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat pengguna tersedia untuk proyek tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint available users dengan project_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "Project tidak ditemukan" }
```

USER-NEG-025 – Lihat dokumen yang ditugaskan ke user tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat dokumen yang ditugaskan ke user tidak ada

Langkah-langkah:

1. Login admin/kepala riset
2. Akses endpoint assigned documents dengan user_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "User atau project tidak ditemukan" }
```

USER-NEG-026 – Lihat peran pengguna dalam proyek - user tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat peran pengguna dalam proyek - user tidak ada

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint roles dengan user_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "User tidak ditemukan" }
```

USER-NEG-027 – Lihat peran pengguna dalam proyek - proyek tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat peran pengguna dalam proyek - proyek tidak ada

Langkah-langkah:

1. Login ke sistem
2. Akses endpoint roles dengan project_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "Project tidak ditemukan" }
```

Negative Case - Modul Manajemen Proyek**PROJ-NEG-001 – Lihat daftar proyek tanpa autentikasi**

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar proyek tanpa autentikasi

Langkah-langkah:

1. Akses endpoint /api/v1/projects/ tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided." }
```

PROJ-NEG-002 – Buat proyek baru sebagai admin (bukan kepala riset)

Deskripsi: Verifikasi sistem menangani skenario Buat proyek baru sebagai admin (bukan kepala riset)

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint create project

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action."}
```

PROJ-NEG-003 – Buat proyek baru tanpa nama

Deskripsi: Verifikasi sistem menangani skenario Buat proyek baru tanpa nama

Langkah-langkah:

1. Login sebagai kepala riset
2. Kirim request create project tanpa field name

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "name": [
  "This field is required."
]}
```

PROJ-NEG-004 – Lihat detail proyek yang tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat detail proyek yang tidak ada

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint project detail dengan ID tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Not found."}
```

PROJ-NEG-005 – Update proyek tanpa akses

Deskripsi: Verifikasi sistem menangani skenario Update proyek tanpa akses

Langkah-langkah:

1. Login sebagai annotator

2. Coba update project

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action." }
```

Negative Case - Modul Manajemen Dokumen

DOC-NEG-001 – Lihat daftar dokumen tanpa autentikasi

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar dokumen tanpa autentikasi

Langkah-langkah:

1. Akses endpoint /api/v1/documents/ tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided." }
```

DOC-NEG-002 – Lihat daftar dokumen sebagai non-admin

Deskripsi: Verifikasi sistem menangani skenario Lihat daftar dokumen sebagai non-admin

Langkah-langkah:

1. Login sebagai annotator

2. Akses endpoint list documents

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "Anda tidak memiliki izin untuk melakukan tindakan ini." }
```

DOC-NEG-003 – Buat dokumen baru tanpa field wajib

Deskripsi: Verifikasi sistem menangani skenario Buat dokumen baru tanpa field wajib

Langkah-langkah:

1. Login sebagai admin
2. Kirim request create document tanpa title atau text atau project

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "title": [
  "This field is required."
],
  "text": [
    "This field is required."
  ],
  "project": [
    "This field is required."
  ]
}
```

DOC-NEG-004 – Buat dokumen baru dengan teks duplikat

Deskripsi: Verifikasi sistem menangani skenario Buat dokumen baru dengan teks duplikat

Langkah-langkah:

1. Login sebagai admin
2. Buat dokumen dengan teks yang sama dengan dokumen existing

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "error": "Dokumen dengan teks yang sama sudah ada. Gunakan
allow_duplicate=true untuk mengizinkan duplikasi.",
  "existing_document": ... }
```

DOC-NEG-005 – Buat dokumen baru sebagai non-admin

Deskripsi: Verifikasi sistem menangani skenario Buat dokumen baru sebagai non-admin

Langkah-langkah:

1. Login sebagai annotator
2. Coba create document

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "Anda tidak memiliki izin untuk melakukan tindakan ini."}
```

DOC-NEG-006 – Lihat detail dokumen yang tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat detail dokumen yang tidak ada

Langkah-langkah:

1. Login sebagai admin
2. Akses endpoint document detail dengan ID tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Not found."}
```

DOC-NEG-007 – Update dokumen sebagai non-admin

Deskripsi: Verifikasi sistem menangani skenario Update dokumen sebagai non-admin

Langkah-langkah:

1. Login sebagai annotator
2. Coba update document

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "Anda tidak memiliki izin untuk melakukan tindakan ini."}
```

Negative Case - Modul Dashboard**DASH-NEG-001 – Lihat ringkasan dashboard tanpa autentikasi**

Deskripsi: Verifikasi sistem menangani skenario Lihat ringkasan dashboard tanpa autentikasi

Langkah-langkah:

1. Akses endpoint /api/v1/dashboard/ tanpa token

Ekspektasi:

Response: 401 Unauthorized;

Body:

```
{ "detail": "Authentication credentials were not provided."} DASH-
```

NEG-002 – Lihat ringkasan dashboard sebagai non-admin

Deskripsi: Verifikasi sistem menangani skenario Lihat ringkasan dashboard sebagai non-admin

Langkah-langkah:

1. Login sebagai annotator/reviewer
2. Akses endpoint dashboard summary

Ekspektasi:

Response: 403 Forbidden;

Body:

```
{ "detail": "You do not have permission to perform this action."}
```

DASH-NEG-003 – Lihat ringkasan dashboard dengan project_id tidak valid

Deskripsi: Verifikasi sistem menangani skenario Lihat ringkasan dashboard dengan project_id tidak valid

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard dengan project_id=-1 atau non-integer

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "project_id harus berupa integer positif"}
```

DASH-NEG-004 – Lihat ringkasan dashboard dengan document_id tidak valid

Deskripsi: Verifikasi sistem menangani skenario Lihat ringkasan dashboard dengan document_id tidak valid

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard dengan document_id=-1 atau non-integer

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "document_id harus berupa integer positif"}
```

DASH-NEG-005 – Lihat ringkasan dashboard dengan project tidak ditemukan

Deskripsi: Verifikasi sistem menangani skenario Lihat ringkasan dashboard dengan project tidak ditemukan

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard dengan project_id yang tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Project tidak ditemukan"}
```

DASH-NEG-006 – Lihat dashboard annotator - user bukan annotator

Deskripsi: Verifikasi sistem menangani skenario Lihat dashboard annotator - user bukan

annotator

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard annotator untuk user bukan annotator

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "error": "User yang ditentukan bukan Annotator" }
```

DASH-NEG-007 – Lihat dashboard annotator - user tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat dashboard annotator - user tidak ada

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard annotator dengan user_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "Annotator tidak ditemukan" }
```

DASH-NEG-008 – Lihat dashboard reviewer - user tidak ada

Deskripsi: Verifikasi sistem menangani skenario Lihat dashboard reviewer - user tidak ada

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses dashboard reviewer dengan user_id tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "error": "Reviewer tidak ditemukan" }
```

DASH-NEG-009 – Hitung IAA tanpa parameter wajib

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA tanpa parameter wajib

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA tanpa annotator_id atau reviewer_id

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "annotator_id dan reviewer_id wajib diisi.",
  "error_code": "MISSING_PARAMETERS" }
```

DASH-NEG-010 – Hitung IAA dengan UUID tidak valid

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA dengan UUID tidak valid

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA dengan annotator_id/reviewer_id format non-UUID

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "annotator_id dan reviewer_id harus berupa UUID yang
valid.",
  "error_code": "INVALID_UUID_FORMAT" }
```

DASH-NEG-011 – Hitung IAA dengan user tidak ditemukan

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA dengan user tidak ditemukan

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA dengan UUID user tidak ada

Ekspektasi:

Response: 404 Not Found;

Body:

```
{ "detail": "Salah satu atau kedua user tidak ditemukan.",  
  "error_code": "USER_NOT_FOUND" }
```

DASH-NEG-012 – Hitung IAA dengan user bukan Annotator

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA dengan user bukan Annotator

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA dengan `annotator_id` yang bukan annotator

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "User pertama bukan Annotator.",  
  "error_code": "INVALID_ANNOTATOR_ROLE" }
```

DASH-NEG-013 – Hitung IAA dengan user bukan Reviewer

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA dengan user bukan Reviewer

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA dengan `reviewer_id` yang bukan reviewer

Ekspektasi:

Response: 400 Bad Request;

Body:

```
{ "detail": "User kedua bukan Reviewer.",  
  "error_code": "INVALID_REVIEWER_ROLE" }
```

DASH-NEG-014 – Hitung IAA tanpa data annotation/review

Deskripsi: Verifikasi sistem menangani skenario Hitung IAA tanpa data annotation/review

Langkah-langkah:

1. Login sebagai admin/kepala riset
2. Akses endpoint IAA untuk users tanpa data

Ekspektasi:

Response: 400 Bad Request;

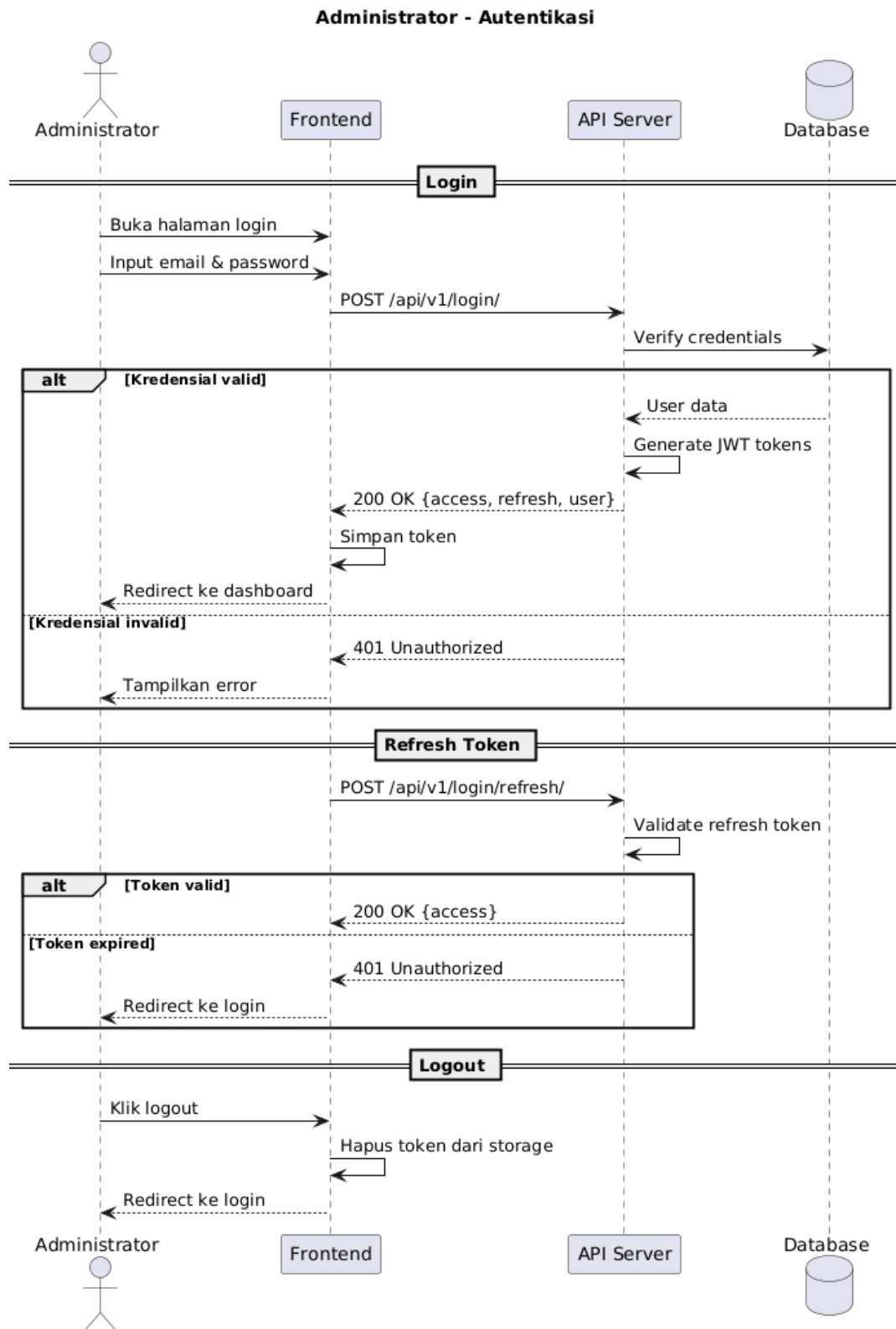
Body:

```
{ "detail": "Tidak ada data annotation atau review untuk  
perbandingan.",  
  "error_code": "NO_DATA_AVAILABLE"}
```

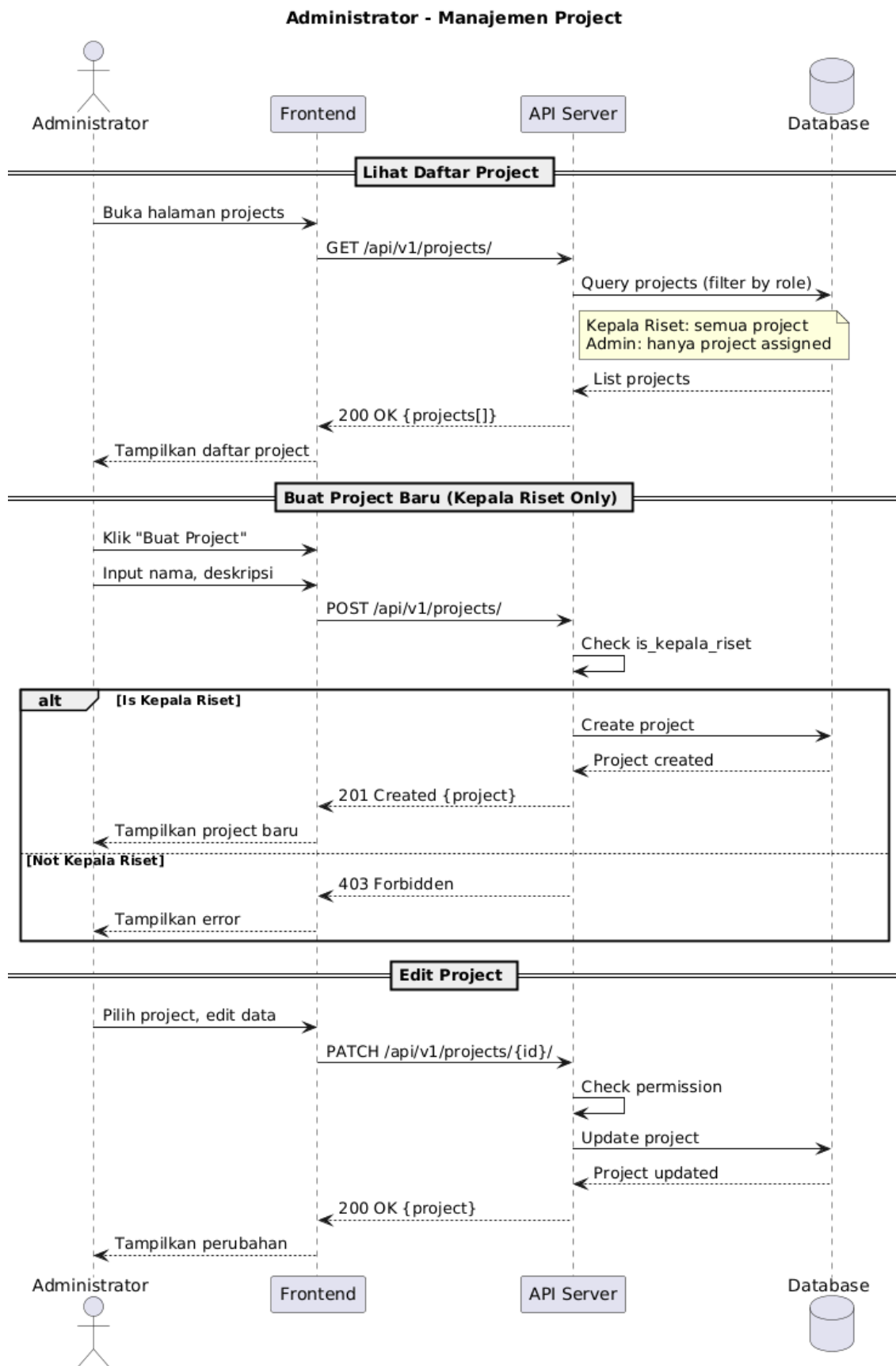
Lampiran 5: Konfigurasi JSON untuk Tes UAT pada Postman *Collection*

Konfigurasi *collection* Postman untuk melakukan UAT yang telah disimpan dalam format *file* JSON dapat diakses pada tautan berikut https://drive.google.com/file/d/1IEZj-XvGGwFNiLsLian4_eNKmFmiUHbj/view?usp=sharing.

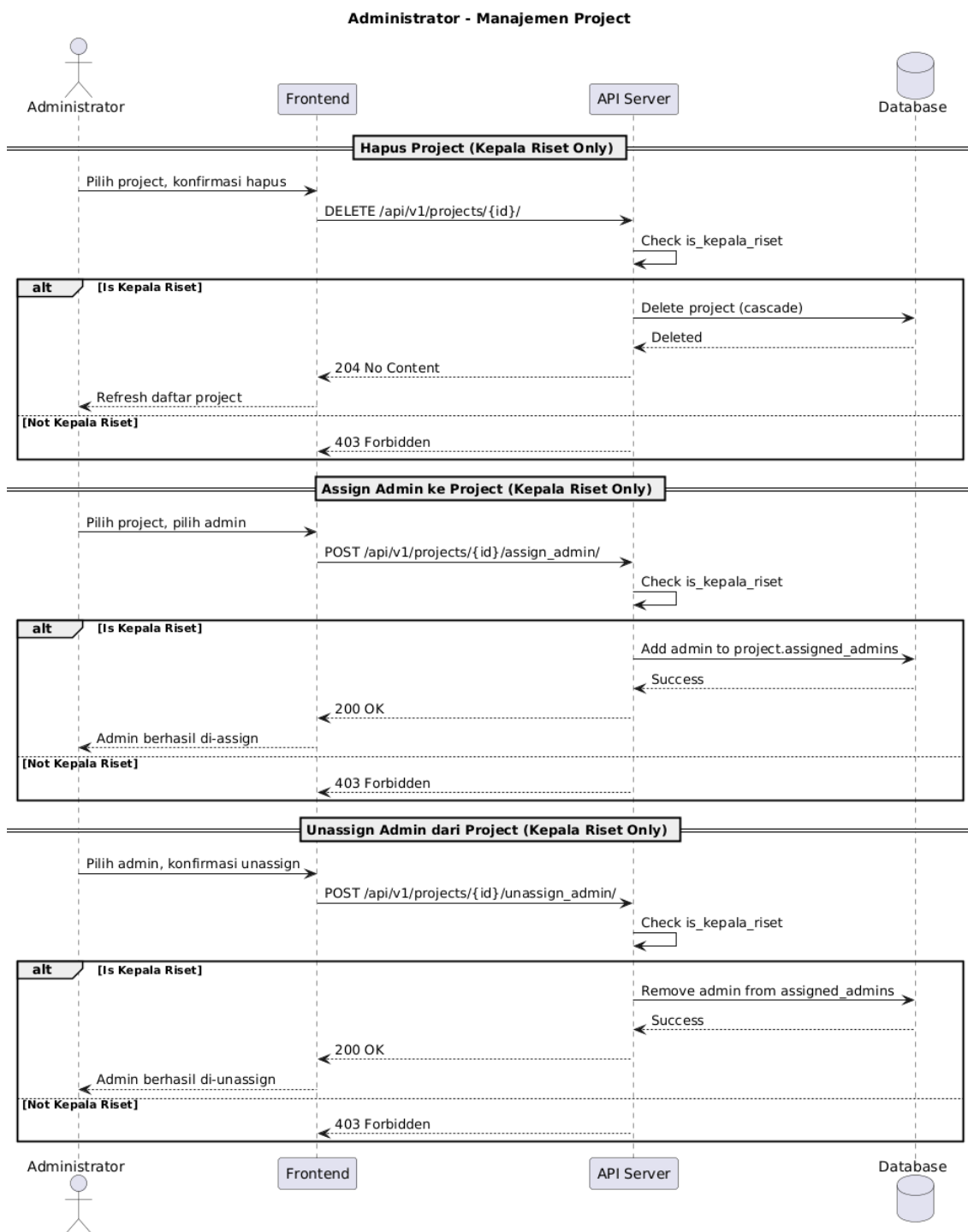
Lampiran 6: *User Flow*



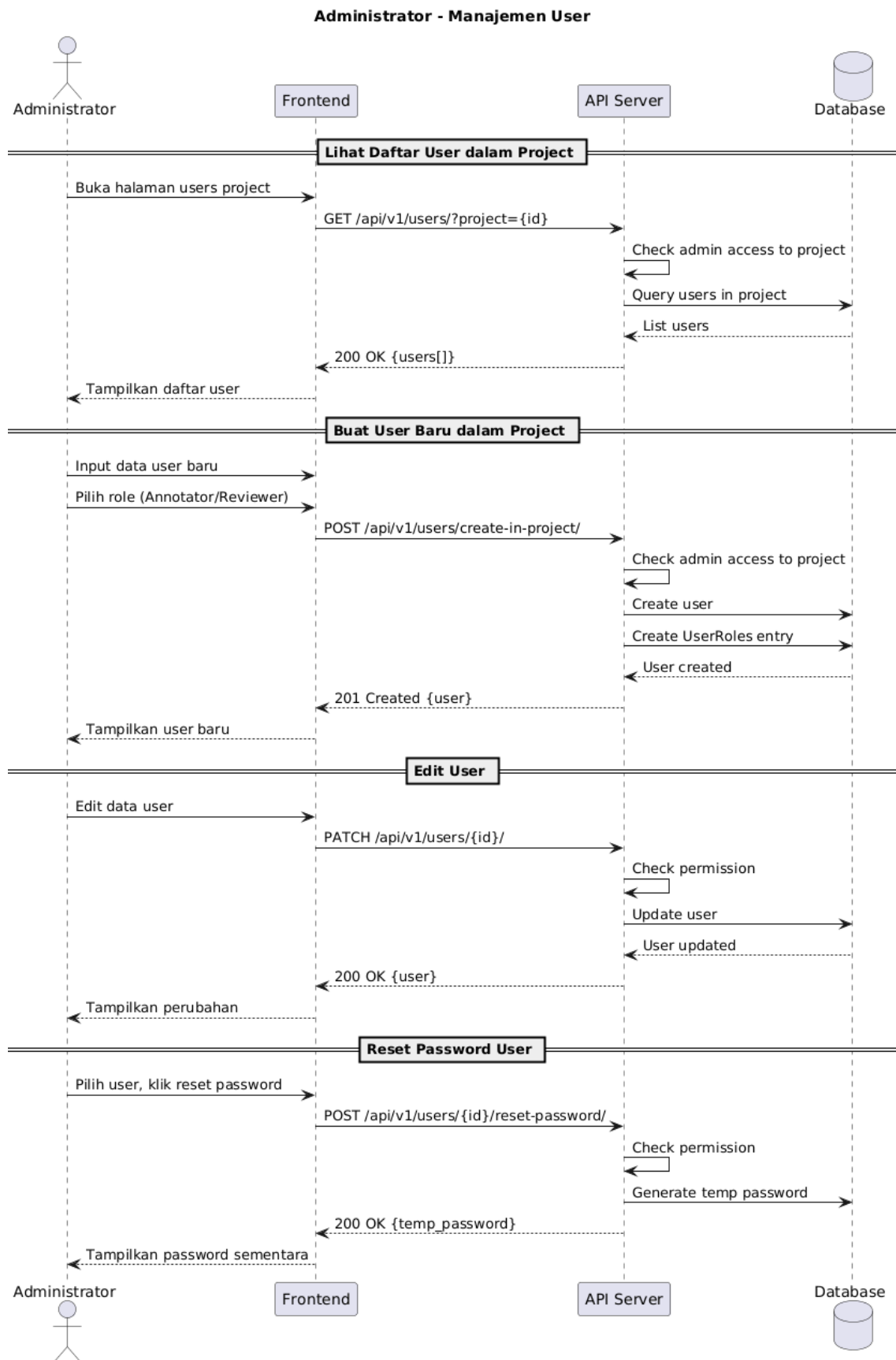
Gambar 1: Alur Autentikasi *Administrator*



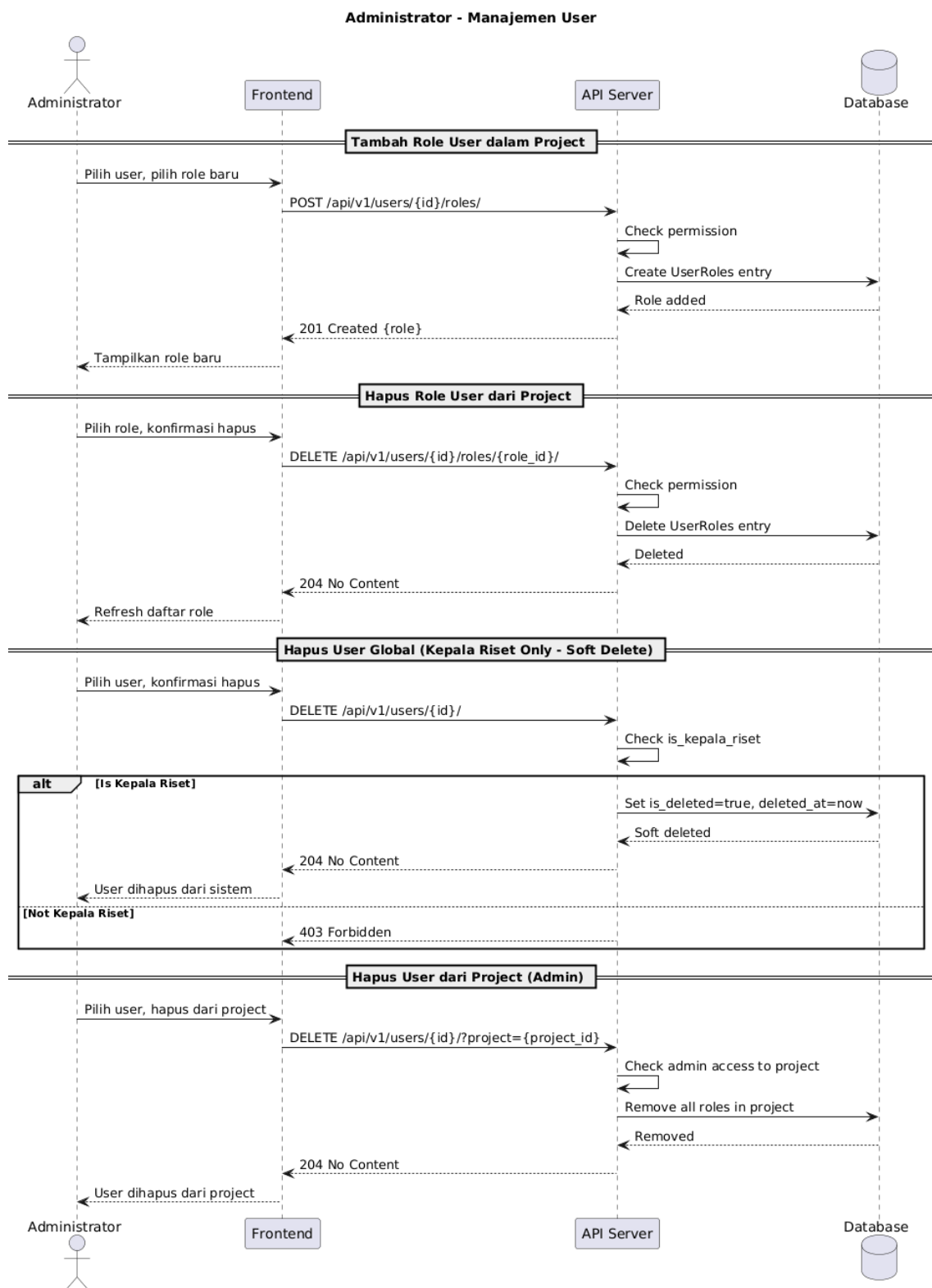
Gambar 2: Alur Manajemen Project Administrator Bagian 1



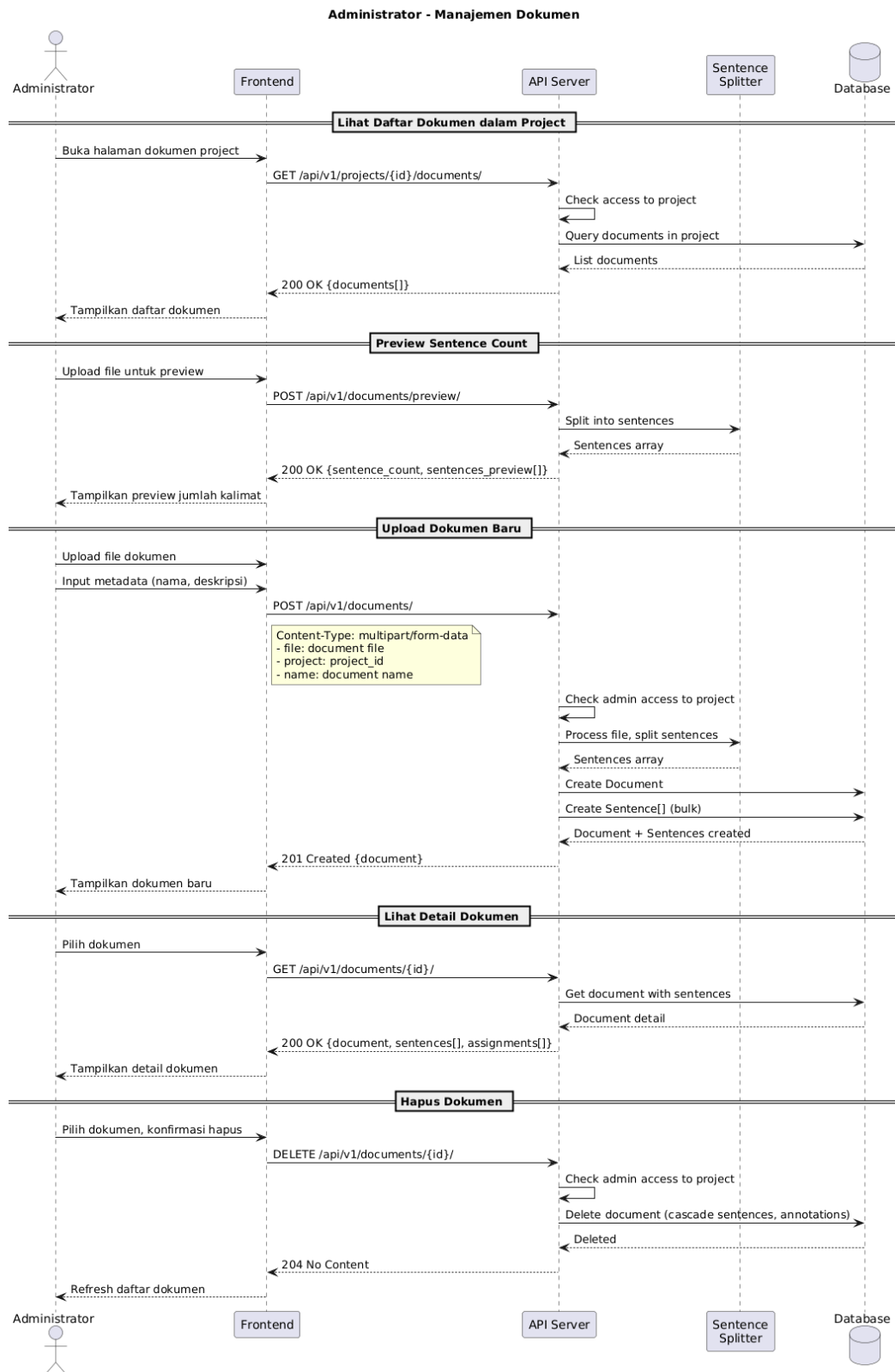
Gambar 3: Alur Manajemen *Project Administrator* Bagian 2



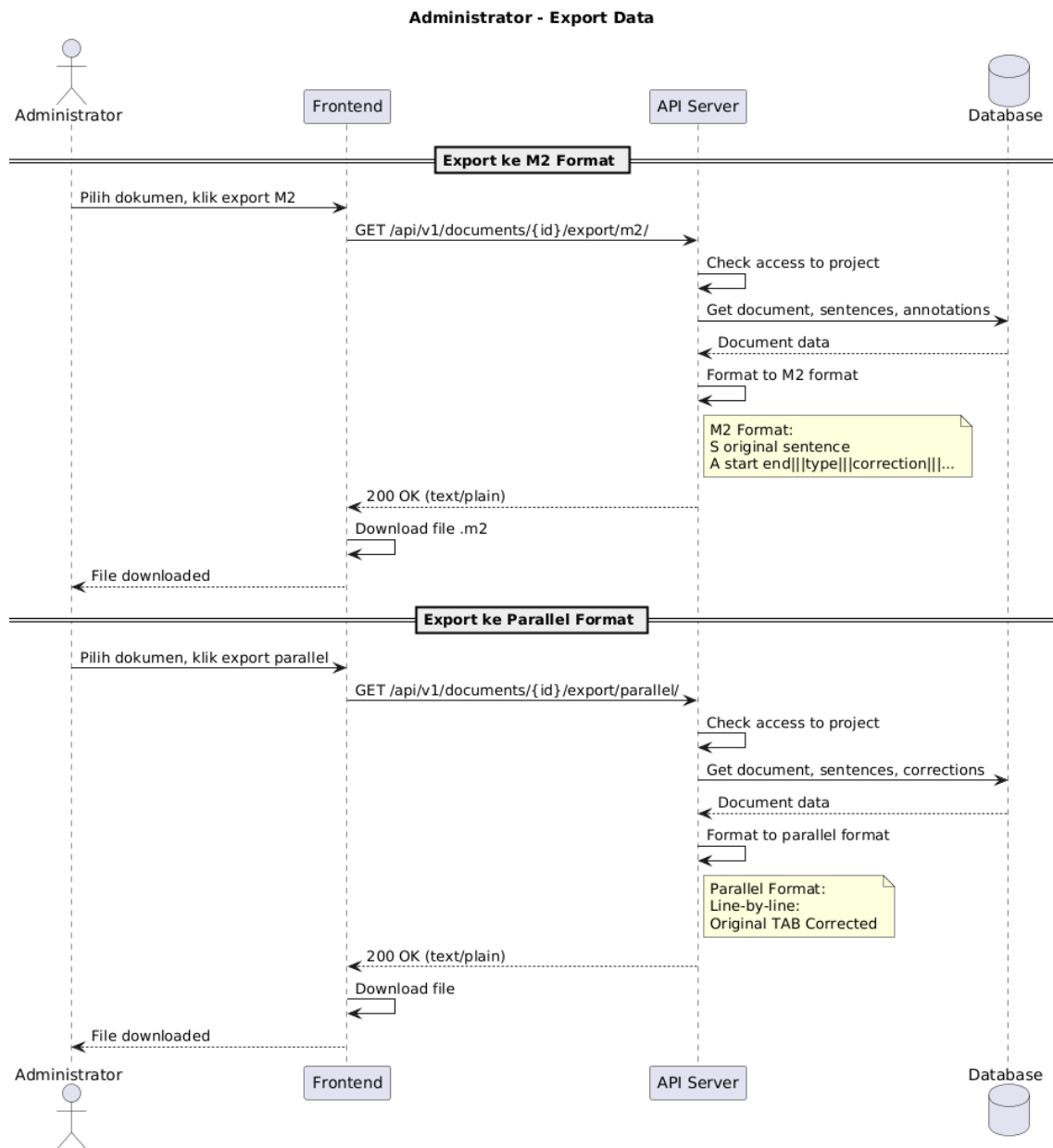
Gambar 4: Alur Manajemen User Administrator Bagian 1



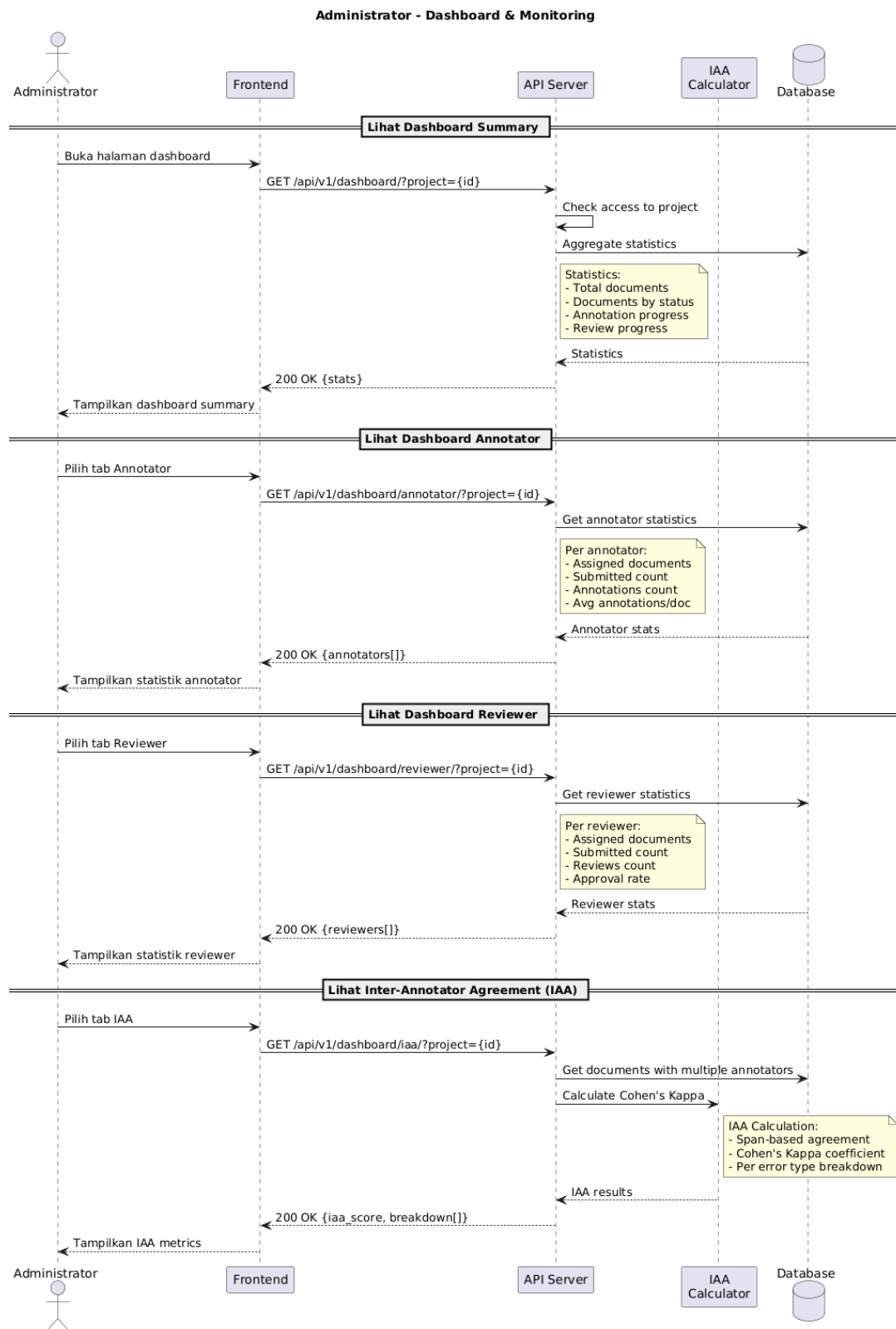
Gambar 5: Alur Manajemen User Administrator Bagian 2



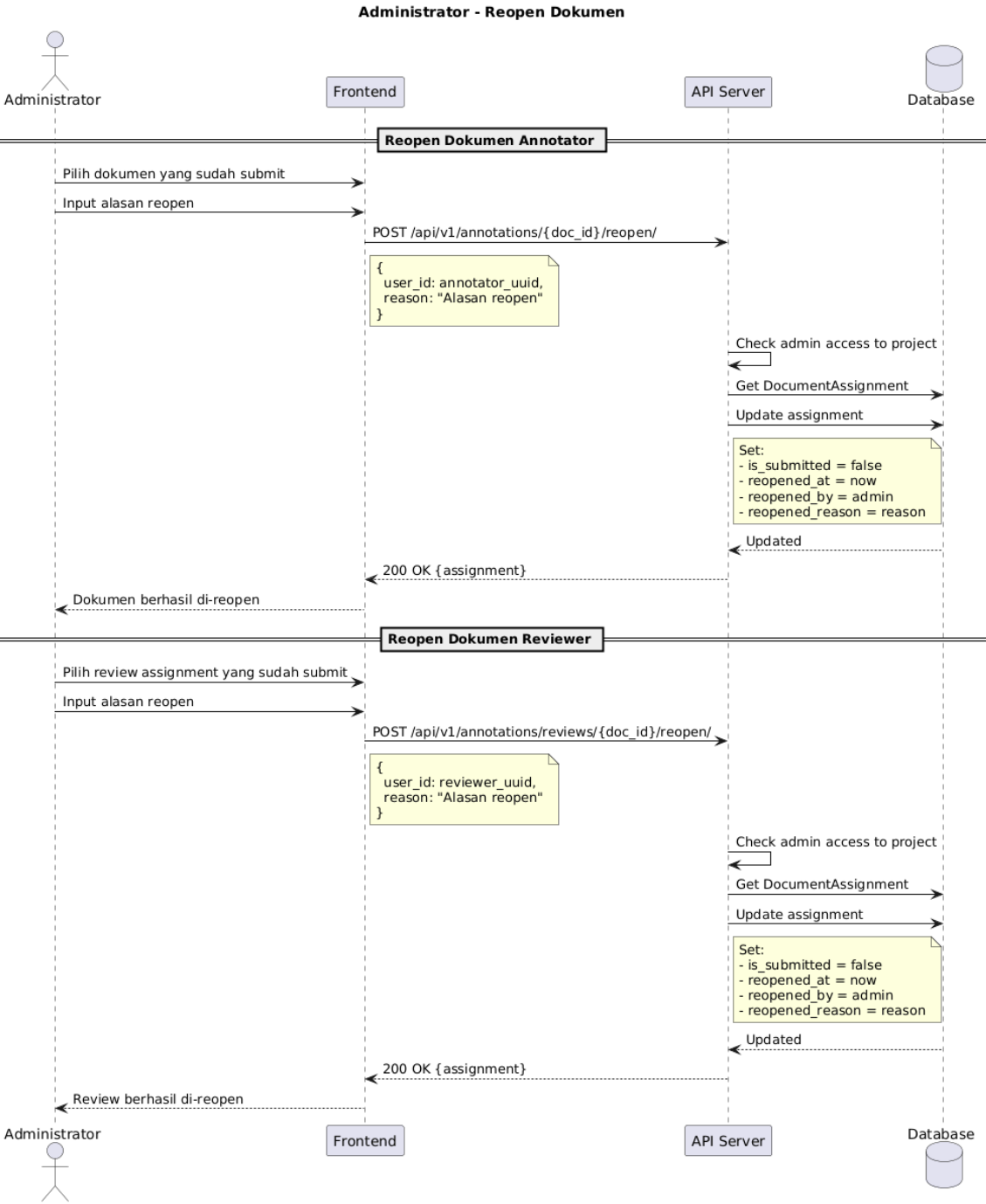
Gambar 6: Alur Manajemen Dokumen *Administrator*



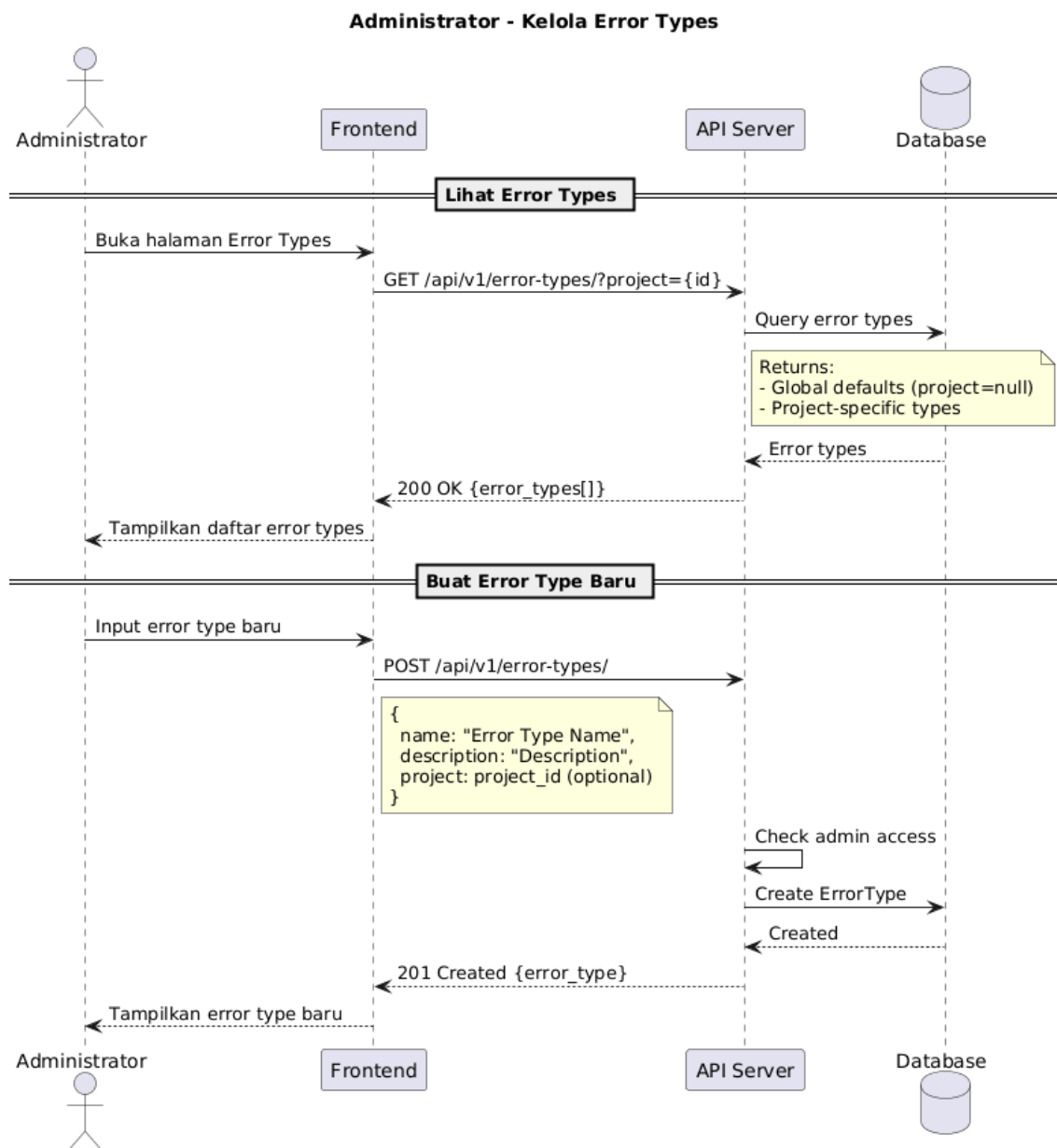
Gambar 7: Flow Export Data Administrator



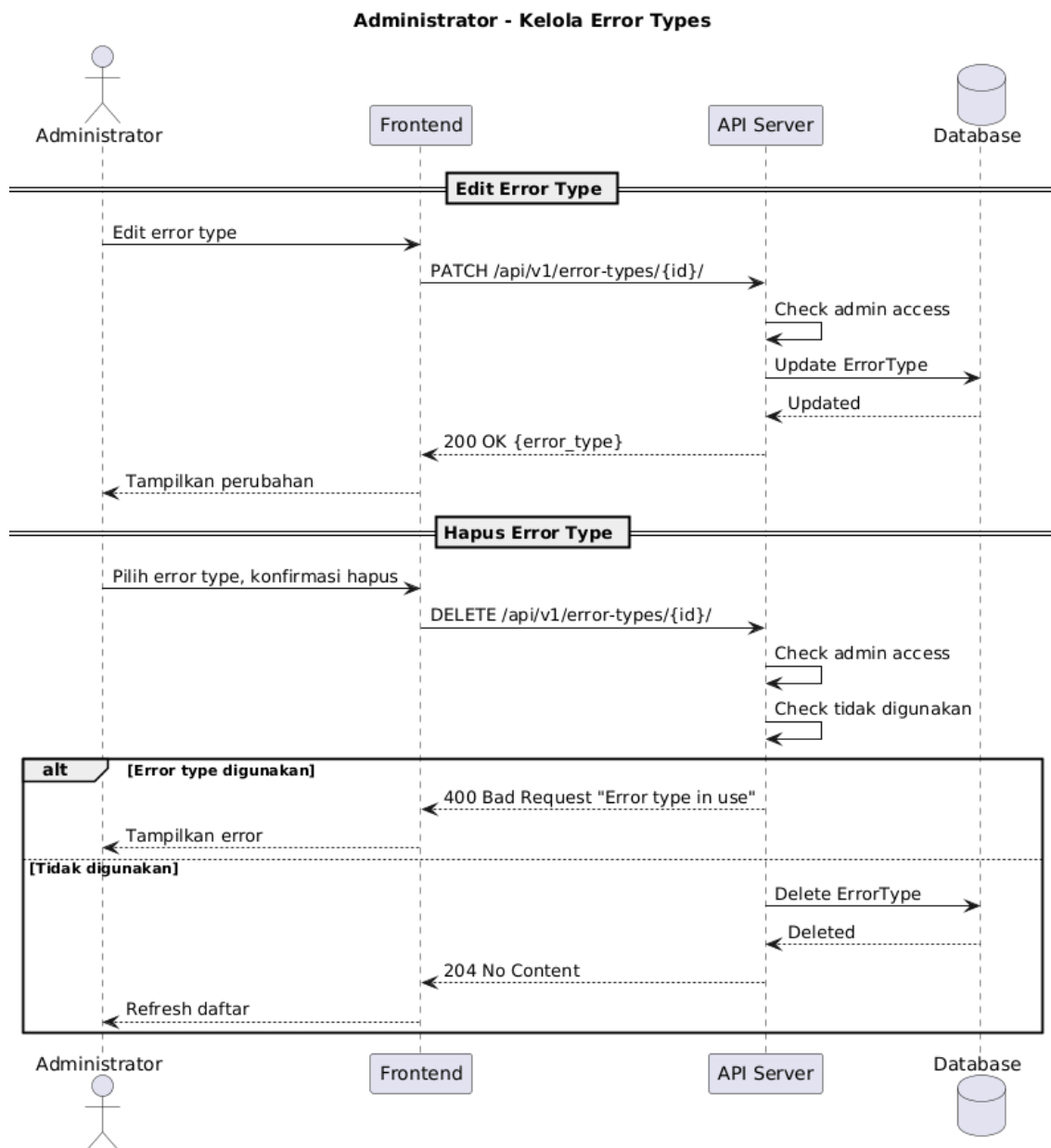
Gambar 8: Alur Dashboard Administrator



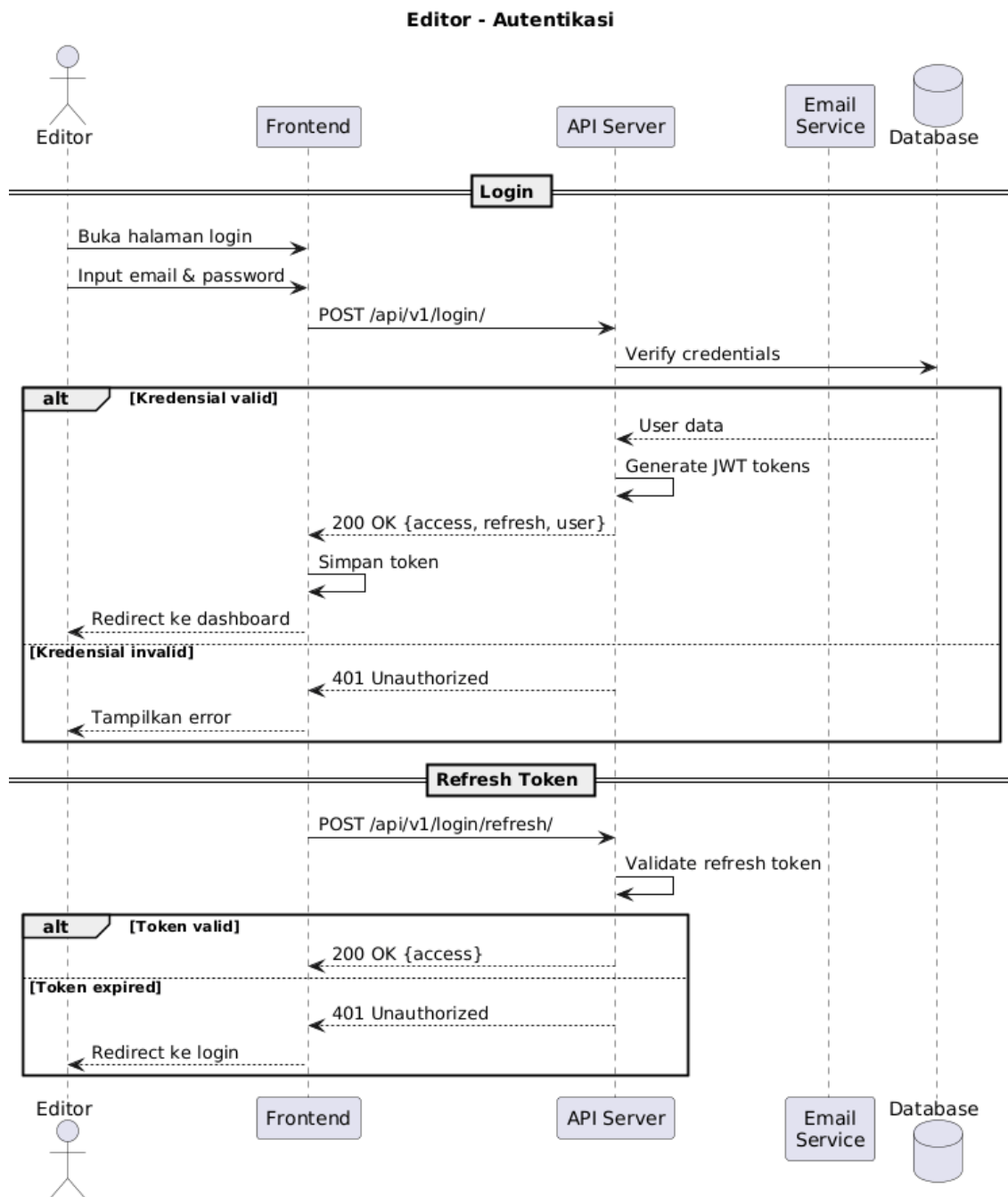
Gambar 9: Alur Buka Kembali Dokumen



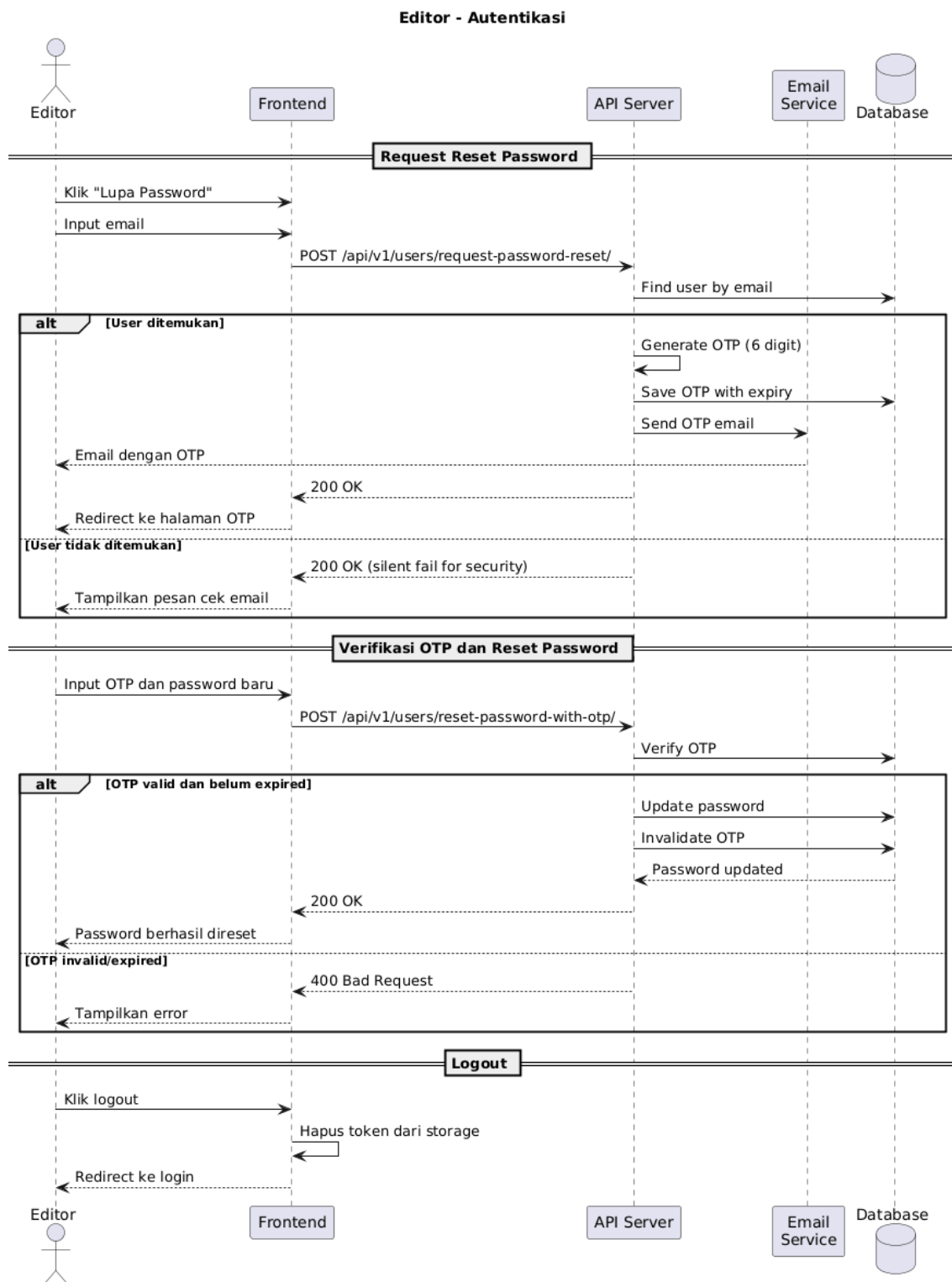
Gambar 10: Alur Kelola Jenis *Error* Bagian 1

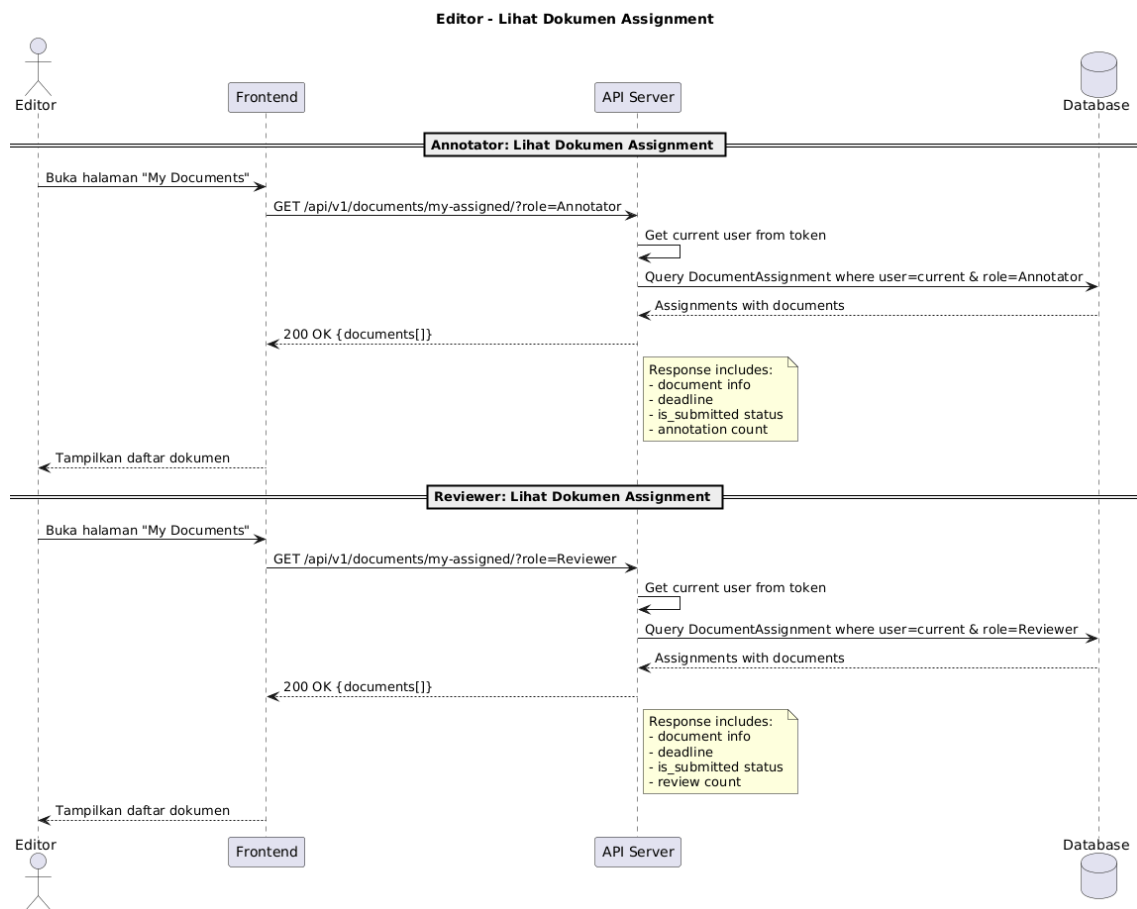


Gambar 11: Alur Kelola Jenis *Error* Bagian 2

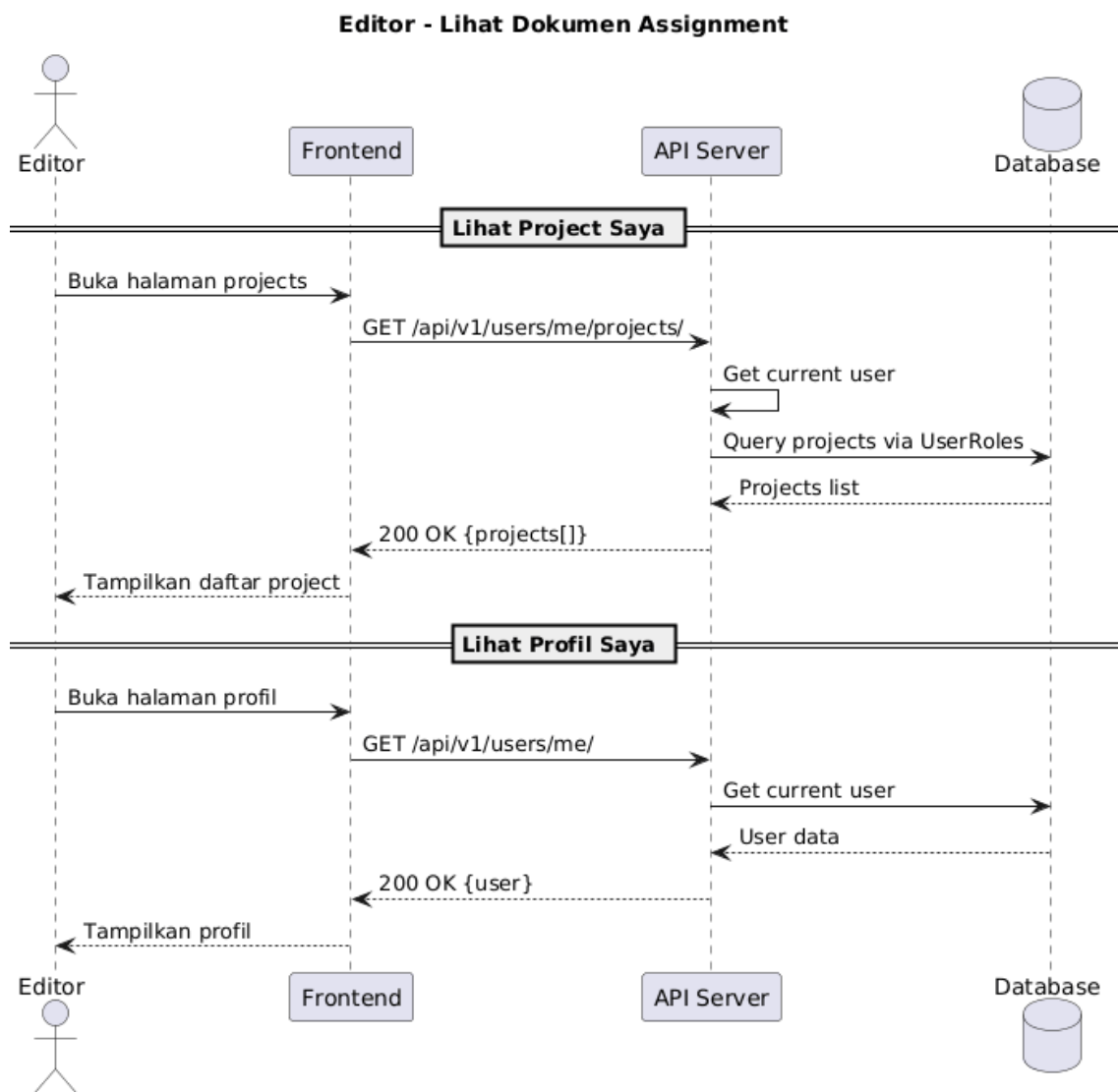


Gambar 12: Alur

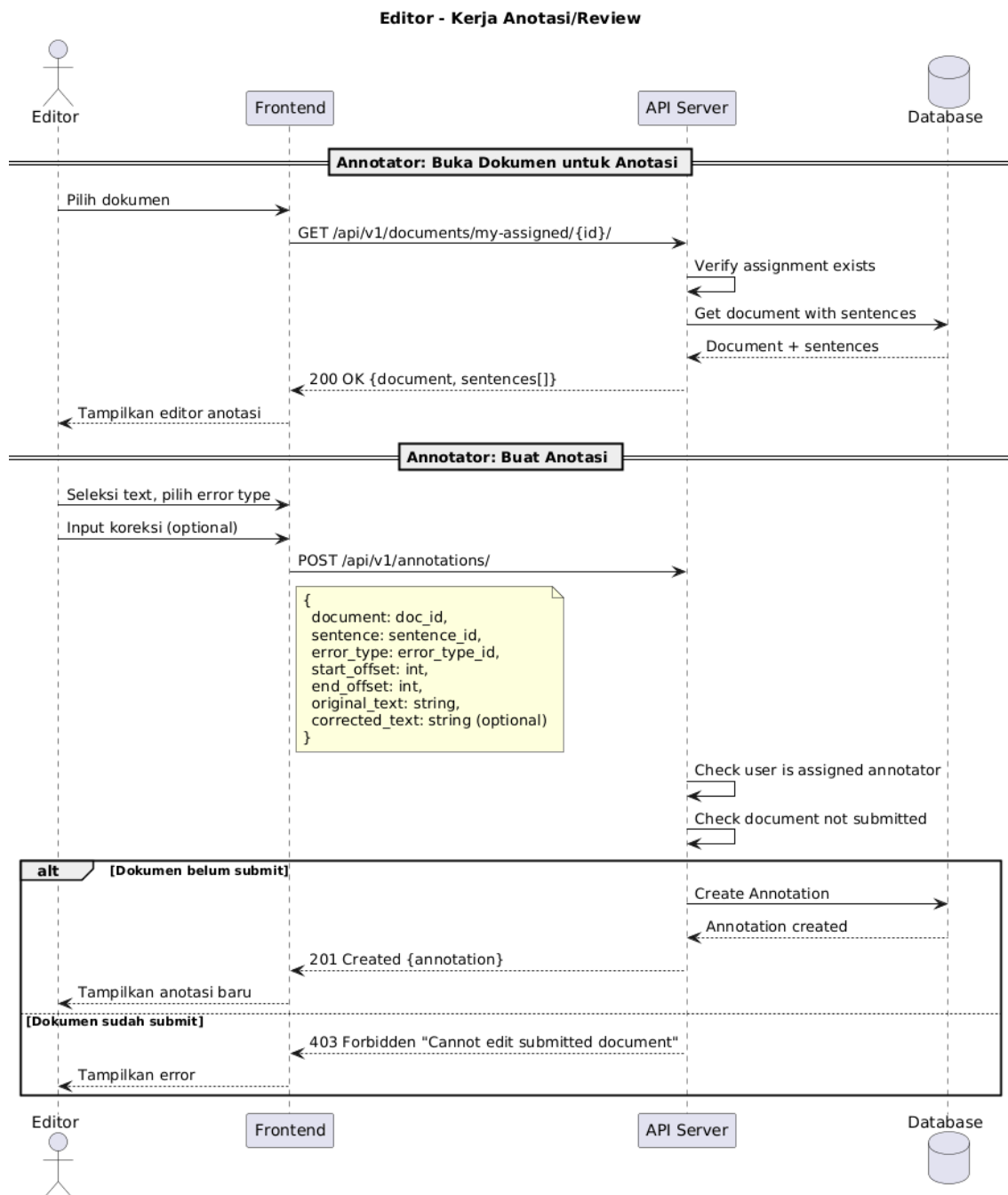
Gambar 13: Alur Autentikasi *Editor* Bagian 2



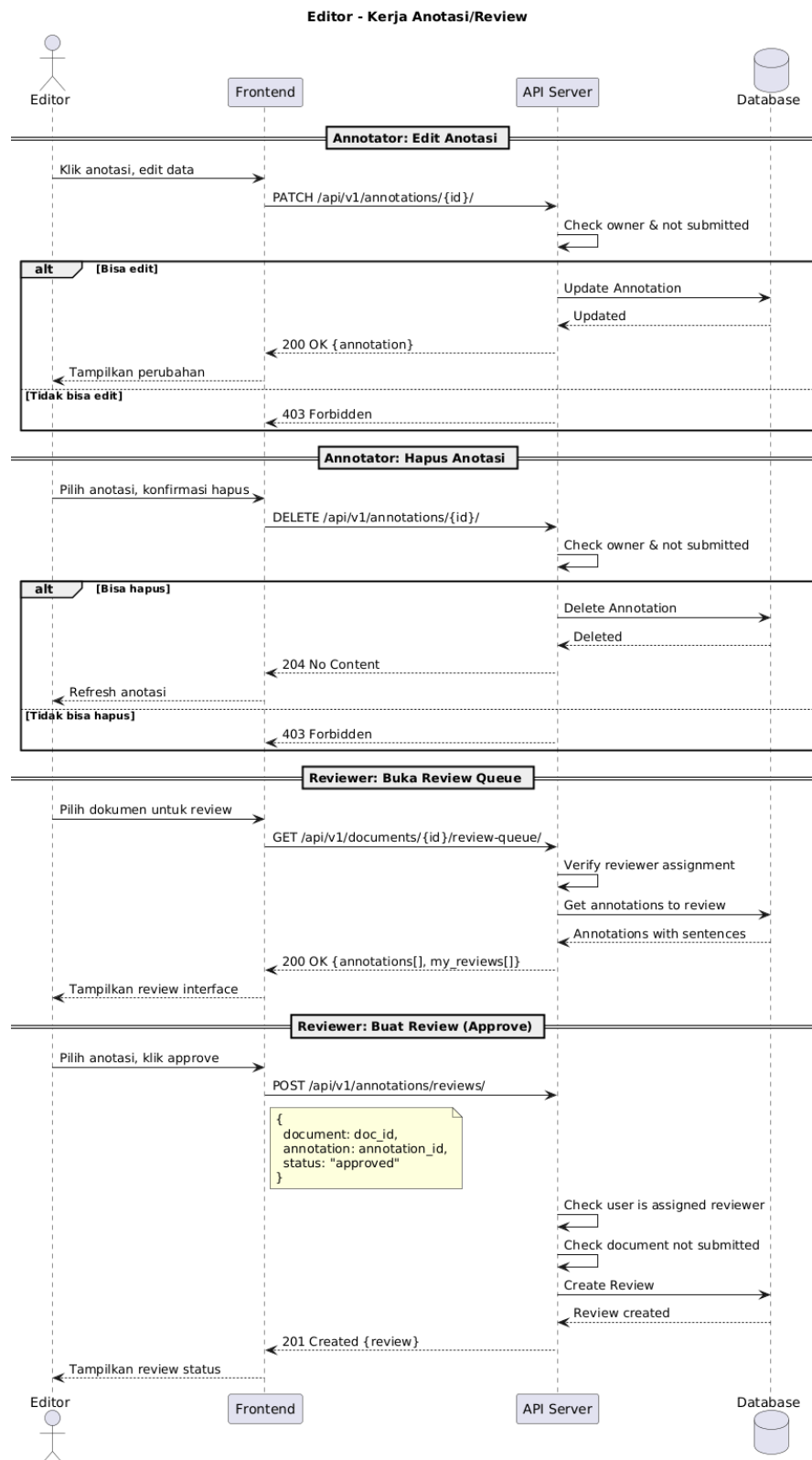
Gambar 14: Alur Penugasan Dokumen Bagian 1

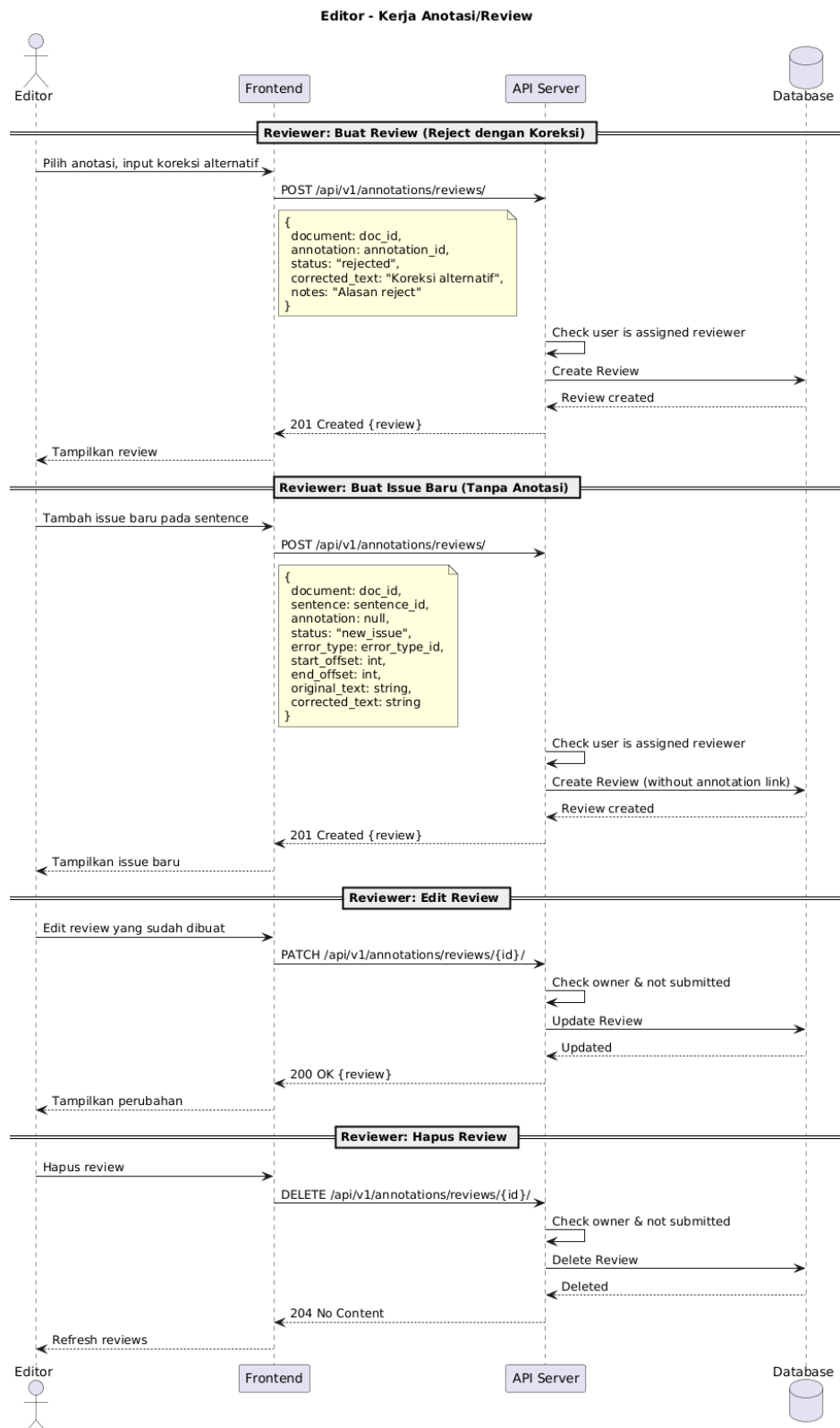


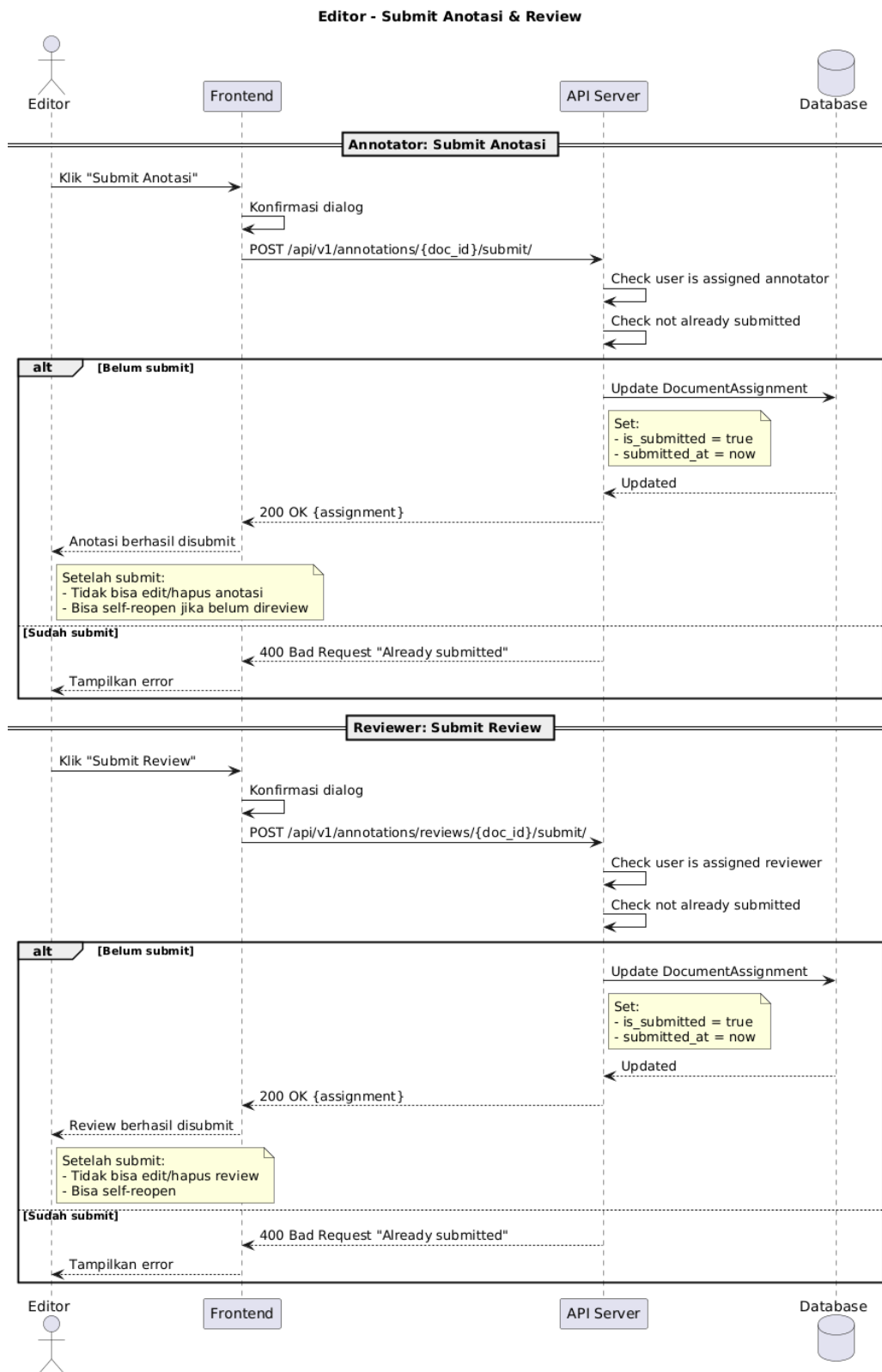
Gambar 15: Alur Penugasan Dokumen Bagian 2

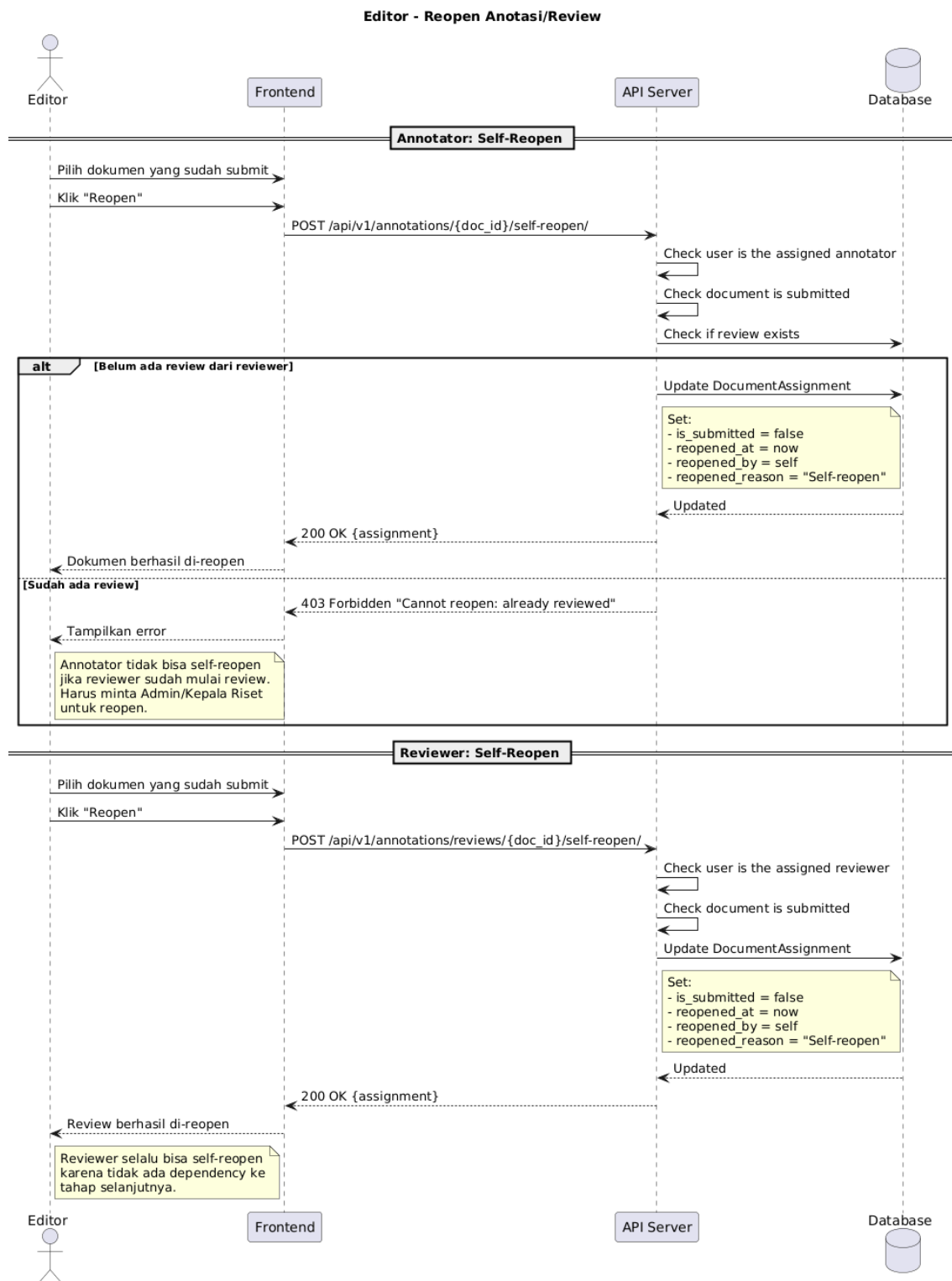


Gambar 16: Alur Review Anotasi Bagian 1

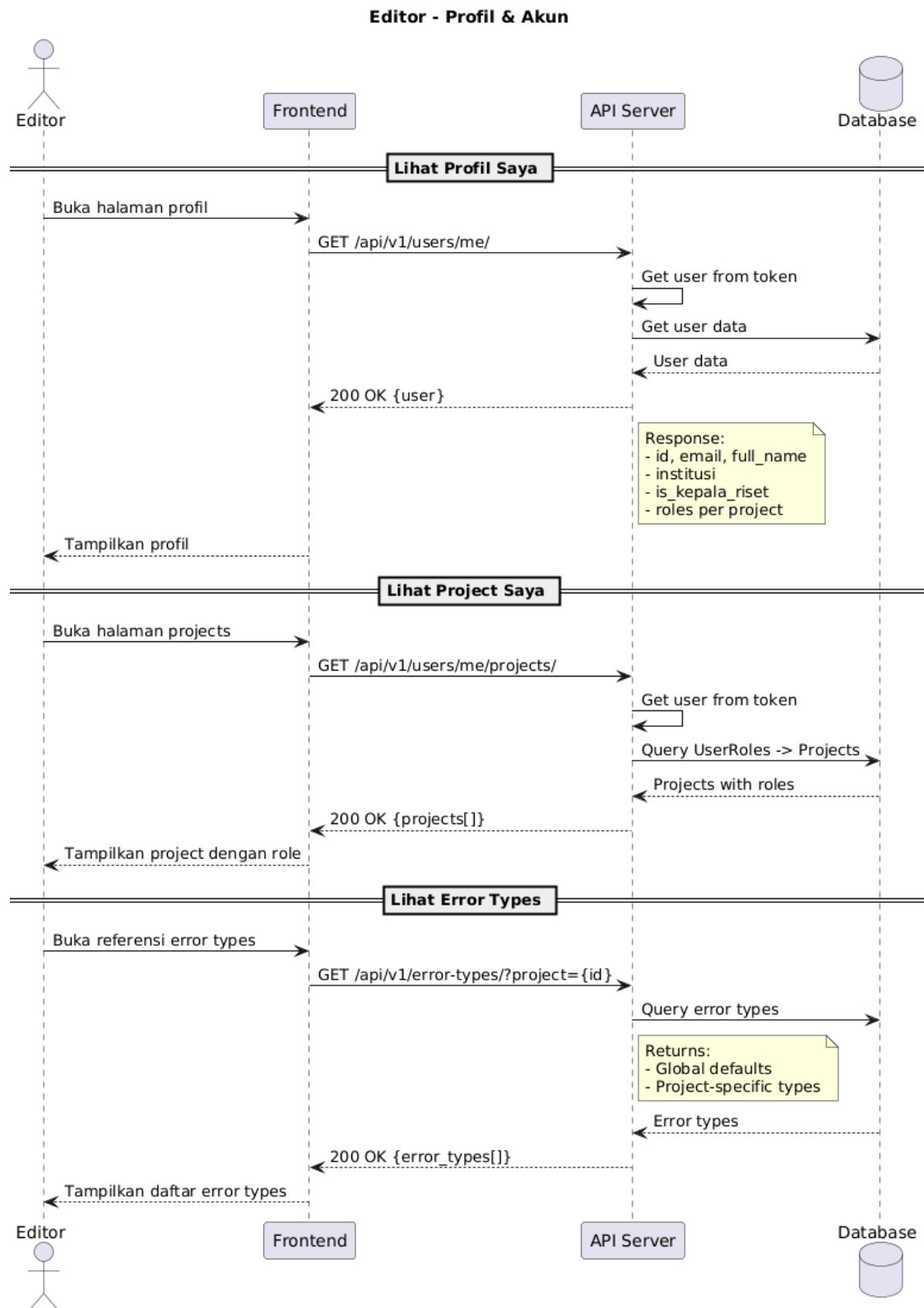
Gambar 17: Alur *Review* dan Anotasi Bagian 2

Gambar 18: Alur *Review* Anotasi Bagian 3

Gambar 19: Alur *Submit* Dokumen



Gambar 20: Alur Buka Kembali Dokumen



Gambar 21: Alur Ubah Profil